

Angle Protocol(多币种稳定币发行协议) 学习文档

文档说明

一、项目概述

1.1 Angle Protocol 是什么

Angle Protocol 是一个去中心化的多币种稳定币发行协议，由 Angle Labs Inc. 开发和维护。该协议的核心目标是提供最可靠、最具流动性的欧元和美元稳定币，同时为用户提供生息能力。

核心产品：

1. **USDA** (美元稳定币)

- 锚定美元 1:1
- 被称为"DeFi 超导体"
- 与 USDC 具有同等流动性
- 支持零滑点、零费用即时兑换

2. **EURA** (欧元稳定币)

- 锚定欧元 1:1
- 市场上最具流动性的欧元稳定币
- 支持链上外汇兑换

3. **stUSD** 和 **stEUR** (生息稳定币)

- USDA 和 EURA 的生息版本
- 自动复利，无需用户操作
- 收益来自 DeFi 借贷和代币化证券 (RWA)

关键特性：

- **超额抵押**：储备资产价值 > 流通稳定币价值
- **盈余缓冲**：额外的安全垫保护协议偿付能力
- **完全透明**：储备金实时可查，链上可验证
- **DAO 治理**：由 ANGLE 代币持有者社区治理
- **多链部署**：支持 Ethereum、Polygon、Arbitrum 等多条公链

1.2 核心价值主张

对于用户：

- 持有稳定币即可获得收益，无需质押或锁定
- 零滑点兑换 USDC 和 USDA，保证无限流动性
- 链上外汇兑换，无需中心化交易所
- 超额抵押提供更高的安全性

对于 DeFi 生态：

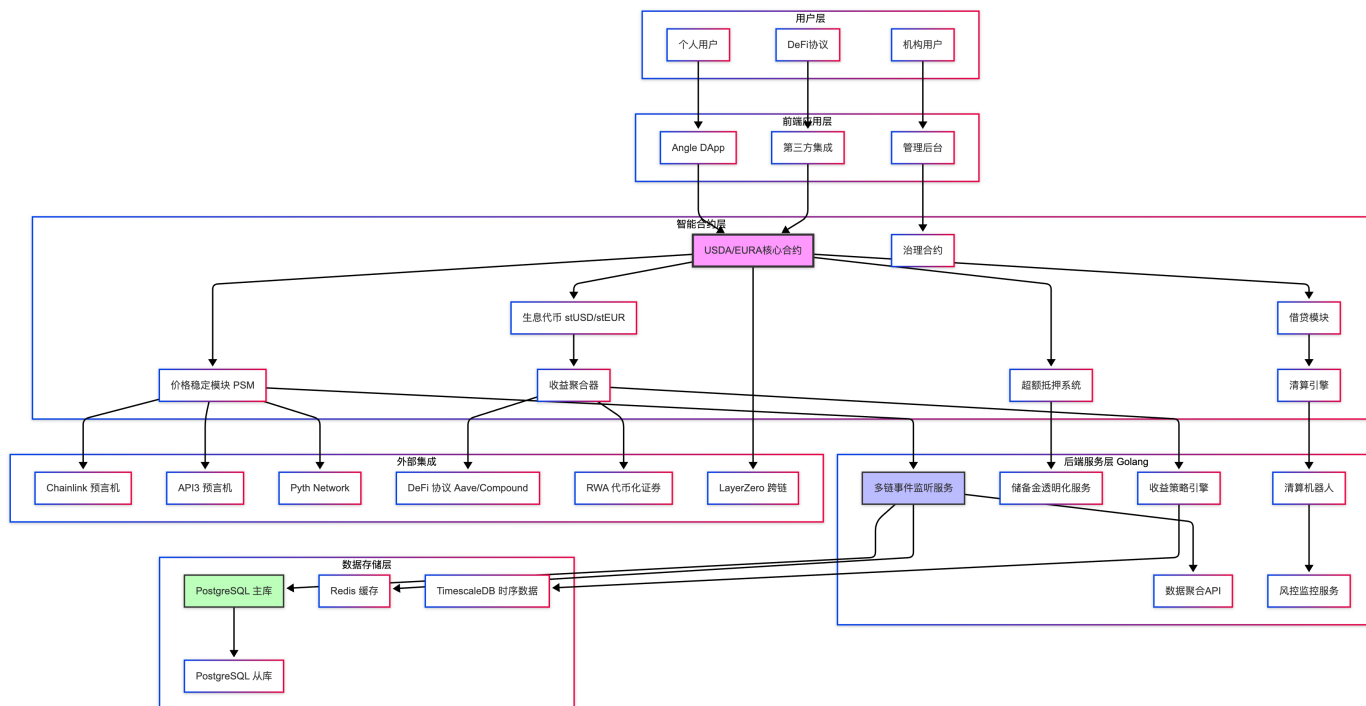
- 提供高流动性的欧元稳定币，填补市场空白
- 作为基础设施支持其他 DeFi 协议集成

- 通过 RWA 引入传统金融收益到链上

对于机构：

- 合规的稳定币解决方案
- 透明的储备金管理
- 多链支持降低部署成本

1.3 技术架构总览



架构层次说明：

1. 用户层：包括个人用户、机构用户和其他 DeFi 协议
2. 前端应用层：Web 应用、管理界面和第三方集成接口
3. 智能合约层：核心业务逻辑，运行在区块链上
4. 后端服务层：Golang 实现的链下服务，处理复杂计算和数据聚合
5. 数据存储层：持久化存储和缓存系统
6. 外部集成：预言机、DeFi 协议、RWA 资产和跨链桥

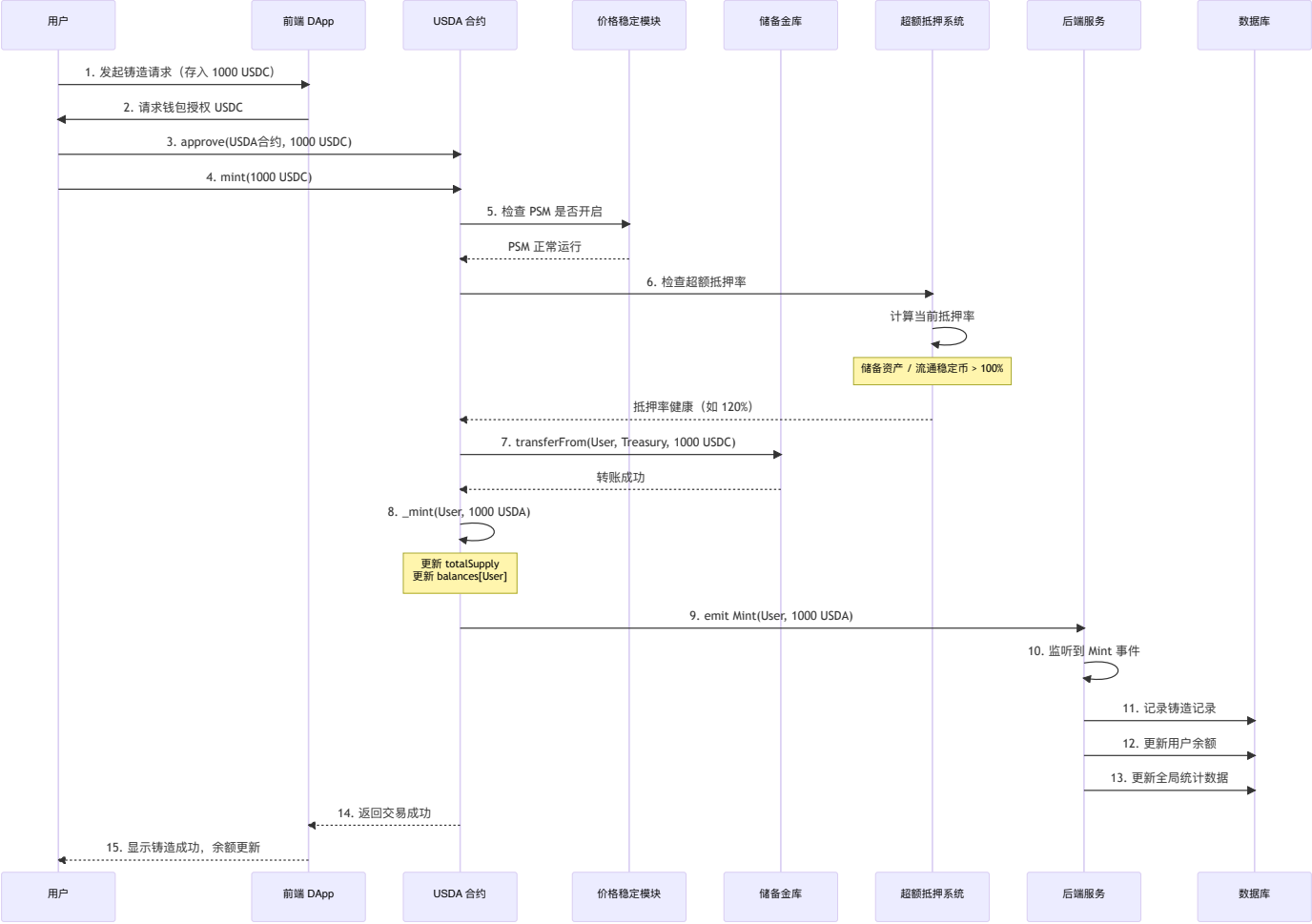
二、核心业务流程详解

2.1 USDA 铸造 (Mint) 流程

业务背景：

用户通过存入 USDC 来铸造 USDA，这是一个 1:1 的兑换过程。协议通过价格稳定模块 (PSM) 确保零滑点和零费用，同时维护超额抵押率保证系统安全。

详细流程：



流程关键点：

- 1. 授权检查：用户必须先授权 USDA 合约可以转移其 USDC
- 2. PSM 状态检查：确保价格稳定模块正常运行，未被暂停
- 3. 抵押率验证：计算铸造后的抵押率是否仍然健康
- 4. 原子性操作：USDC 转入和 USDA 铸造在同一交易中完成
- 5. 事件发射：链上事件触发后端服务更新数据库
- 6. 零费用：整个过程不收取任何手续费

抵押率计算公式：

$$\text{抵押率} = (\text{储备资产总价值}) / (\text{流通 USDA 总量}) \times 100\%$$
$$\text{储备资产总价值} = \text{USDC储备} + \text{DeFi借贷资产} + \text{RWA资产价值}$$

Gas 优化要点：

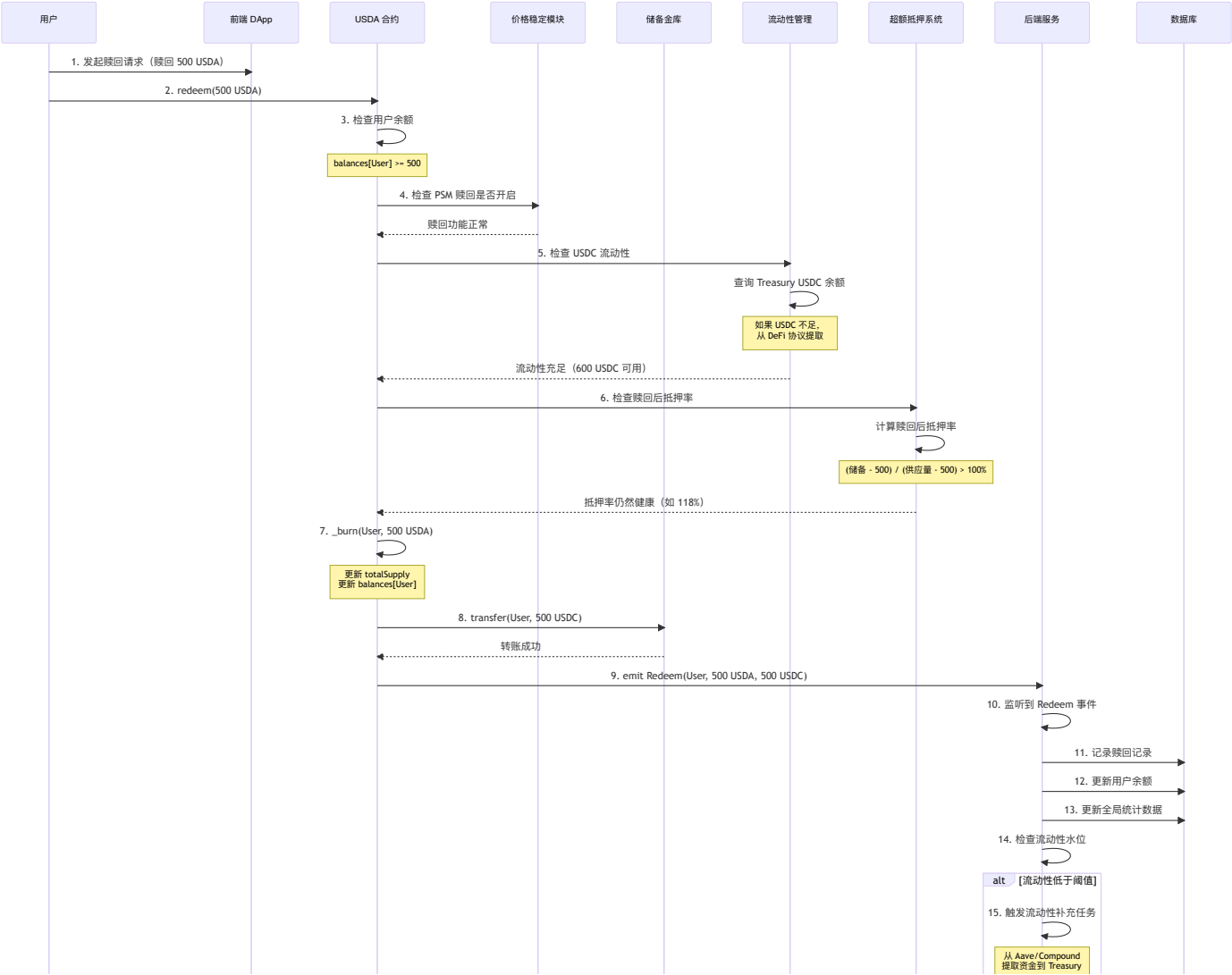
- 使用 `_mint` 内部函数避免重复检查
- 批量更新存储变量减少 SSTORE 操作
- 事件日志记录关键信息，避免链上存储历史数据

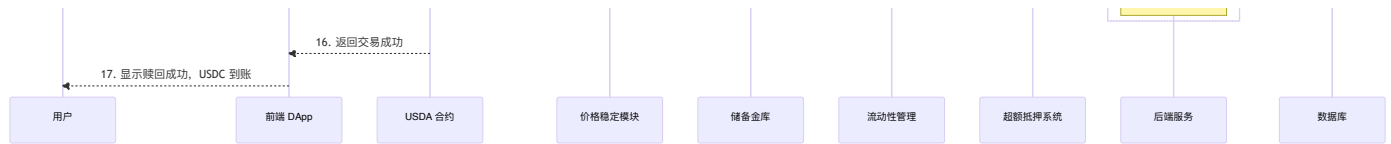
2.2 USDA 赎回（Redeem）流程

业务背景：

用户可以随时将 USDA 赎回为 USDC，这是铸造的逆过程。协议需要确保储备金充足，并在赎回后维持健康的抵押率。

详细流程：





流程关键点：

- 1. 余额检查：确保用户有足够的 USDA 余额
- 2. 流动性管理：如果 Treasury 中 USDC 不足，自动从 DeFi 协议提取
- 3. 抵押率保护：确保赎回后抵押率不会低于最低阈值
- 4. 先销毁后转账：遵循 CEI 模式（Checks-Effects-Interactions）
- 5. 流动性监控：后端服务实时监控并补充流动性
- 6. 零费用：赎回过程同样不收取手续费

流动性管理策略：

Treasury USDC 目标水位：流通 USDA 的 10-20%

当 Treasury USDC < 10% 时：

- 从 Aave 提取借出的 USDC
- 从 Compound 赎回 cUSDC
- 从 RWA 资产池提取流动性

当 Treasury USDC > 20% 时：

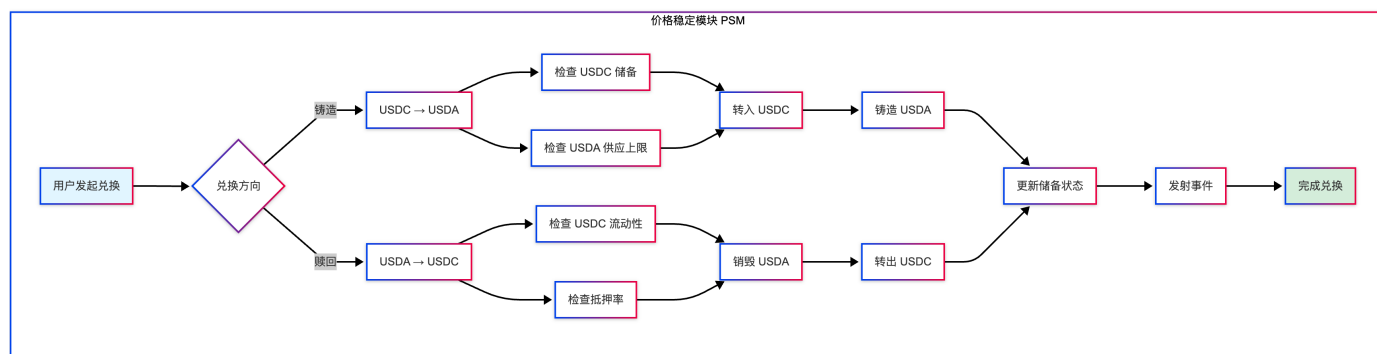
- 将多余 USDC 存入 Aave 赚取利息
- 购买 RWA 代币化证券
- 优化收益率

2.3 零滑点兑换机制（PSM）

业务背景：

价格稳定模块（PSM）是 Angle Protocol 的核心创新，它确保 USDC 和 USDA 之间可以随时以 1:1 的比例兑换，没有滑点，没有费用。这种机制保证了 USDA 的无限流动性。

PSM 工作原理：



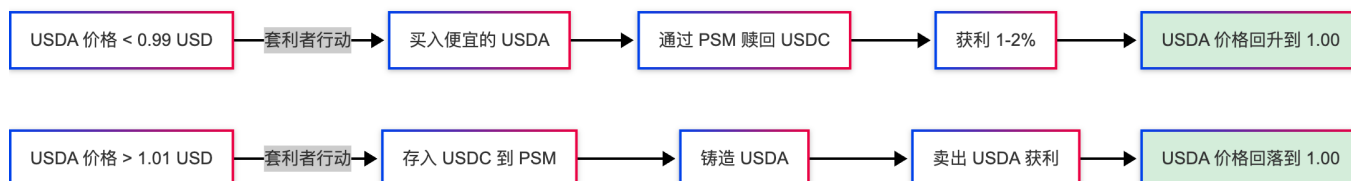
PSM 核心机制详解：

1. 恒定汇率保证

PSM 通过以下机制确保 1:1 汇率：

- **USDC 储备池**：协议维护一个 USDC 储备池，作为兑换缓冲
- **无限铸造能力**：只要有 USDC 存入，就可以铸造等量 USDA
- **无限赎回能力**：只要储备金充足，就可以赎回等量 USDC
- **无价格发现**：不依赖 AMM 曲线，直接 1:1 兑换

2. 套利机制维护锚定



3. 盈余缓冲 (Surplus Buffer)

盈余缓冲是 PSM 的安全垫，计算公式：

盈余缓冲 = 储备资产总价值 - 流通 USDA 总量

例如：
储备资产 = 120,000,000 USD
流通 USDA = 100,000,000 USDA
盈余缓冲 = 20,000,000 USD (20% 超额抵押)

盈余缓冲的作用：

- 吸收 RWA 资产价值波动
- 覆盖 DeFi 协议坏账风险
- 应对极端市场条件
- 保护用户赎回权利

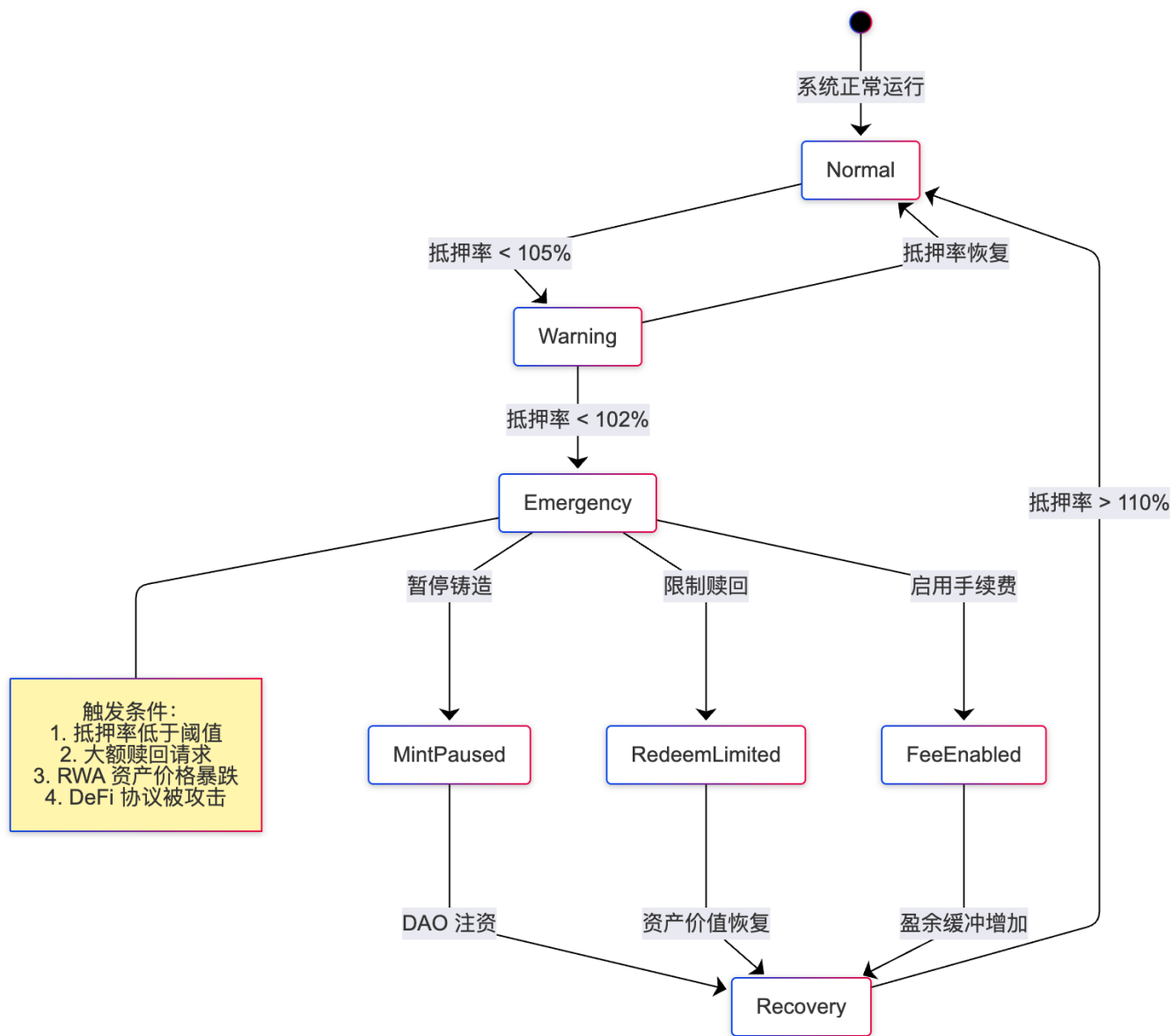
4. PSM 参数管理

PSM 有多个可调节参数，由 DAO 治理：

参数	说明	典型值	调整条件
mintFee	铸造手续费	0%	正常为 0，紧急时可调高
redeemFee	赎回手续费	0%	正常为 0，紧急时可调高
mintCap	铸造上限	无限制	可设置防止快速增长
redeemCap	赎回上限	无限制	可设置防止挤兑
minCollateralRatio	最低抵押率	100%	通常设为 105-110%
targetSurplus	目标盈余率	10-20%	根据风险调整

5. 紧急熔断机制

当系统检测到异常情况时，PSM 会自动触发保护措施：

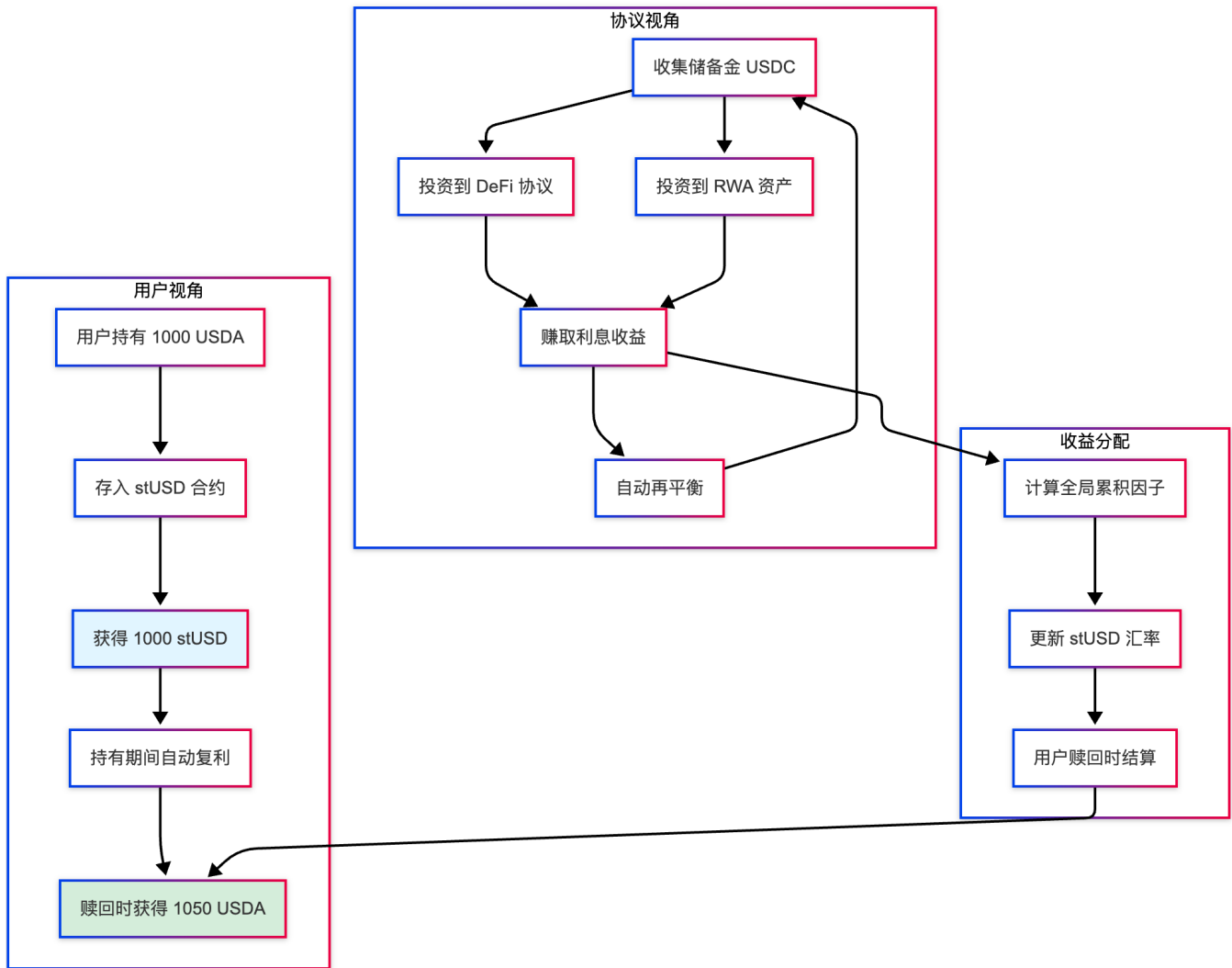


2.4 生息稳定币（stUSD/stEUR）机制

业务背景：

stUSD 和 stEUR 是 USDA 和 EURA 的生息版本。用户持有这些代币即可自动获得收益，无需质押或锁定。收益来自协议将储备资产投资到 DeFi 借贷协议和 RWA 代币化证券。

stUSD 工作原理：



stUSD 汇率计算：

stUSD 的价值会随着时间的增长，汇率计算公式：

$$\text{stUSD 汇率} = (\text{储备资产总价值} + \text{累积收益}) / \text{stUSD 总供应量}$$

示例：

初始状态：

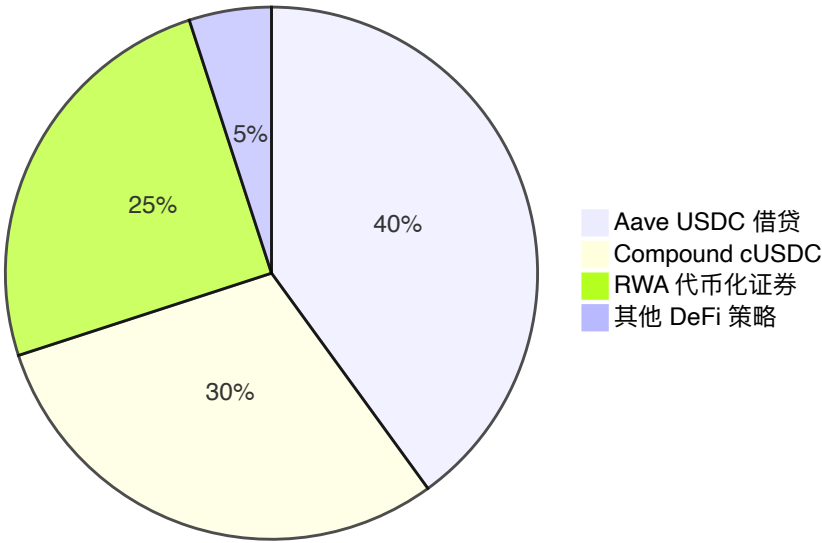
- 用户存入 1000 USDA
- 获得 1000 stUSD
- 汇率 = 1.0

30 天后：

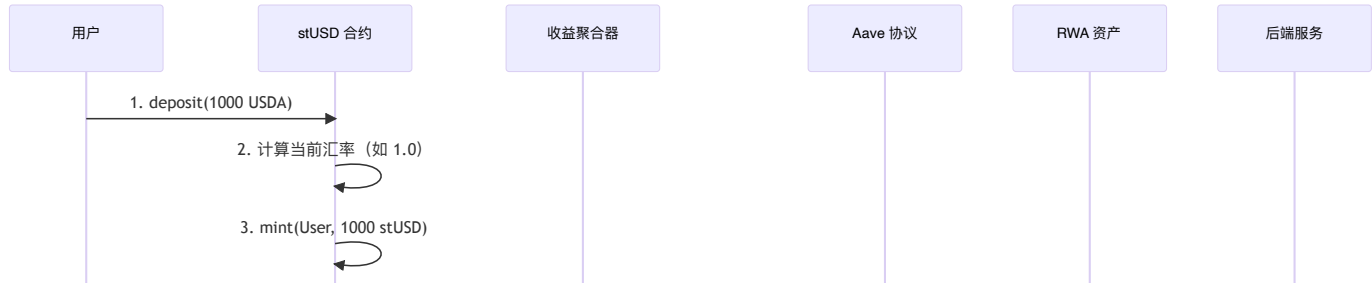
- 储备资产赚取 50 USDA 收益
- stUSD 总供应量仍为 1000
- 汇率 = $1050 / 1000 = 1.05$
- 用户赎回可得 $1000 \times 1.05 = 1050 \text{ USDA}$

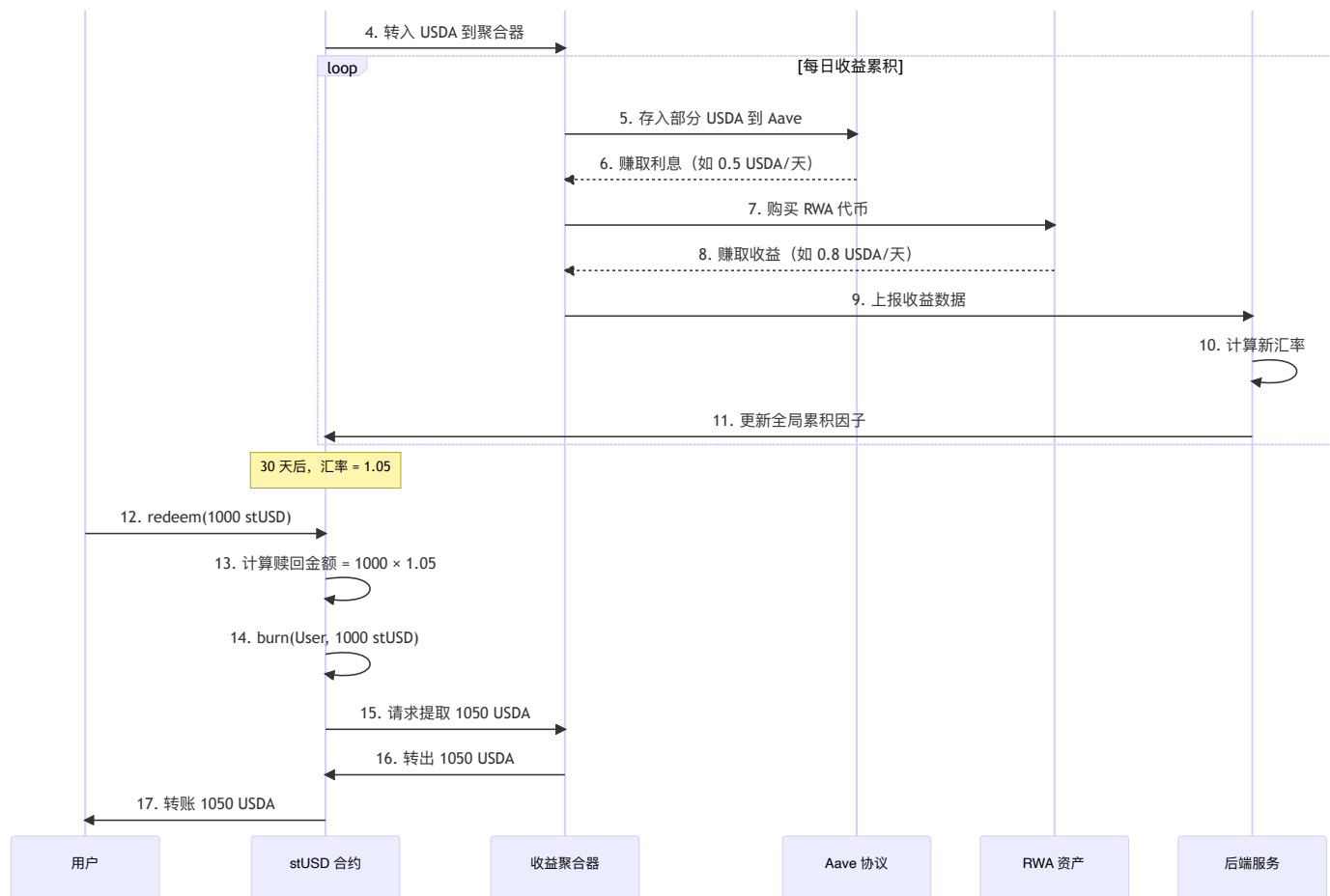
收益来源分配：

收益来源分配（示例）



收益分配流程：



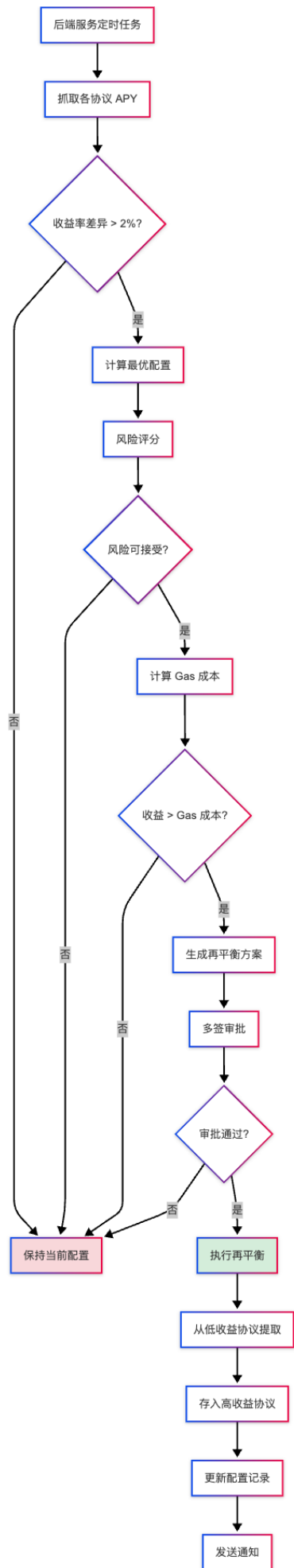


自动再平衡策略：

收益聚合器会根据以下因素动态调整资产配置：

1. **收益率比较**：实时监控各协议的 APY
2. **风险评估**：评估协议安全性和历史表现
3. **流动性需求**：保留足够流动性应对赎回
4. **Gas 成本**：考虑再平衡的 Gas 成本是否值得

再平衡决策流程：

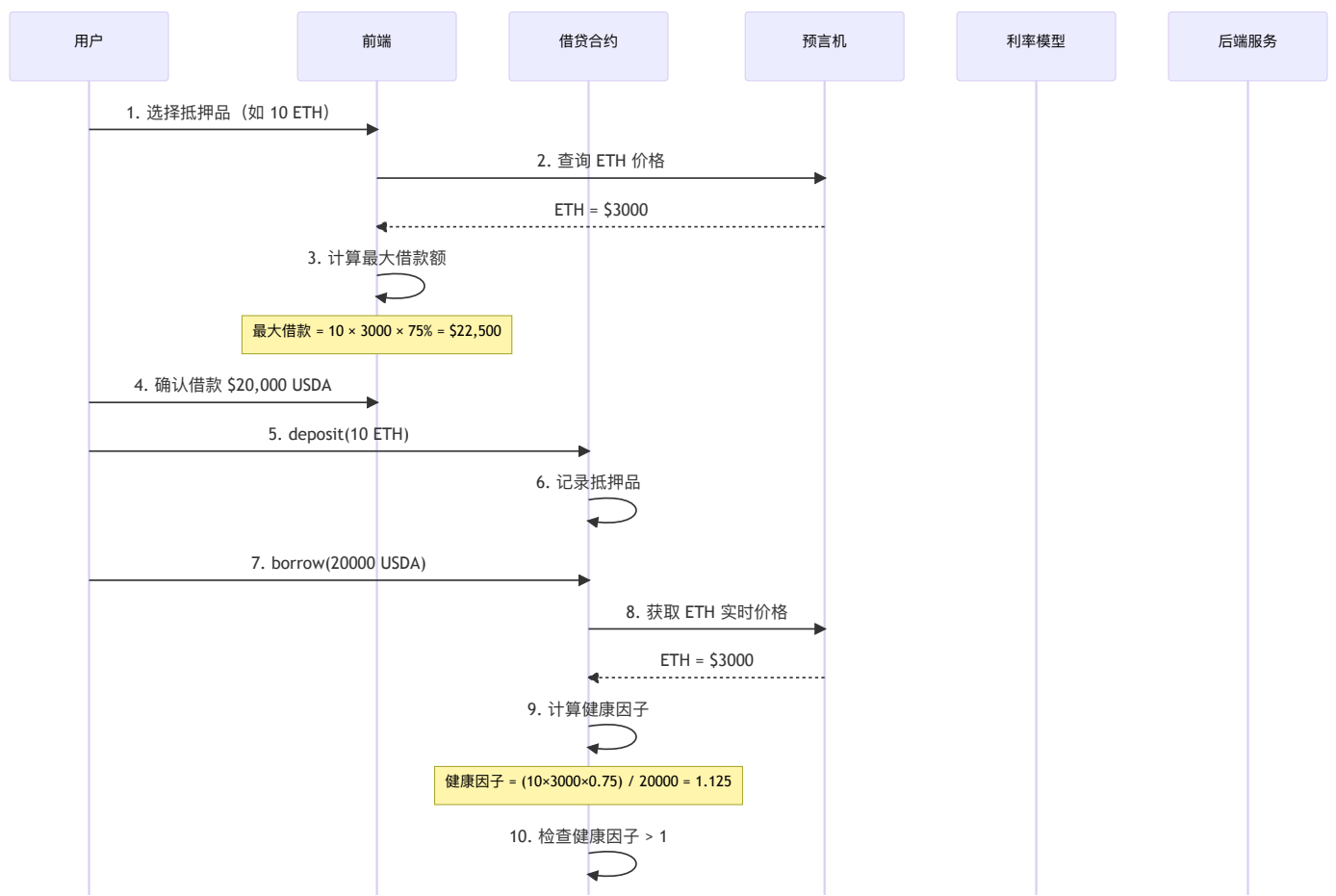


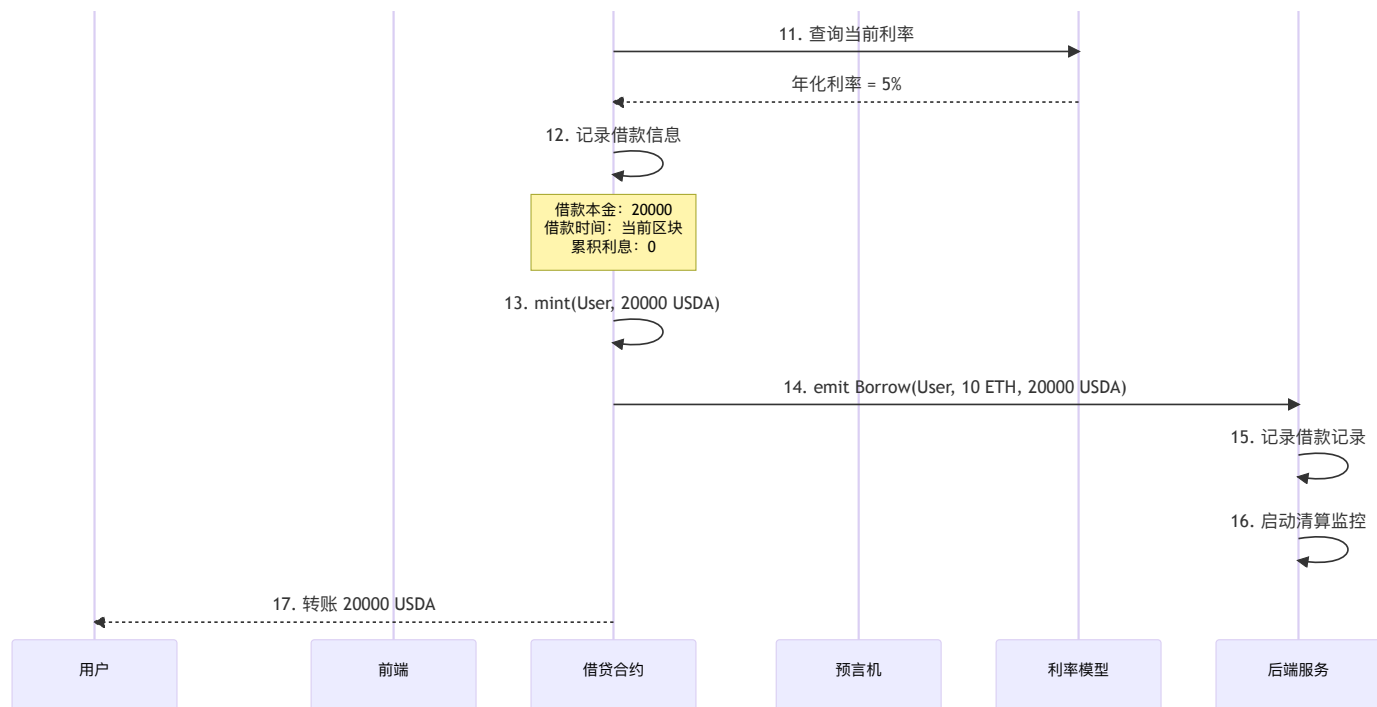
2.5 借贷与清算流程

业务背景：

Angle Protocol 不仅提供稳定币发行，还支持用户使用 USDC、wstETH 等资产作为抵押品借出 USDA/EURA。这为用户提供了杠杆能力，同时为协议创造额外收益。

借贷流程：





健康因子计算：

健康因子 = (抵押品价值 × 清算阈值) / 借款金额

清算阈值 (Liquidation Threshold) :

- USDC: 95% (低风险)
- wstETH: 80% (中风险)
- wBTC: 75% (中风险)
- 其他代币: 60-70% (高风险)

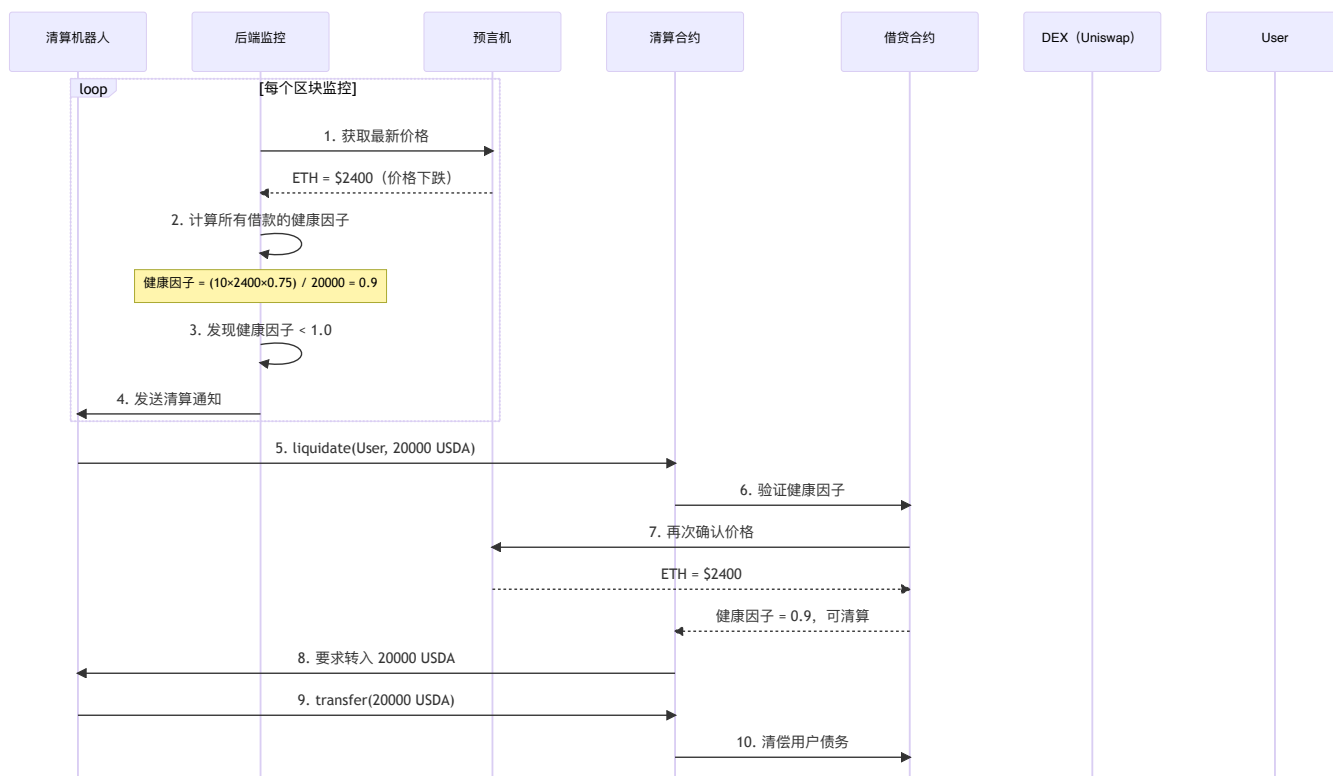
示例:

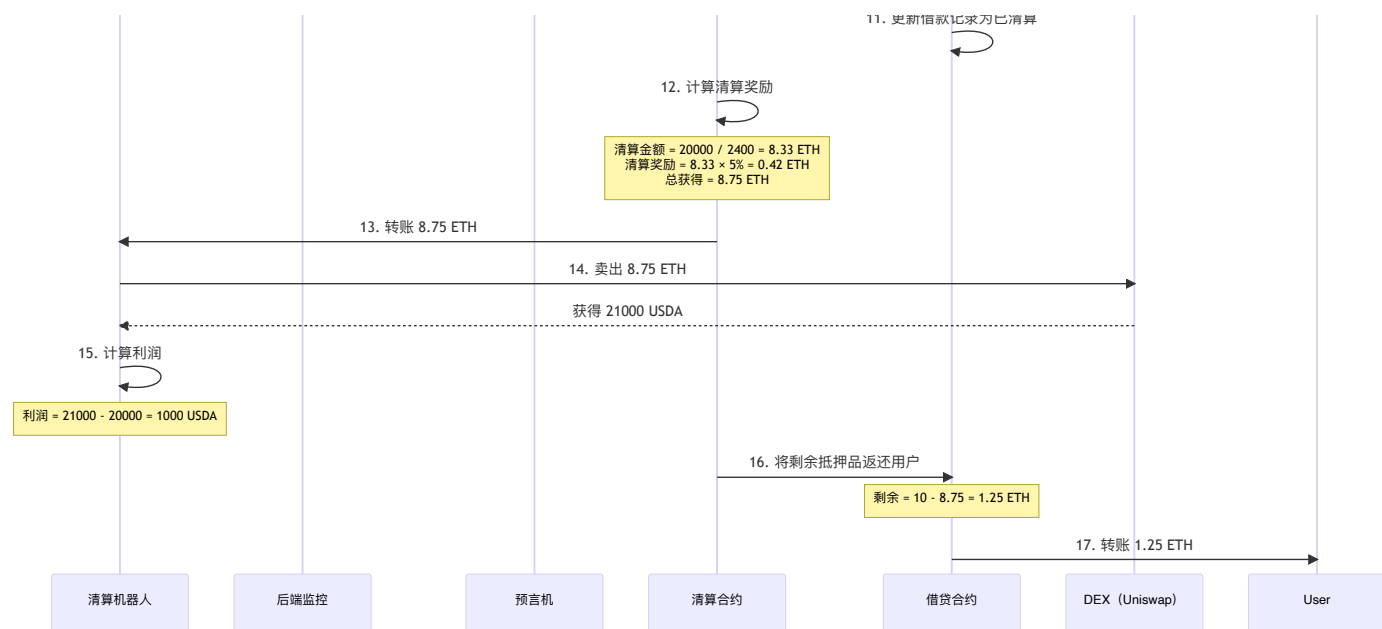
抵押品: 10 ETH × \$3000 = \$30,000

清算阈值：75%
借款金额：\$20,000
健康因子 = $(30000 \times 0.75) / 20000 = 1.125$

当健康因子 < 1.0 时，触发清算

清算流程：





清算激励机制：

抵押品类型	清算奖励	说明
稳定币（USDC）	2%	低风险，低奖励
wstETH	5%	中风险，中奖励
wBTC	5%	中风险，中奖励
其他代币	8-10%	高风险，高奖励

清算保护机制：

- 1. 渐进式清算：不是一次性清算全部抵押品，而是只清算恢复健康因子所需的最小金额
- 2. 价格延迟保护：使用多个预言机的中位数价格，防止价格操控
- 3. 清算冷却期：同一用户在 1 小时内只能被清算一次
- 4. 用户自救机制：健康因子 < 1.05 时发送预警，用户可以追加抵押品

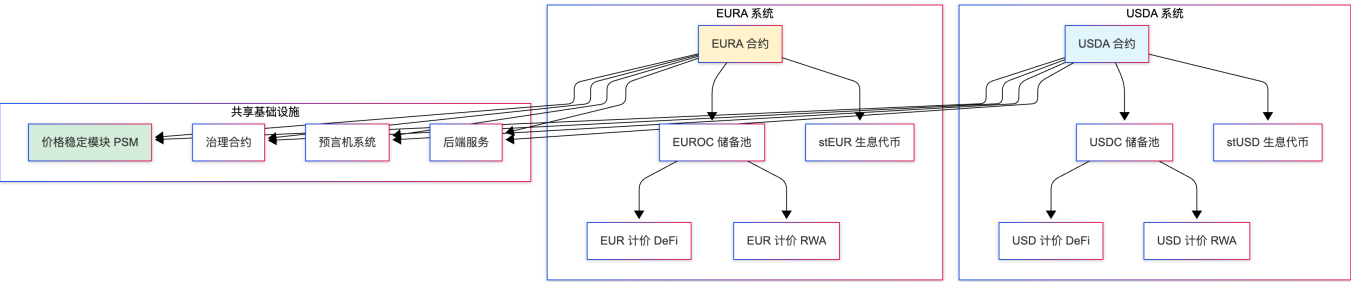
三、多币种与跨链架构

3.1 双币种设计（USDA + EURA）

为什么需要欧元稳定币？

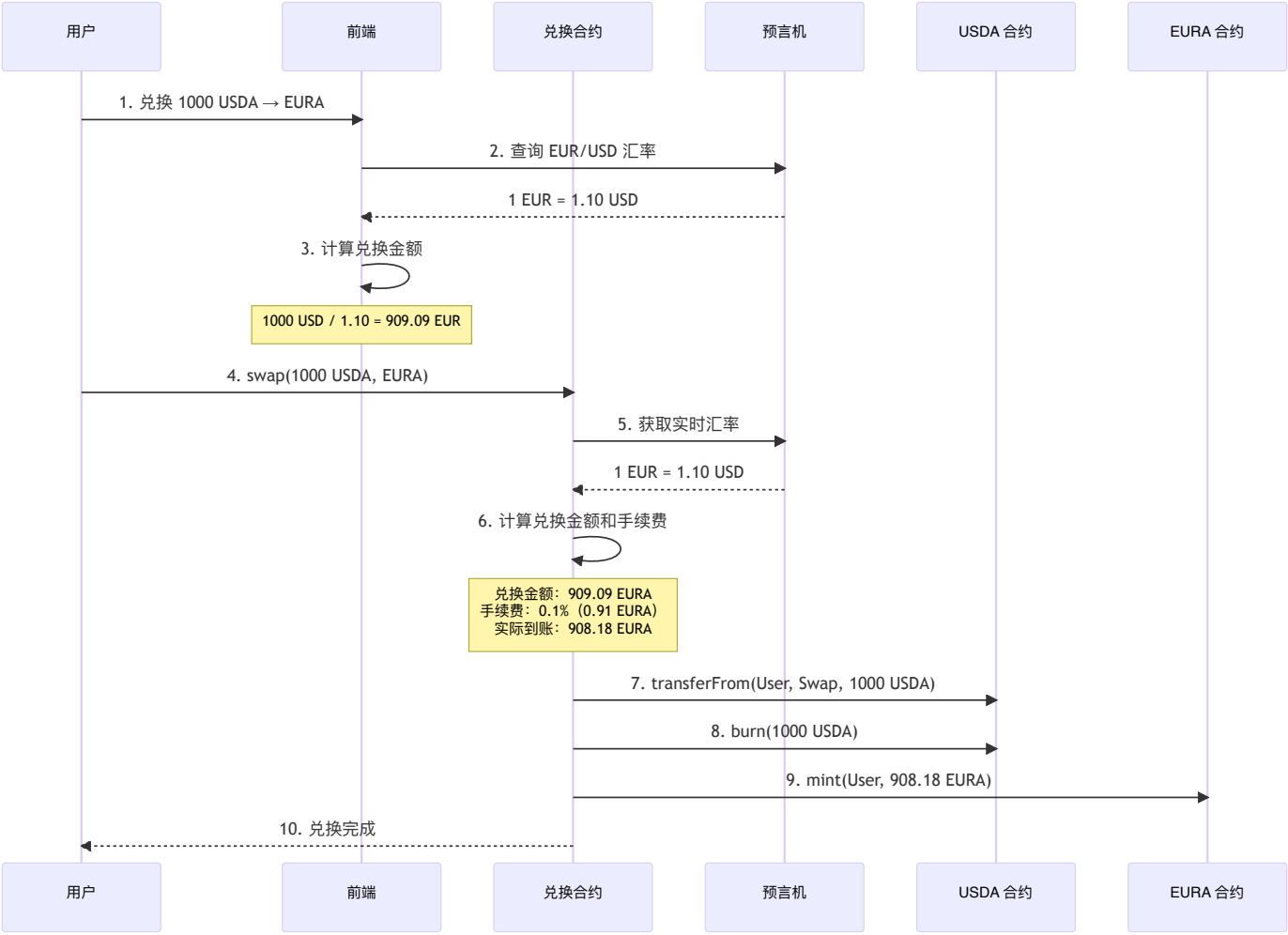
- 1. 市场需求：欧洲用户需要欧元计价的稳定币
- 2. 外汇交易：支持链上 EUR/USD 兑换
- 3. 风险分散：不同法币分散单一货币风险
- 4. 合规优势：符合欧洲监管要求

USDA 和 EURA 的独立性：



跨币种兑换机制：

用户可以在链上直接兑换 USDA 和 EURA，无需通过中心化交易所：



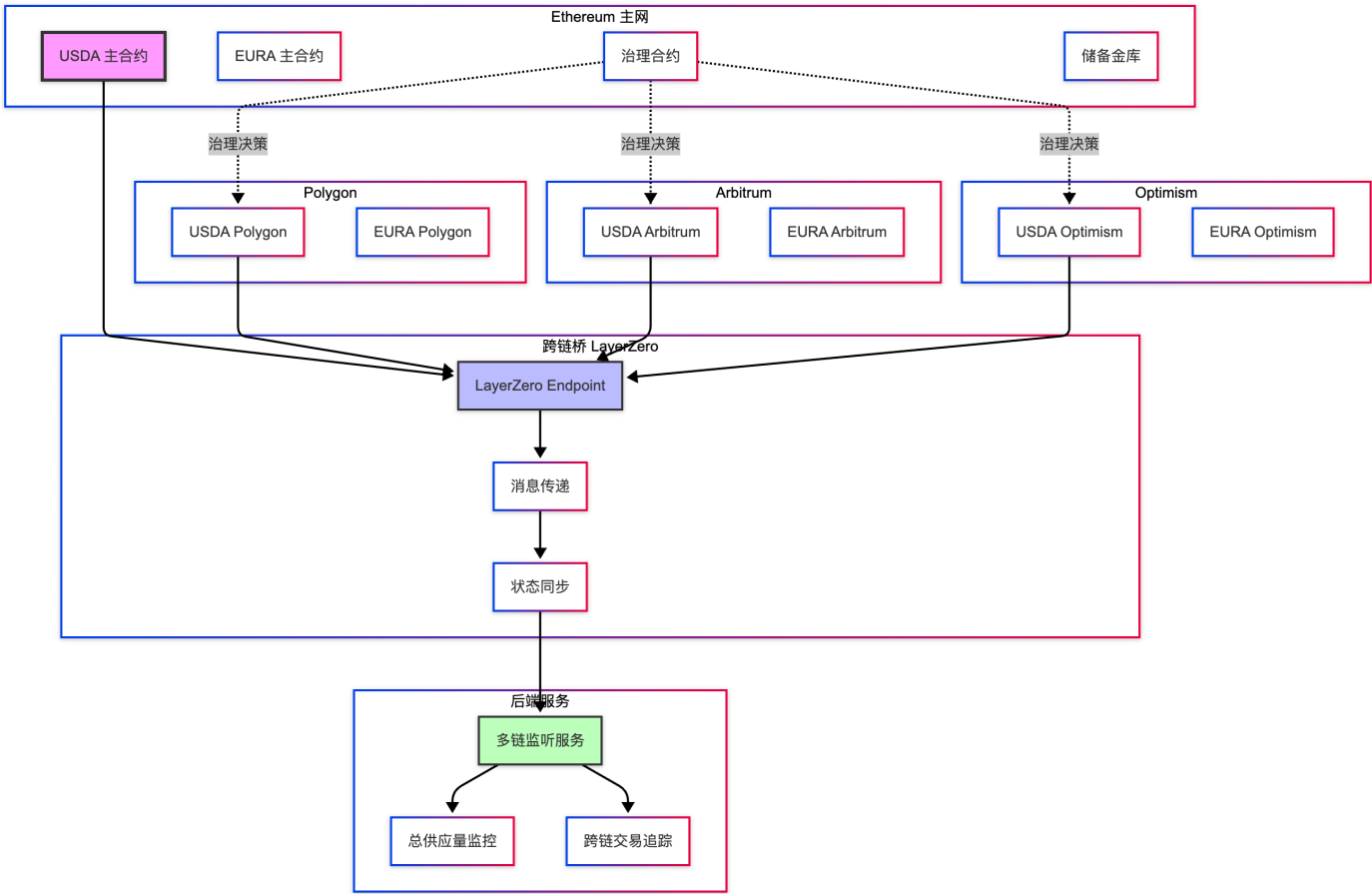
3.2 多链部署架构

支持的区块链：

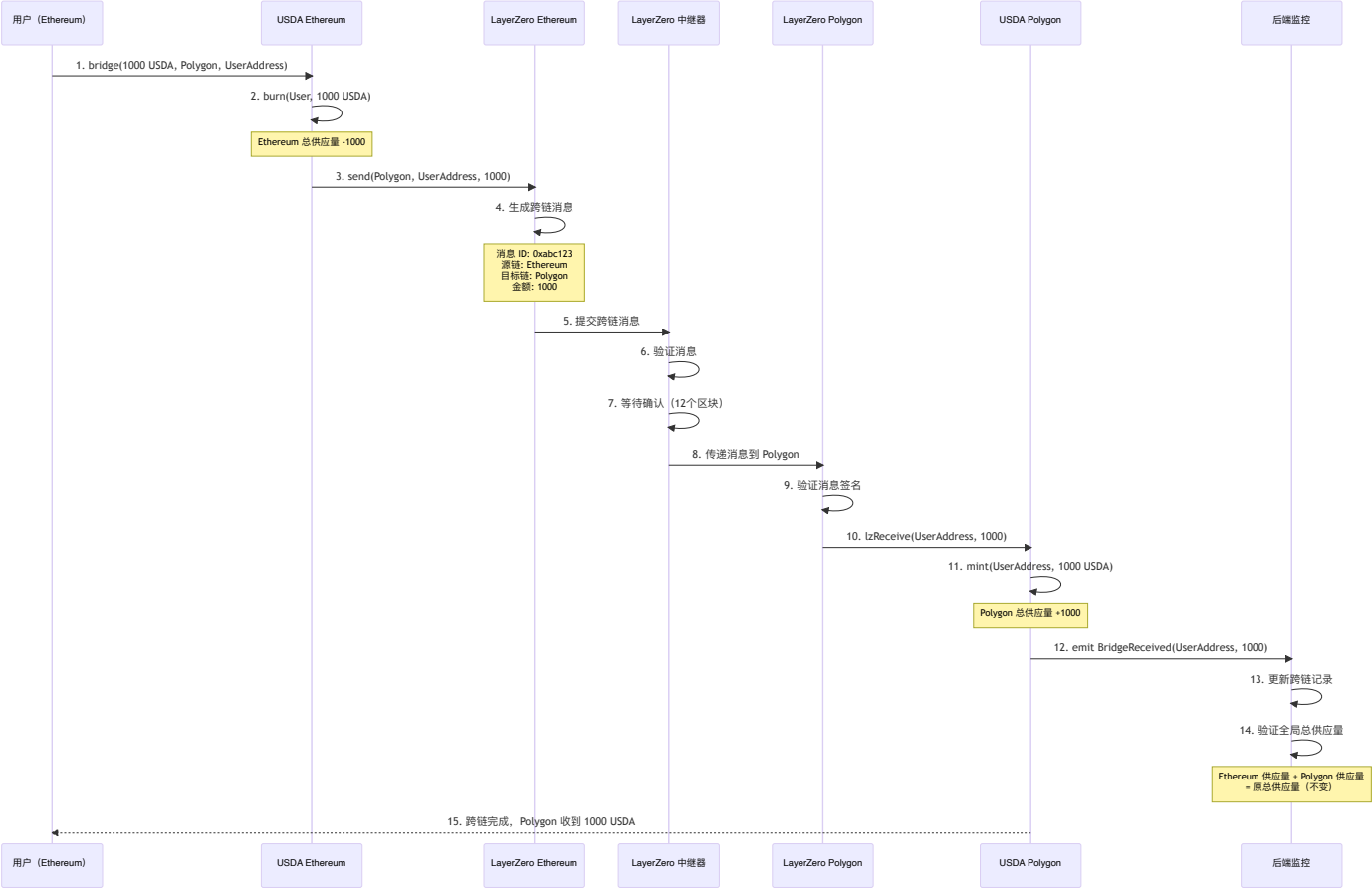
根据 Angle Protocol 的多链战略，协议部署在以下链上：

- 1. **Ethereum 主网**：主要部署链，最高安全性
- 2. **Polygon**：低 Gas 成本，适合小额交易
- 3. **Arbitrum**：Layer 2，高吞吐量
- 4. **Optimism**：Layer 2，低延迟
- 5. **Base**：Coinbase 的 Layer 2
- 6. **其他链**：根据需求扩展

多链架构设计：



跨链转账流程：



全局供应量监控：

为了确保跨链安全，后端服务持续监控所有链上的 USDA/EURA 总供应量：

```
全局总供应量 = Σ(各链供应量)

Ethereum: 50,000,000  USDA
Polygon:   30,000,000  USDA
Arbitrum:  15,000,000  USDA
Optimism:   5,000,000  USDA
-----
全局总供应量: 100,000,000  USDA

跨链前后，全局总供应量必须保持不变
```

四、关键技术点深度解析

4.1 价格稳定模块（PSM）技术实现

PSM 合约核心存储结构：

```
contract PriceStabilityModule {
    // 储备资产 (USDC)
    IERC20 public immutable reserveAsset;

    // 稳定币 (USDA)
    IStablecoin public immutable stablecoin;

    // 铸造和赎回费率 (基点, 10000 = 100%)
    uint256 public mintFee;           // 默认 0
    uint256 public redeemFee;         // 默认 0

    // 铸造和赎回上限
    uint256 public mintCap;           // 默认无限制
    uint256 public redeemCap;         // 默认无限制

    // 暂停标志
    bool public mintPaused;
```

```

    bool public redeemPaused;

    // 累积统计
    uint256 public totalMinted;
    uint256 public totalRedeemed;

    // 盈余缓冲目标
    uint256 public targetSurplusRatio; // 默认 10% (1000 基点)
}

```

PSM 核心函数设计：

1. mint 函数（铸造）：

函数签名：

```
function mint(uint256 amount) external nonReentrant whenNotPaused returns (uint256)
```

参数：

- amount：要存入的 USDC 数量

返回值：

- 铸造的 USDC 数量

执行流程：

1. 检查 mintPaused 是否为 false
2. 检查 amount 是否超过 mintCap
3. 计算手续费：fee = amount × mintFee / 10000
4. 计算实际铸造量：mintAmount = amount - fee
5. 从用户转入 USDC：reserveAsset.transferFrom(msg.sender, address(this), amount)
6. 铸造 USDC：stablecoin.mint(msg.sender, mintAmount)
7. 更新统计：totalMinted += mintAmount
8. 发射事件：emit Mint(msg.sender, amount, mintAmount, fee)
9. 返回 mintAmount

Gas 优化：

- 使用 immutable 变量存储合约地址
- 批量更新存储变量
- 使用 unchecked 块优化无溢出风险的计算

2. redeem 函数（赎回）：

函数签名：

```
function redeem(uint256 amount) external nonReentrant whenNotPaused returns (uint256)
```

参数：

- amount：要赎回的 USDC 数量

返回值：

- 获得的 USDC 数量

执行流程：

1. 检查 `redeemPaused` 是否为 `false`
2. 检查 `amount` 是否超过 `redeemCap`
3. 检查储备金是否充足: `reserveAsset.balanceOf(address(this)) >= amount`
4. 计算手续费: `fee = amount * redeemFee / 10000`
5. 计算实际赎回量: `redeemAmount = amount - fee`
6. 销毁 `USDA`: `stablecoin.burn(msg.sender, amount)`
7. 转出 `USDC`: `reserveAsset.transfer(msg.sender, redeemAmount)`
8. 更新统计: `totalRedeemed += amount`
9. 发射事件: `emit Redeem(msg.sender, amount, redeemAmount, fee)`
10. 返回 `redeemAmount`

安全检查:

- 遵循 CEI 模式 (Checks-Effects-Interactions)
- 先销毁 `USDA`, 再转出 `USDC`
- 使用 `nonReentrant` 防止重入攻击

3. 盈余缓冲计算:

函数签名:

```
function getSurplusBuffer() public view returns (uint256, uint256)
```

返回值:

- 盈余缓冲金额 (USD)
- 盈余缓冲率 (基点)

计算逻辑:

1. 获取储备资产总价值:
`reserveValue = reserveAsset.balanceOf(address(this))`
2. 获取稳定币总供应量:
`totalSupply = stablecoin.totalSupply()`
3. 计算盈余缓冲:
`surplusBuffer = reserveValue - totalSupply`
4. 计算盈余缓冲率:
`surplusRatio = (surplusBuffer * 10000) / totalSupply`
5. 返回 (`surplusBuffer`, `surplusRatio`)

示例:

```
reserveValue = 110,000,000 USDC  
totalSupply = 100,000,000 USDA  
surplusBuffer = 10,000,000 USDC  
surplusRatio = 1000 (10%)
```

4.2 超额抵押系统设计

抵押系统合约结构:

```
contract CollateralManager {
```



```

// 支持的抵押品列表
struct CollateralType {
    address token;                // 代币地址
    uint256 liquidationThreshold; // 清算阈值（基点）
    uint256 maxLTV;               // 最大贷款价值比
    bool isActive;               // 是否启用
    address priceOracle;          // 价格预言机
}

mapping(bytes32 => CollateralType) public collateralTypes;

// 用户抵押品记录
struct UserCollateral {
    uint256 amount;               // 抵押品数量
    uint256 borrowedAmount;      // 借款金额
    uint256 lastUpdateBlock;      // 最后更新区块
}

mapping(address => mapping(bytes32 => UserCollateral)) public userCollaterals;

// 全局抵押率
uint256 public globalCollateralRatio;
uint256 public minCollateralRatio = 10500; // 105%
}

```

抵押率实时计算：

函数签名：

```
function calculateCollateralRatio() public view returns (uint256)
```

计算逻辑：

1. 计算总储备资产价值：

```
totalReserveValue = 0
```

a. USDC 储备：

```
usdcBalance = USDC.balanceOf(treasury)
totalReserveValue += usdcBalance × 1.0 // USDC = $1
```

b. DeFi 借贷资产：

```
aaveBalance = aUSDC.balanceOf(treasury)
aaveValue = aaveBalance × aUSDC.exchangeRate()
totalReserveValue += aaveValue

compoundBalance = cUSDC.balanceOf(treasury)
compoundValue = compoundBalance × cUSDC.exchangeRate()
totalReserveValue += compoundValue
```

c. RWA 资产：

```
rwaBalance = RWAToken.balanceOf(treasury)
rwaPrice = RWAOracle.getPrice()
rwaValue = rwaBalance × rwaPrice
```

```
totalReserveValue += rwaValue
```

2. 获取稳定币总供应量:

```
totalSupply = USDA.totalSupply() + EURA.totalSupply() × EUR_USD_Rate
```

3. 计算抵押率:

```
collateralRatio = (totalReserveValue × 10000) / totalSupply
```

4. 返回 collateralRatio

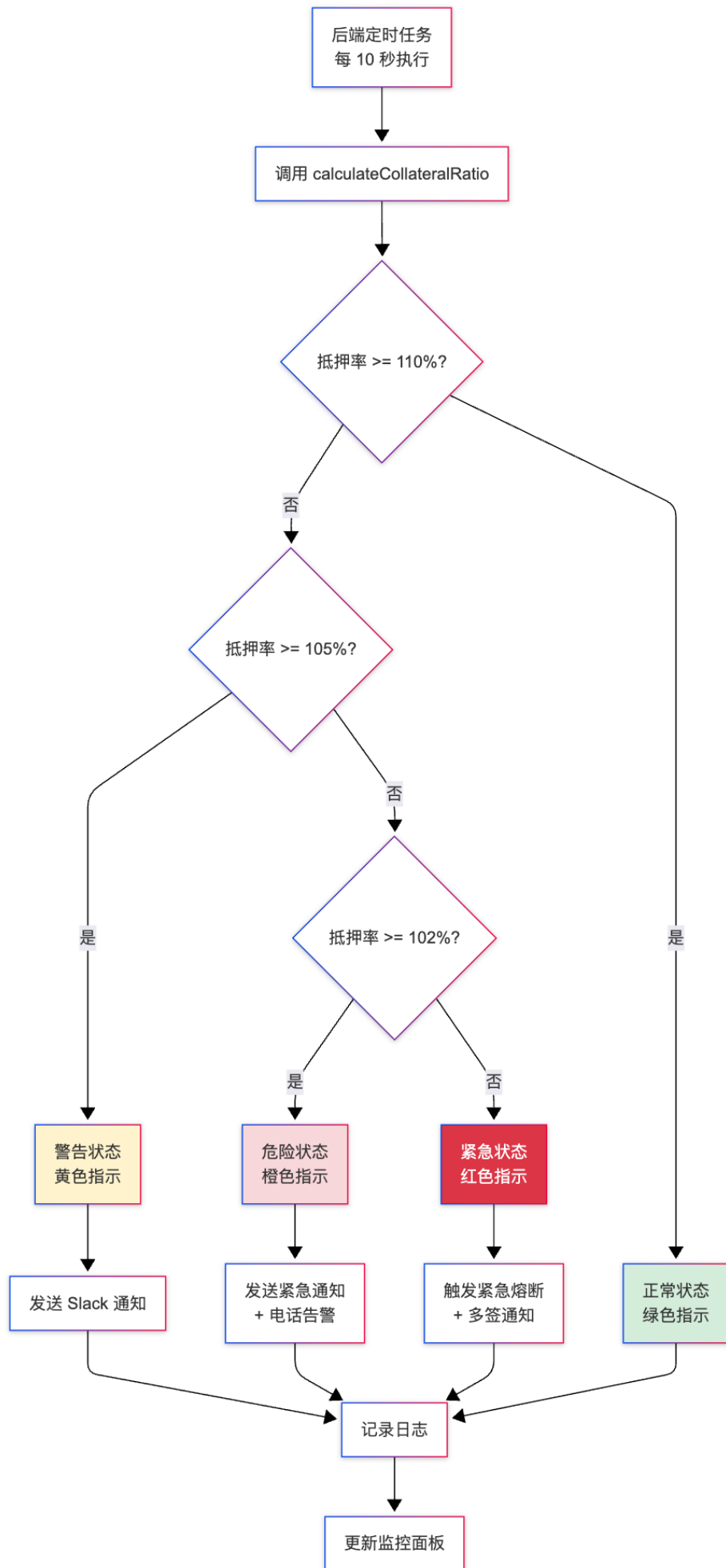
示例:

```
totalReserveValue = 120,000,000 USD
```

```
totalSupply = 100,000,000 USDA
```

```
collateralRatio = 12000 (120%)
```

抵押率监控与告警:



4.3 收益聚合器架构

收益聚合器合约设计：

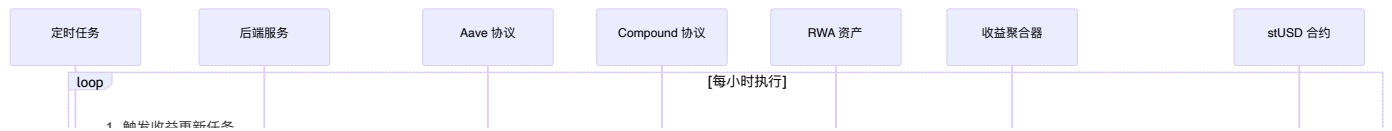
```
contract YieldAggregator {
    // 收益策略
    struct YieldStrategy {
        address protocol;           // 协议地址 (Aave/Compound)
        uint256 allocation;         // 分配比例 (基点)
        uint256 currentAPY;        // 当前 APY
        uint256 tvl;               // 锁定价值
        bool isActive;             // 是否启用
    }

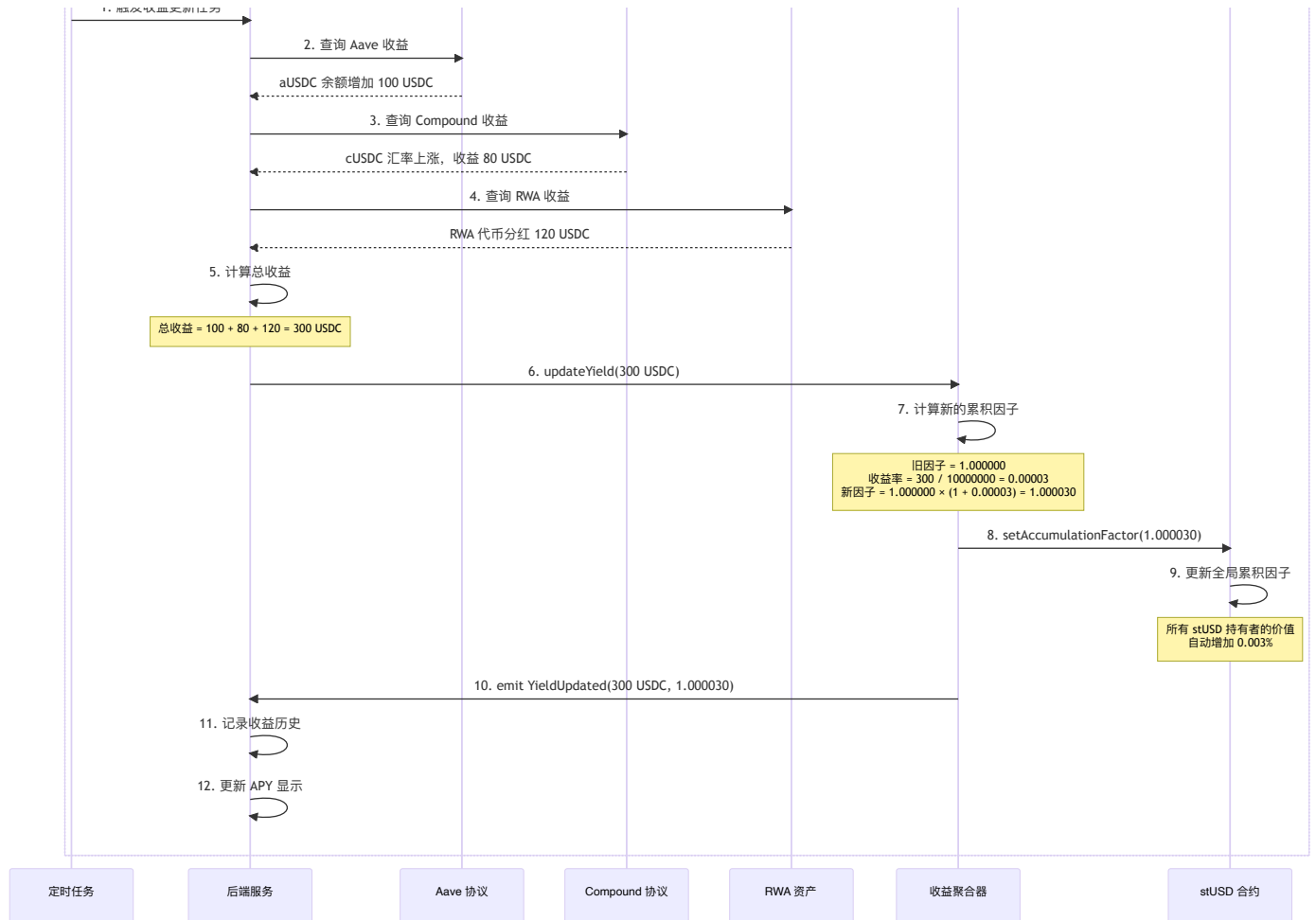
    YieldStrategy[] public strategies;

    // 全局收益统计
    uint256 public totalYieldEarned;
    uint256 public lastRebalanceBlock;
    uint256 public rebalanceInterval = 28800; // 约 4 小时 (以太坊)

    // 累积因子 (用于计算 stUSD 汇率)
    uint256 public globalAccumulationFactor = 1e18;
    uint256 public lastAccumulationBlock;
}
```

收益累积机制：





stUSD 汇率计算：

$\text{stUSD 汇率} = \text{全局累积因子}$

$\text{用户赎回时的 USDA 数量} = \text{stUSD 数量} \times \text{全局累积因子}$

示例：

用户在第 1 天存入 1000 USDA, 获得 1000 stUSD

当时累积因子 = 1.000000

第 30 天, 累积因子 = 1.050000 (增长 5%)

用户赎回 1000 stUSD

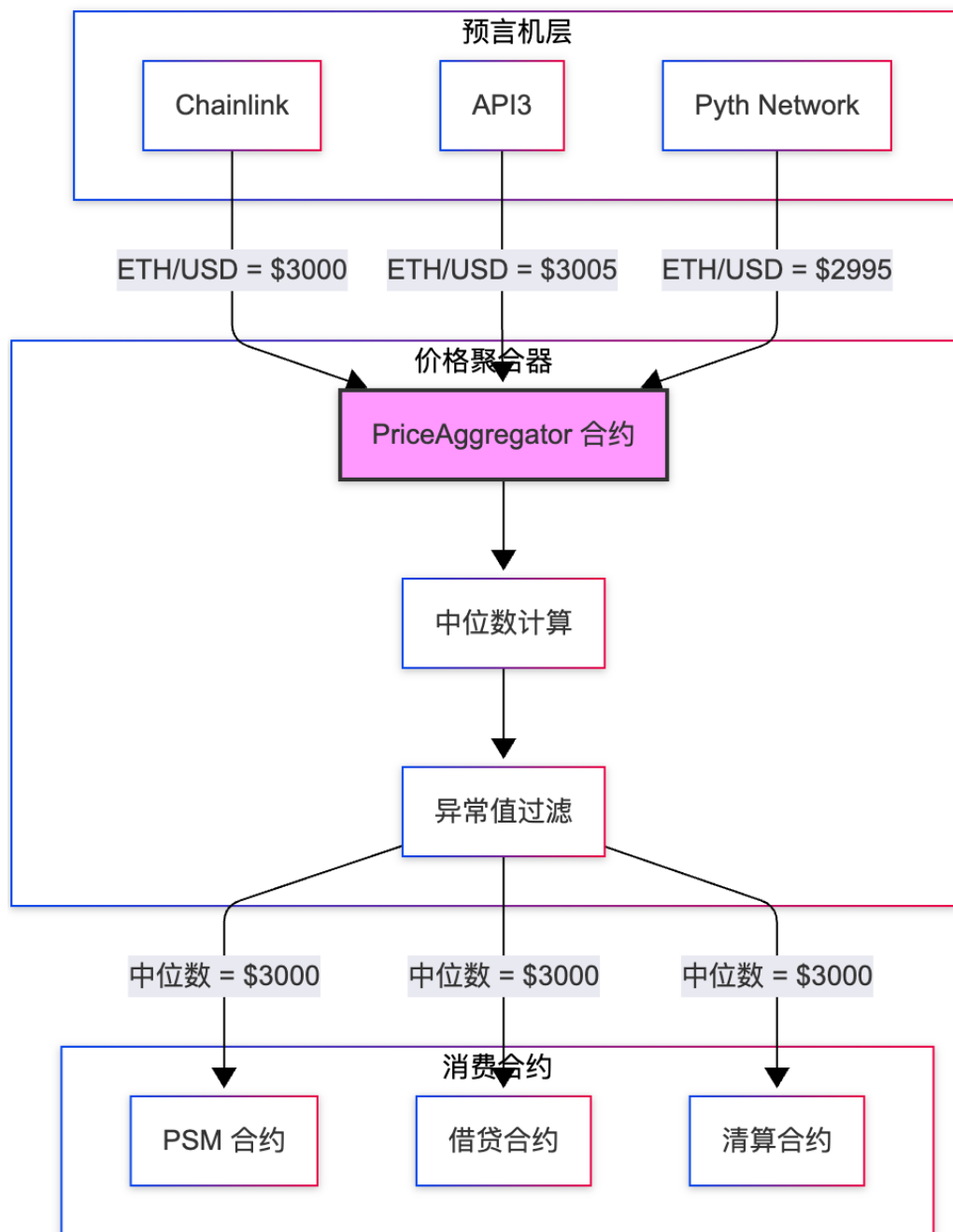
获得 USDA = $1000 \times 1.050000 = 1050$ USDA

收益 = 50 USDA

4.4 预言机系统集成

多预言机架构：

Angle Protocol 集成多个预言机以确保价格准确性和防止操控：



价格聚合逻辑:

函数签名:

```
function getAggregatedPrice(address asset) public view returns (uint256)
```

参数:

- asset: 资产地址 (如 WETH)

返回值:

- 聚合后的价格 (USD, 18 位小数)

执行流程:

1. 从 Chainlink 获取价格:

```
priceChainlink = chainlinkOracle.latestAnswer()
```

```
2. 从 API3 获取价格:
priceAPI3 = api3Oracle.read()

3. 从 Pyth 获取价格:
pricePyth = pythOracle.getPrice(assetId)

4. 将价格放入数组并排序:
prices = [priceChainlink, priceAPI3, pricePyth]
prices.sort()

5. 计算中位数:
medianPrice = prices[1] // 取中间值

6. 异常值检测:
deviation = max(prices) - min(prices)
if (deviation > medianPrice * 0.02) { // 偏差超过 2%
    revert("Price deviation too high")
}

7. 返回 medianPrice
```

示例:

Chainlink: \$3000

API3: \$3005

Pyth: \$2995

中位数: \$3000

偏差: $\$10 / \$3000 = 0.33\% < 2\%$, 通过检查

价格更新频率:

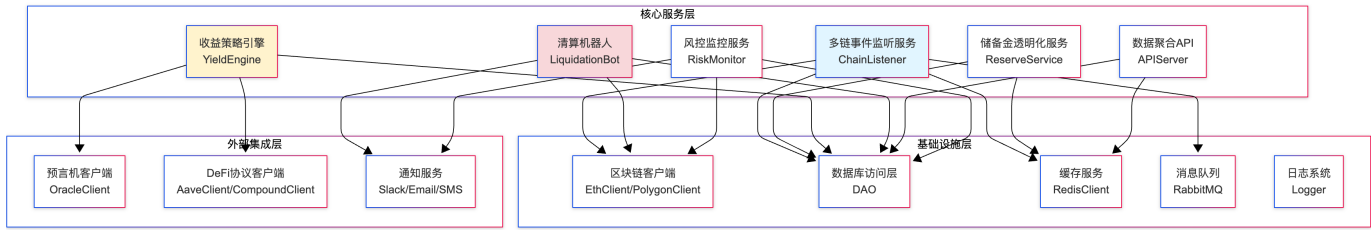
预言机	更新频率	延迟	成本
Chainlink	0.5% 价格变化或 1 小时	低	中
API3	1% 价格变化或 24 小时	低	低
Pyth Network	实时 (400ms)	极低	高

五、Golang 后端服务架构

5.1 后端服务总览

Angle Protocol 的后端服务使用 Golang 构建, 负责链下数据处理、事件监听、收益计算、风控监控等关键功能。后端服务与智能合约协同工作, 提供完整的 DeFi 基础设施。

后端服务模块划分:

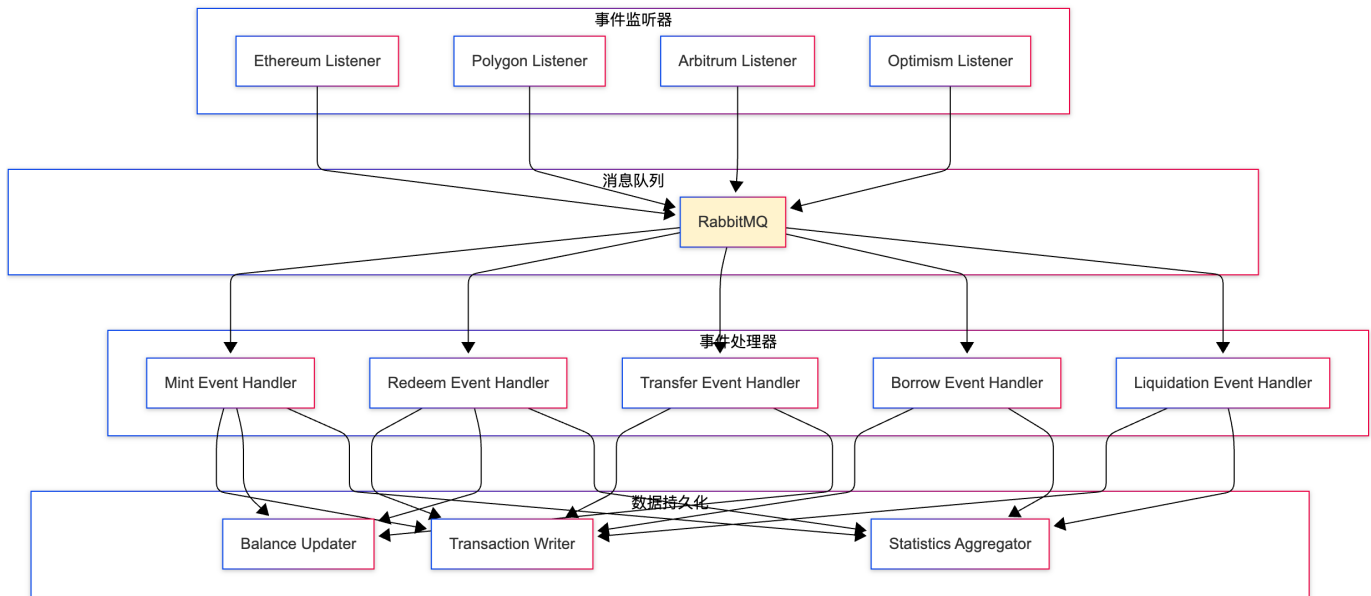


5.2 多链事件监听服务（ChainListener）

服务职责：

ChainListener 是后端服务的核心模块，负责实时监听多条区块链上的智能合约事件，并将事件数据同步到数据库。这是整个系统的数据入口。

技术架构：



核心代码结构：

1. 区块链客户端封装：

```
// pkg/blockchain/client.go

type ChainClient struct {
    chainID      uint64
    chainName    string
    rpcURL       string
    wsURL        string
    ethClient    *ethclient.Client
    wsClient     *ethclient.Client
    logger       *zap.Logger
}
```

```

// 合约实例
usdaContract      *usda.USDA
euraContract      *eura.EURA
psmContract       *psm.PSM
borrowContract    *borrow.Borrow

// 重连机制
reconnectDelay    time.Duration
maxReconnect      int
}

// 初始化客户端
func NewChainClient(config ChainConfig) (*ChainClient, error) {
    // 连接 RPC 节点
    ethClient, err := ethclient.Dial(config.RPCURL)
    if err != nil {
        return nil, fmt.Errorf("failed to connect to RPC: %w", err)
    }

    // 连接 WebSocket 节点（用于事件监听）
    wsClient, err := ethclient.Dial(config.WSURL)
    if err != nil {
        return nil, fmt.Errorf("failed to connect to WebSocket: %w", err)
    }

    // 加载合约 ABI 并创建合约实例
    usdaContract, err := usda.NewUSDA(
        common.HexToAddress(config.USDAAddress),
        ethClient,
    )
    if err != nil {
        return nil, fmt.Errorf("failed to load USDA contract: %w", err)
    }

    return &ChainClient{
        chainID:      config.ChainID,
        chainName:    config.ChainName,
        rpcURL:       config.RPCURL,
        wsURL:        config.WSURL,
        ethClient:    ethClient,
        wsClient:     wsClient,
        usdaContract: usdaContract,
        logger:       logger,
        reconnectDelay: 5 * time.Second,
        maxReconnect: 10,
    }, nil
}

// 获取当前区块高度
func (c *ChainClient) GetCurrentBlockNumber() (uint64, error) {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)

```

```

    defer cancel()

    blockNumber, err := c.ethClient.BlockNumber(ctx)
    if err != nil {
        return 0, fmt.Errorf("failed to get block number: %w", err)
    }

    return blockNumber, nil
}

// 获取区块详情
func (c *ChainClient) GetBlockByNumber(blockNumber uint64) (*types.Block, error) {
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()

    block, err := c.ethClient.BlockByNumber(ctx, big.NewInt(int64(blockNumber)))
    if err != nil {
        return nil, fmt.Errorf("failed to get block: %w", err)
    }

    return block, nil
}

```

2. 事件监听器实现:

```

// pkg/listener/chain_listener.go

type ChainListener struct {
    client      *blockchain.ChainClient
    eventChan   chan interface{}
    errorChan   chan error
    stopChan    chan struct{}

    lastProcessedBlock uint64
    batchSize          uint64

    db      *gorm.DB
    redis   *redis.Client
    mq       *rabbitmq.Client

    logger *zap.Logger
}

// 启动监听器
func (l *ChainListener) Start() error {
    l.logger.Info("Starting chain listener",
        zap.String("chain", l.client.ChainName()),
        zap.Uint64("from_block", l.lastProcessedBlock),
    )

    // 从数据库加载最后处理的区块高度

```

```

    if err := l.loadLastProcessedBlock(); err != nil {
        return fmt.Errorf("failed to load last processed block: %w", err)
    }

    // 启动事件处理协程
    go l.processEvents()

    // 启动区块扫描协程
    go l.scanBlocks()

    return nil
}

```

// 扫描区块

```

func (l *ChainListener) scanBlocks() {
    ticker := time.NewTicker(2 * time.Second) // 每 2 秒扫描一次
    defer ticker.Stop()

    for {
        select {
            case <-ticker.C:
                if err := l.scanNewBlocks(); err != nil {
                    l.logger.Error("Failed to scan blocks", zap.Error(err))
                    l.errorChan <- err
                }

            case <-l.stopChan:
                l.logger.Info("Stopping block scanner")
                return
        }
    }
}

```

// 扫描新区块

```

func (l *ChainListener) scanNewBlocks() error {
    // 获取当前链上最新区块
    currentBlock, err := l.client.GetCurrentBlockNumber()
    if err != nil {
        return fmt.Errorf("failed to get current block: %w", err)
    }

    // 计算需要扫描的区块范围
    fromBlock := l.lastProcessedBlock + 1
    toBlock := currentBlock

    // 如果没有新区块, 直接返回
    if fromBlock > toBlock {
        return nil
    }

    // 分批扫描 (避免单次查询区块过多)
    for from := fromBlock; from <= toBlock; from += l.batchSize {

```

```

    to := from + l.batchSize - 1
    if to > toBlock {
        to = toBlock
    }

    l.logger.Debug("Scanning blocks",
        zap.Uint64("from", from),
        zap.Uint64("to", to),
    )

    // 扫描 USDA Mint 事件
    if err := l.scanMintEvents(from, to); err != nil {
        return fmt.Errorf("failed to scan mint events: %w", err)
    }

    // 扫描 USDA Redeem 事件
    if err := l.scanRedeemEvents(from, to); err != nil {
        return fmt.Errorf("failed to scan redeem events: %w", err)
    }

    // 扫描 Transfer 事件
    if err := l.scanTransferEvents(from, to); err != nil {
        return fmt.Errorf("failed to scan transfer events: %w", err)
    }

    // 扫描 Borrow 事件
    if err := l.scanBorrowEvents(from, to); err != nil {
        return fmt.Errorf("failed to scan borrow events: %w", err)
    }

    // 扫描 Liquidation 事件
    if err := l.scanLiquidationEvents(from, to); err != nil {
        return fmt.Errorf("failed to scan liquidation events: %w", err)
    }

    // 更新最后处理的区块
    l.lastProcessedBlock = to
    if err := l.saveLastProcessedBlock(); err != nil {
        return fmt.Errorf("failed to save last processed block: %w", err)
    }
}

return nil
}

// 扫描 Mint 事件
func (l *ChainListener) scanMintEvents(fromBlock, toBlock uint64) error {
    // 构建过滤器
    filterOpts := &bind.FilterOpts{
        Start:    fromBlock,
        End:      &toBlock,
        Context:  context.Background(),
    }

```

```

}

// 查询 Mint 事件
iter, err := l.client.USDAContract().FilterMint(filterOpts, nil)
if err != nil {
    return fmt.Errorf("failed to filter mint events: %w", err)
}
defer iter.Close()

// 遍历事件
for iter.Next() {
    event := iter.Event

    l.logger.Info("Found Mint event",
        zap.String("user", event.User.Hex()),
        zap.String("amount", event.Amount.String()),
        zap.Uint64("block", event.Raw.BlockNumber),
        zap.String("tx", event.Raw.TxHash.Hex()),
    )

    // 发送到消息队列
    if err := l.publishEvent("mint", event); err != nil {
        return fmt.Errorf("failed to publish mint event: %w", err)
    }
}

if err := iter.Error(); err != nil {
    return fmt.Errorf("iterator error: %w", err)
}

return nil
}

// 发布事件到消息队列
func (l *ChainListener) publishEvent(eventType string, event interface{}) error {
    // 序列化事件
    eventData, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("failed to marshal event: %w", err)
    }

    // 构建消息
    message := rabbitmq.Message{
        Type:      eventType,
        ChainID:   l.client.ChainID(),
        ChainName: l.client.ChainName(),
        Data:      eventData,
        Timestamp: time.Now(),
    }

    // 发布到 RabbitMQ
    if err := l.mq.Publish("events", message); err != nil {

```

```

        return fmt.Errorf("failed to publish to MQ: %w", err)
    }

    return nil
}

// 保存最后处理的区块
func (l *ChainListener) saveLastProcessedBlock() error {
    key := fmt.Sprintf("last_processed_block:%s", l.client.ChainName())

    // 保存到 Redis
    if err := l.redis.Set(
        context.Background(),
        key,
        l.lastProcessedBlock,
        0, // 不过期
    ).Err(); err != nil {
        return fmt.Errorf("failed to save to redis: %w", err)
    }

    // 保存到数据库
    syncStatus := &models.ChainSyncStatus{
        ChainID:      l.client.ChainID(),
        ChainName:    l.client.ChainName(),
        LastProcessedBlock: l.lastProcessedBlock,
        UpdatedAt:    time.Now(),
    }

    if err := l.db.Save(syncStatus).Error; err != nil {
        return fmt.Errorf("failed to save to db: %w", err)
    }

    return nil
}

```

3. 事件处理器实现:

```

// pkg/handler/mint_handler.go

type MintEventHandler struct {
    db      *gorm.DB
    redis   *redis.Client
    logger  *zap.Logger
}

// 处理 Mint 事件
func (h *MintEventHandler) Handle(event *contracts.USDAMint) error {
    h.logger.Info("Processing Mint event",
        zap.String("user", event.User.Hex()),
        zap.String("amount", event.Amount.String()),
    )
}

```

```

// 开启数据库事务
tx := h.db.Begin()
defer func() {
    if r := recover(); r != nil {
        tx.Rollback()
    }
}()

// 1. 保存交易记录
transaction := &models.Transaction{
    ChainID:      event.Raw.ChainID,
    TxHash:       event.Raw.TxHash.Hex(),
    BlockNumber:  event.Raw.BlockNumber,
    Type:         "mint",
    User:         event.User.Hex(),
    Amount:       event.Amount.String(),
    Timestamp:    time.Now(),
}

if err := tx.Create(transaction).Error; err != nil {
    tx.Rollback()
    return fmt.Errorf("failed to save transaction: %w", err)
}

// 2. 更新用户余额
var userBalance models.UserBalance
err := tx.Where("chain_id = ? AND user_address = ?",
    event.Raw.ChainID,
    event.User.Hex(),
).First(&userBalance).Error

if err == gorm.ErrRecordNotFound {
    // 创建新记录
    userBalance = models.UserBalance{
        ChainID:      event.Raw.ChainID,
        UserAddress:   event.User.Hex(),
        Balance:       event.Amount.String(),
        UpdatedAt:    time.Now(),
    }
    if err := tx.Create(&userBalance).Error; err != nil {
        tx.Rollback()
        return fmt.Errorf("failed to create user balance: %w", err)
    }
} else if err != nil {
    tx.Rollback()
    return fmt.Errorf("failed to query user balance: %w", err)
} else {
    // 更新余额
    oldBalance := new(big.Int)
    oldBalance.SetString(userBalance.Balance, 10)
    newBalance := new(big.Int).Add(oldBalance, event.Amount)

```



```

    userBalance.Balance = newBalance.String()
    userBalance.UpdatedAt = time.Now()

    if err := tx.Save(&userBalance).Error; err != nil {
        tx.Rollback()
        return fmt.Errorf("failed to update user balance: %w", err)
    }
}

// 3. 更新全局统计
var stats models.GlobalStats
err = tx.Where("chain_id = ?", event.Raw.ChainID).First(&stats).Error

if err == gorm.ErrRecordNotFound {
    stats = models.GlobalStats{
        ChainID:      event.Raw.ChainID,
        TotalSupply:  event.Amount.String(),
        TotalMinted:  event.Amount.String(),
        TotalRedeemed: "0",
        UpdatedAt:    time.Now(),
    }
    if err := tx.Create(&stats).Error; err != nil {
        tx.Rollback()
        return fmt.Errorf("failed to create stats: %w", err)
    }
} else if err != nil {
    tx.Rollback()
    return fmt.Errorf("failed to query stats: %w", err)
} else {
    // 更新统计
    totalSupply := new(big.Int)
    totalSupply.SetString(stats.TotalSupply, 10)
    totalSupply.Add(totalSupply, event.Amount)

    totalMinted := new(big.Int)
    totalMinted.SetString(stats.TotalMinted, 10)
    totalMinted.Add(totalMinted, event.Amount)

    stats.TotalSupply = totalSupply.String()
    stats.TotalMinted = totalMinted.String()
    stats.UpdatedAt = time.Now()

    if err := tx.Save(&stats).Error; err != nil {
        tx.Rollback()
        return fmt.Errorf("failed to update stats: %w", err)
    }
}

// 提交事务
if err := tx.Commit().Error; err != nil {
    return fmt.Errorf("failed to commit transaction: %w", err)
}

```

```

    }

    // 4. 更新 Redis 缓存
    if err := h.updateCache(event); err != nil {
        h.logger.Error("Failed to update cache", zap.Error(err))
        // 缓存更新失败不影响主流程
    }

    h.logger.Info("Mint event processed successfully")
    return nil
}

// 更新缓存
func (h *MintEventHandler) updateCache(event *contracts.USDAMint) error {
    ctx := context.Background()

    // 更新用户余额缓存
    userBalanceKey := fmt.Sprintf("balance:%d:%s",
        event.Raw.ChainID,
        event.User.Hex(),
    )

    // 删除旧缓存，下次查询时重新加载
    if err := h.redis.Del(ctx, userBalanceKey).Err(); err != nil {
        return fmt.Errorf("failed to delete cache: %w", err)
    }

    // 更新全局统计缓存
    statsKey := fmt.Sprintf("stats:%d", event.Raw.ChainID)
    if err := h.redis.Del(ctx, statsKey).Err(); err != nil {
        return fmt.Errorf("failed to delete stats cache: %w", err)
    }

    return nil
}

```

4. 重连机制:

```

// pkg/listener/reconnect.go

// WebSocket 重连机制
func (l *ChainListener) maintainConnection() {
    reconnectCount := 0

    for {
        select {
        case <-l.stopChan:
            return

        case err := <-l.errorChan:
            l.logger.Error("Connection error", zap.Error(err))

```

```

        reconnectCount++
        if reconnectCount > l.client.MaxReconnect() {
            l.logger.Fatal("Max reconnect attempts reached")
            return
        }

        l.logger.Info("Attempting to reconnect",
            zap.Int("attempt", reconnectCount),
            zap.Duration("delay", l.client.ReconnectDelay()),
        )

        time.Sleep(l.client.ReconnectDelay())

        // 重新连接
        if err := l.client.Reconnect(); err != nil {
            l.logger.Error("Reconnect failed", zap.Error(err))
            continue
        }

        l.logger.Info("Reconnected successfully")
        reconnectCount = 0

        // 重新启动监听
        go l.scanBlocks()
    }
}

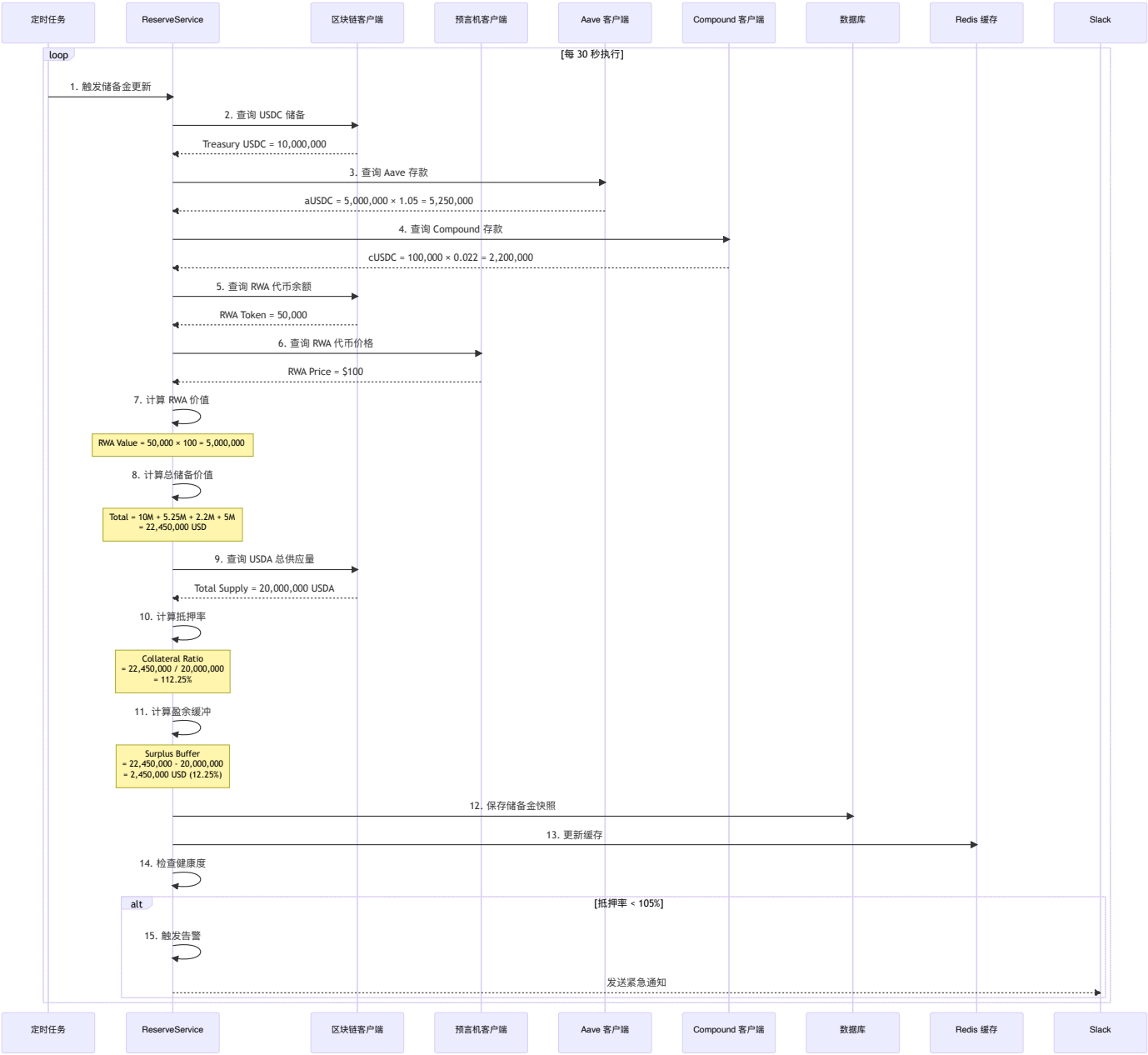
```

5.3 储备金透明化服务（ReserveService）

服务职责：

ReserveService 负责实时计算和展示协议的储备金状态，包括各类资产的价值、抵押率、盈余缓冲等关键指标。这是协议透明度的核心体现。

储备金计算流程：



核心代码实现：

```
// pkg/service/reserve_service.go

type ReserveService struct {
    chainClients    map[uint64]*blockchain.ChainClient
    oracleClient     *oracle.Client
    aaveClient       *aave.Client
    compoundClient   *compound.Client

    db               *gorm.DB
    redis            *redis.Client
    logger           *zap.Logger

    updateInterval time.Duration
}

// 储备金资产结构
type ReserveAssets struct {
    ChainID          uint64    `json:"chain_id"`

    // USDC 储备
    USDCBalance      string    `json:"usdc_balance"`
}
```

```

// DeFi 借贷资产
AaveBalance      string    `json:"aave_balance"`
AaveValue         string    `json:"aave_value"`
CompoundBalance   string    `json:"compound_balance"`
CompoundValue     string    `json:"compound_value"`

// RWA 资产
RWABalance        string    `json:"rwa_balance"`
RWAPrice          string    `json:"rwa_price"`
RWAValue          string    `json:"rwa_value"`

// 总计
TotalReserveValue string    `json:"total_reserve_value"`
TotalSupply       string    `json:"total_supply"`
CollateralRatio   string    `json:"collateral_ratio"`
SurplusBuffer     string    `json:"surplus_buffer"`
SurplusRatio      string    `json:"surplus_ratio"`

Timestamp         time.Time `json:"timestamp"`
}

// 启动服务
func (s *ReserveService) Start() error {
    s.logger.Info("Starting reserve service")

    ticker := time.NewTicker(s.updateInterval)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            if err := s.updateReserves(); err != nil {
                s.logger.Error("Failed to update reserves", zap.Error(err))
            }
        }
    }
}

// 更新储备金数据
func (s *ReserveService) updateReserves() error {
    // 遍历所有链
    for chainID, client := range s.chainClients {
        s.logger.Debug("Updating reserves",
            zap.Uint64("chain_id", chainID),
            zap.String("chain", client.ChainName()),
        )

        reserves, err := s.calculateReserves(chainID, client)
        if err != nil {
            return fmt.Errorf("failed to calculate reserves for chain %d: %w",
                chainID, err)
        }
    }
}

```

```

    }

    // 保存到数据库
    if err := s.saveReserves(reserves); err != nil {
        return fmt.Errorf("failed to save reserves: %w", err)
    }

    // 更新缓存
    if err := s.cacheReserves(reserves); err != nil {
        s.logger.Error("Failed to cache reserves", zap.Error(err))
    }

    // 检查健康度
    if err := s.checkHealth(reserves); err != nil {
        s.logger.Error("Health check failed", zap.Error(err))
    }
}

return nil
}

// 计算储备金
func (s *ReserveService) calculateReserves(
    chainID uint64,
    client *blockchain.ChainClient,
) (*ReserveAssets, error) {

    reserves := &ReserveAssets{
        ChainID:    chainID,
        Timestamp:  time.Now(),
    }

    // 1. 查询 USDC 储备
    usdcBalance, err := client.GetUSDCBalance()
    if err != nil {
        return nil, fmt.Errorf("failed to get USDC balance: %w", err)
    }
    reserves.USDCBalance = usdcBalance.String()

    // 2. 查询 Aave 存款
    aaveBalance, err := s.aaveClient.GetBalance(chainID)
    if err != nil {
        return nil, fmt.Errorf("failed to get Aave balance: %w", err)
    }
    reserves.AaveBalance = aaveBalance.String()

    // 获取 Aave 汇率
    aaveRate, err := s.aaveClient.GetExchangeRate(chainID)
    if err != nil {
        return nil, fmt.Errorf("failed to get Aave rate: %w", err)
    }
}

```

```

// 计算 Aave 价值
aaveValue := new(big.Int).Mul(aaveBalance, aaveRate)
aaveValue.Div(aaveValue, big.NewInt(1e18))
reserves.AaveValue = aaveValue.String()

// 3. 查询 Compound 存款
compoundBalance, err := s.compoundClient.GetBalance(chainID)
if err != nil {
    return nil, fmt.Errorf("failed to get Compound balance: %w", err)
}
reserves.CompoundBalance = compoundBalance.String()

// 获取 Compound 汇率
compoundRate, err := s.compoundClient.GetExchangeRate(chainID)
if err != nil {
    return nil, fmt.Errorf("failed to get Compound rate: %w", err)
}

// 计算 Compound 价值
compoundValue := new(big.Int).Mul(compoundBalance, compoundRate)
compoundValue.Div(compoundValue, big.NewInt(1e18))
reserves.CompoundValue = compoundValue.String()

// 4. 查询 RWA 代币余额
rwaBalance, err := client.GetRWABalance()
if err != nil {
    return nil, fmt.Errorf("failed to get RWA balance: %w", err)
}
reserves.RWABalance = rwaBalance.String()

// 获取 RWA 价格
rwaPrice, err := s.oracleClient.GetRWAPrice()
if err != nil {
    return nil, fmt.Errorf("failed to get RWA price: %w", err)
}
reserves.RWAPrice = rwaPrice.String()

// 计算 RWA 价值
rwaValue := new(big.Int).Mul(rwaBalance, rwaPrice)
rwaValue.Div(rwaValue, big.NewInt(1e18))
reserves.RWAValue = rwaValue.String()

// 5. 计算总储备价值
totalReserve := new(big.Int).Set(usdcBalance)
totalReserve.Add(totalReserve, aaveValue)
totalReserve.Add(totalReserve, compoundValue)
totalReserve.Add(totalReserve, rwaValue)
reserves.TotalReserveValue = totalReserve.String()

// 6. 查询 USDA 总供应量
totalSupply, err := client.GetTotalSupply()
if err != nil {

```



```

        return nil, fmt.Errorf("failed to get total supply: %w", err)
    }
    reserves.TotalSupply = totalSupply.String()

    // 7. 计算抵押率
    if totalSupply.Cmp(big.NewInt(0)) > 0 {
        collateralRatio := new(big.Int).Mul(totalReserve, big.NewInt(10000))
        collateralRatio.Div(collateralRatio, totalSupply)
        reserves.CollateralRatio = collateralRatio.String() // 基点
    } else {
        reserves.CollateralRatio = "0"
    }

    // 8. 计算盈余缓冲
    surplusBuffer := new(big.Int).Sub(totalReserve, totalSupply)
    reserves.SurplusBuffer = surplusBuffer.String()

    if totalSupply.Cmp(big.NewInt(0)) > 0 {
        surplusRatio := new(big.Int).Mul(surplusBuffer, big.NewInt(10000))
        surplusRatio.Div(surplusRatio, totalSupply)
        reserves.SurplusRatio = surplusRatio.String() // 基点
    } else {
        reserves.SurplusRatio = "0"
    }

    return reserves, nil
}

// 存储备金数据
func (s *ReserveService) saveReserves(reserves *ReserveAssets) error {
    // 保存到数据库
    snapshot := &models.ReserveSnapshot{
        ChainID:      reserves.ChainID,
        USDCBalance:   reserves.USDCBalance,
        AaveValue:     reserves.AaveValue,
        CompoundValue: reserves.CompoundValue,
        RWAValue:      reserves.RWAValue,
        TotalReserveValue: reserves.TotalReserveValue,
        TotalSupply:   reserves.TotalSupply,
        CollateralRatio: reserves.CollateralRatio,
        SurplusBuffer: reserves.SurplusBuffer,
        SurplusRatio:  reserves.SurplusRatio,
        Timestamp:     reserves.Timestamp,
    }

    if err := s.db.Create(snapshot).Error; err != nil {
        return fmt.Errorf("failed to save snapshot: %w", err)
    }

    return nil
}

```

```

// 缓存储备金数据
func (s *ReserveService) cacheReserves(reserves *ReserveAssets) error {
    ctx := context.Background()

    // 序列化数据
    data, err := json.Marshal(reserves)
    if err != nil {
        return fmt.Errorf("failed to marshal reserves: %w", err)
    }

    // 保存到 Redis
    key := fmt.Sprintf("reserves:%d", reserves.ChainID)
    if err := s.redis.Set(ctx, key, data, 5*time.Minute).Err(); err != nil {
        return fmt.Errorf("failed to cache reserves: %w", err)
    }

    return nil
}

// 健康度检查
func (s *ReserveService) checkHealth(reserves *ReserveAssets) error {
    collateralRatio := new(big.Int)
    collateralRatio.SetString(reserves.CollateralRatio, 10)

    // 抵押率阈值 (基点)
    const (
        CRITICAL_THRESHOLD = 10200 // 102%
        WARNING_THRESHOLD   = 10500 // 105%
        HEALTHY_THRESHOLD   = 11000 // 110%
    )

    if collateralRatio.Cmp(big.NewInt(CRITICAL_THRESHOLD)) < 0 {
        // 紧急状态
        s.logger.Error("CRITICAL: Collateral ratio too low",
            zap.String("ratio", reserves.CollateralRatio),
            zap.Uint64("chain_id", reserves.ChainID),
        )

        // 发送紧急告警
        s.sendCriticalAlert(reserves)
    } else if collateralRatio.Cmp(big.NewInt(WARNING_THRESHOLD)) < 0 {
        // 警告状态
        s.logger.Warn("WARNING: Collateral ratio below threshold",
            zap.String("ratio", reserves.CollateralRatio),
            zap.Uint64("chain_id", reserves.ChainID),
        )

        // 发送警告通知
        s.sendWarningAlert(reserves)
    } else if collateralRatio.Cmp(big.NewInt(HEALTHY_THRESHOLD)) >= 0 {

```

```
// 健康状态
s.logger.Info("Collateral ratio healthy",
    zap.String("ratio", reserves.CollateralRatio),
    zap.Uint64("chain_id", reserves.ChainID),
)
}

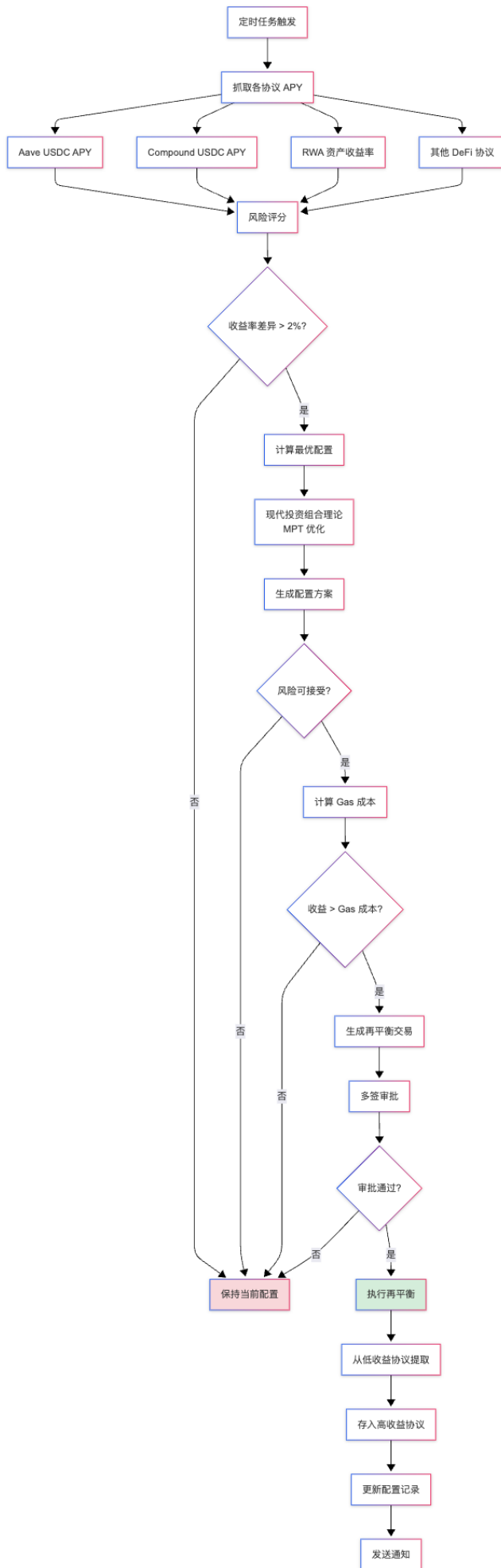
return nil
}
```

5.4 收益策略引擎 (YieldEngine)

服务职责：

YieldEngine 负责动态评估各 DeFi 协议和 RWA 资产的收益率，生成最优资产配置方案，并执行自动再平衡策略。

**收益策略决策流程：



核心代码实现：

```
// pkg/service/yield_engine.go

type YieldEngine struct {
    chainClients    map[uint64]*blockchain.ChainClient
    aaveClient      *aave.Client
    compoundClient  *compound.Client
    rwaClient       *rwa.Client

    db              *gorm.DB
    redis           *redis.Client
    logger          *zap.Logger

    // 配置参数
    minYieldDiff    float64 // 最小收益率差异 (2%)
    maxRiskScore    float64 // 最大风险评分 (7/10)
    rebalanceInterval time.Duration
}

// 协议收益率数据
type ProtocolYield struct {
    Protocol    string    `json:"protocol"`
    APY         float64   `json:"apy"`
    TVL         *big.Int  `json:"tvl"`
    RiskScore   float64   `json:"risk_score"`
    Timestamp   time.Time `json:"timestamp"`
}

// 资产配置方案
type AllocationPlan struct {
    Allocations []Allocation `json:"allocations"`
    ExpectedAPY float64       `json:"expected_apy"`
    TotalRisk   float64       `json:"total_risk"`
    GasCost     *big.Int      `json:"gas_cost"`
    NetBenefit  *big.Int      `json:"net_benefit"`
}

type Allocation struct {
    Protocol    string    `json:"protocol"`
    Percentage  float64   `json:"percentage"`
    Amount      *big.Int  `json:"amount"`
}

// 启动收益引擎
func (e *YieldEngine) Start() error {
    e.logger.Info("Starting yield engine")

    ticker := time.NewTicker(e.rebalanceInterval)
    defer ticker.Stop()

    for {
```

```

select {
case <-ticker.C:
    if err := e.evaluateAndRebalance(); err != nil {
        e.logger.Error("Failed to evaluate and rebalance",
            zap.Error(err))
    }
}
}

// 评估并执行再平衡
func (e *YieldEngine) evaluateAndRebalance() error {
    e.logger.Info("Starting yield evaluation")

    // 1. 抓取各协议收益率
    yields, err := e.fetchProtocolYields()
    if err != nil {
        return fmt.Errorf("failed to fetch yields: %w", err)
    }

    // 2. 计算当前配置
    currentAllocation, err := e.getCurrentAllocation()
    if err != nil {
        return fmt.Errorf("failed to get current allocation: %w", err)
    }

    // 3. 生成最优配置
    optimalAllocation, err := e.calculateOptimalAllocation(yields)
    if err != nil {
        return fmt.Errorf("failed to calculate optimal allocation: %w", err)
    }

    // 4. 比较配置差异
    needRebalance, diff := e.compareAllocations(
        currentAllocation,
        optimalAllocation,
    )

    if !needRebalance {
        e.logger.Info("No rebalance needed", zap.Float64("diff", diff))
        return nil
    }

    e.logger.Info("Rebalance needed",
        zap.Float64("diff", diff),
        zap.Float64("expected_apy", optimalAllocation.ExpectedAPY),
    )

    // 5. 风险检查
    if optimalAllocation.TotalRisk > e.maxRiskScore {
        e.logger.Warn("Risk score too high",
            zap.Float64("risk", optimalAllocation.TotalRisk),

```

```

    )
    return nil
}

// 6. 计算 Gas 成本
gasCost, err := e.estimateGasCost(currentAllocation, optimalAllocation)
if err != nil {
    return fmt.Errorf("failed to estimate gas cost: %w", err)
}
optimalAllocation.GasCost = gasCost

// 7. 计算净收益
netBenefit := e.calculateNetBenefit(
    currentAllocation,
    optimalAllocation,
    gasCost,
)
optimalAllocation.NetBenefit = netBenefit

if netBenefit.Cmp(big.NewInt(0)) <= 0 {
    e.logger.Info("Net benefit not positive",
        zap.String("net_benefit", netBenefit.String()),
    )
    return nil
}

// 8. 生成再平衡交易
transactions, err := e.generateRebalanceTransactions(
    currentAllocation,
    optimalAllocation,
)
if err != nil {
    return fmt.Errorf("failed to generate transactions: %w", err)
}

// 9. 保存方案到数据库 (等待多签审批)
if err := e.saveRebalancePlan(optimalAllocation, transactions); err != nil {
    return fmt.Errorf("failed to save plan: %w", err)
}

// 10. 发送通知
e.sendRebalanceNotification(optimalAllocation)

e.logger.Info("Rebalance plan generated successfully")
return nil
}

// 抓取协议收益率
func (e *YieldEngine) fetchProtocolYields() ([]ProtocolYield, error) {
    var yields []ProtocolYield

    // 1. Aave USDC APY

```

```

aaveAPY, err := e.aaveClient.GetSupplyAPY("USDC")
if err != nil {
    return nil, fmt.Errorf("failed to get Aave APY: %w", err)
}

aaveTVL, err := e.aaveClient.GetTVL("USDC")
if err != nil {
    return nil, fmt.Errorf("failed to get Aave TVL: %w", err)
}

yields = append(yields, ProtocolYield{
    Protocol: "Aave",
    APY:      aaveAPY,
    TVL:      aaveTVL,
    RiskScore: 3.0, // 低风险
    Timestamp: time.Now(),
})

// 2. Compound USDC APY
compoundAPY, err := e.compoundClient.GetSupplyAPY("USDC")
if err != nil {
    return nil, fmt.Errorf("failed to get Compound APY: %w", err)
}

compoundTVL, err := e.compoundClient.GetTVL("USDC")
if err != nil {
    return nil, fmt.Errorf("failed to get Compound TVL: %w", err)
}

yields = append(yields, ProtocolYield{
    Protocol: "Compound",
    APY:      compoundAPY,
    TVL:      compoundTVL,
    RiskScore: 3.5, // 低风险
    Timestamp: time.Now(),
})

// 3. RWA 资产收益率
rwaAPY, err := e.rwaClient.GetYield()
if err != nil {
    return nil, fmt.Errorf("failed to get RWA yield: %w", err)
}

rwaTVL, err := e.rwaClient.GetTVL()
if err != nil {
    return nil, fmt.Errorf("failed to get RWA TVL: %w", err)
}

yields = append(yields, ProtocolYield{
    Protocol: "RWA",
    APY:      rwaAPY,
    TVL:      rwaTVL,

```



```

        RiskScore: 5.0, // 中等风险
        Timestamp: time.Now(),
    })

    // 保存收益率历史
    for _, y := range yields {
        if err := e.saveYieldHistory(&y); err != nil {
            e.logger.Error("Failed to save yield history", zap.Error(err))
        }
    }

    return yields, nil
}

// 计算最优配置（使用现代投资组合理论）
func (e *YieldEngine) calculateOptimalAllocation(
    yields []ProtocolYield,
) (*AllocationPlan, error) {

    // 简化的 MPT 实现：最大化夏普比率
    // Sharpe Ratio = (Expected Return - Risk Free Rate) / Risk

    // 获取总可用资金
    totalFunds, err := e.getTotalFunds()
    if err != nil {
        return nil, fmt.Errorf("failed to get total funds: %w", err)
    }

    // 按风险调整后收益排序
    type ScoredProtocol struct {
        Protocol      string
        APY            float64
        RiskScore      float64
        AdjustedScore float64
    }

    var scored []ScoredProtocol
    for _, y := range yields {
        // 风险调整后收益 = APY / RiskScore
        adjustedScore := y.APY / y.RiskScore
        scored = append(scored, ScoredProtocol{
            Protocol:      y.Protocol,
            APY:            y.APY,
            RiskScore:      y.RiskScore,
            AdjustedScore: adjustedScore,
        })
    }

    // 排序
    sort.Slice(scored, func(i, j int) bool {
        return scored[i].AdjustedScore > scored[j].AdjustedScore
    })
}

```

```

// 生成配置 (简化版: 按风险调整后收益分配)
var allocations []Allocation
var totalWeight float64

for _, s := range scored {
    totalWeight += s.AdjustedScore
}

var expectedAPY float64
var totalRisk float64

for _, s := range scored {
    percentage := s.AdjustedScore / totalWeight
    amount := new(big.Int).Mul(
        totalFunds,
        big.NewInt(int64(percentage * 10000)),
    )
    amount.Div(amount, big.NewInt(10000))

    allocations = append(allocations, Allocation{
        Protocol:  s.Protocol,
        Percentage: percentage,
        Amount:     amount,
    })

    expectedAPY += s.APY * percentage
    totalRisk += s.RiskScore * percentage
}

return &AllocationPlan{
    Allocations: allocations,
    ExpectedAPY: expectedAPY,
    TotalRisk:   totalRisk,
}, nil
}

// 生成再平衡交易
func (e *YieldEngine) generateRebalanceTransactions(
    current *AllocationPlan,
    optimal *AllocationPlan,
) ([]Transaction, error) {

    var transactions []Transaction

    // 计算每个协议的资金变动
    changes := make(map[string]*big.Int)

    for _, alloc := range optimal.Allocations {
        changes[alloc.Protocol] = alloc.Amount
    }

```

```

for _, alloc := range current.Allocations {
    if existing, ok := changes[alloc.Protocol]; ok {
        changes[alloc.Protocol] = new(big.Int).Sub(existing, alloc.Amount)
    } else {
        changes[alloc.Protocol] = new(big.Int).Neg(alloc.Amount)
    }
}

// 生成提取交易（负数变动）
for protocol, change := range changes {
    if change.Cmp(big.NewInt(0)) < 0 {
        withdrawAmount := new(big.Int).Neg(change)

        tx := Transaction{
            Type:      "withdraw",
            Protocol: protocol,
            Amount:    withdrawAmount,
        }
        transactions = append(transactions, tx)
    }
}

// 生成存入交易（正数变动）
for protocol, change := range changes {
    if change.Cmp(big.NewInt(0)) > 0 {
        tx := Transaction{
            Type:      "deposit",
            Protocol: protocol,
            Amount:    change,
        }
        transactions = append(transactions, tx)
    }
}

return transactions, nil
}

```

收益率历史追踪：

```

// 保存收益率历史到 TimescaleDB
func (e *YieldEngine) saveYieldHistory(yield *ProtocolYield) error {
    history := &models.YieldHistory{
        Protocol: yield.Protocol,
        APY:      yield.APY,
        TVL:      yield.TVL.String(),
        RiskScore: yield.RiskScore,
        Timestamp: yield.Timestamp,
    }

    if err := e.db.Create(history).Error; err != nil {
        return fmt.Errorf("failed to save yield history: %w", err)
    }
}

```

```

    }

    return nil
}

// 查询历史收益率趋势
func (e *YieldEngine) GetYieldTrend(
    protocol string,
    startTime time.Time,
    endTime time.Time,
) ([]ProtocolYield, error) {

    var history []models.YieldHistory

    err := e.db.Where("protocol = ? AND timestamp BETWEEN ? AND ?",
        protocol, startTime, endTime,
    ).Order("timestamp ASC").Find(&history).Error

    if err != nil {
        return nil, fmt.Errorf("failed to query yield history: %w", err)
    }

    var yields []ProtocolYield
    for _, h := range history {
        tvl := new(big.Int)
        tvl.SetString(h.TVL, 10)

        yields = append(yields, ProtocolYield{
            Protocol: h.Protocol,
            APY:      h.APY,
            TVL:      tvl,
            RiskScore: h.RiskScore,
            Timestamp: h.Timestamp,
        })
    }

    return yields, nil
}

```

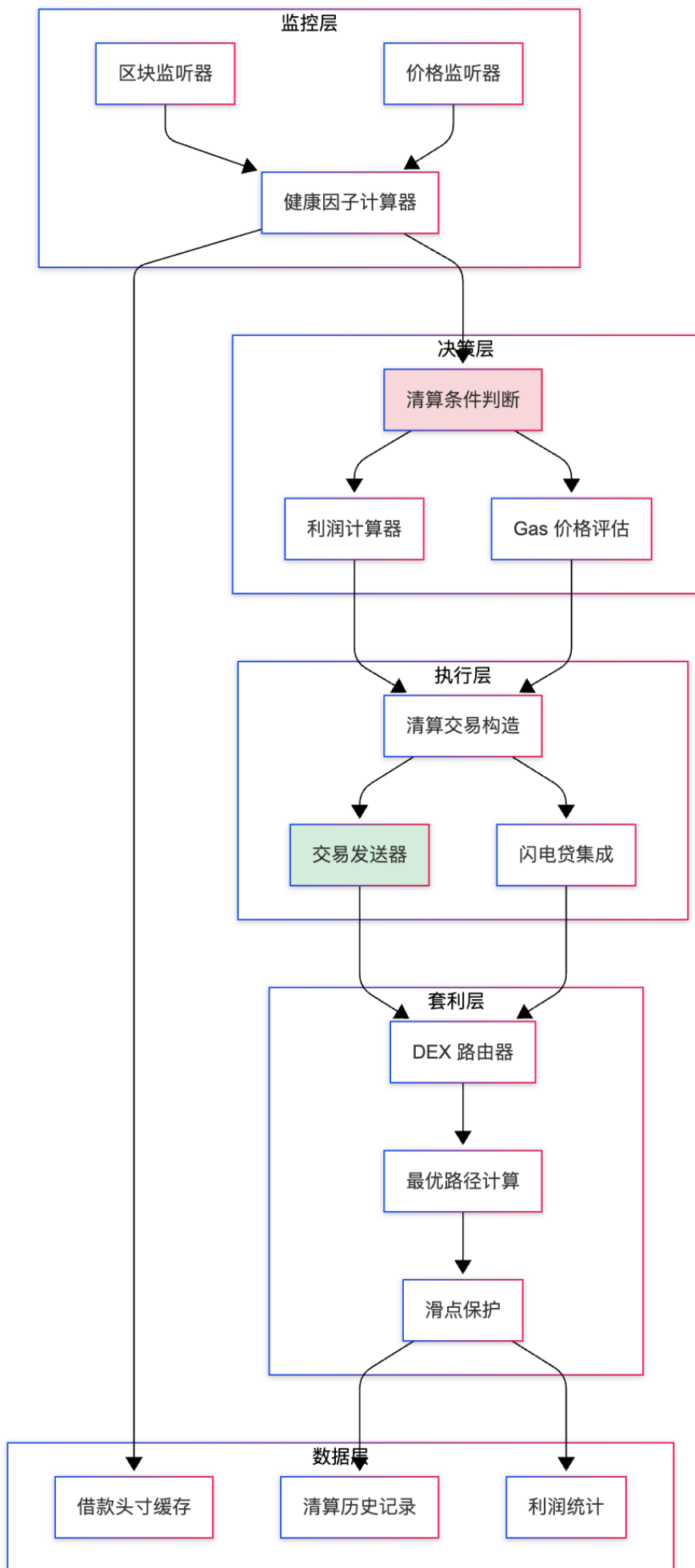
六、清算机器人实现

6.1 清算机器人架构

服务职责：

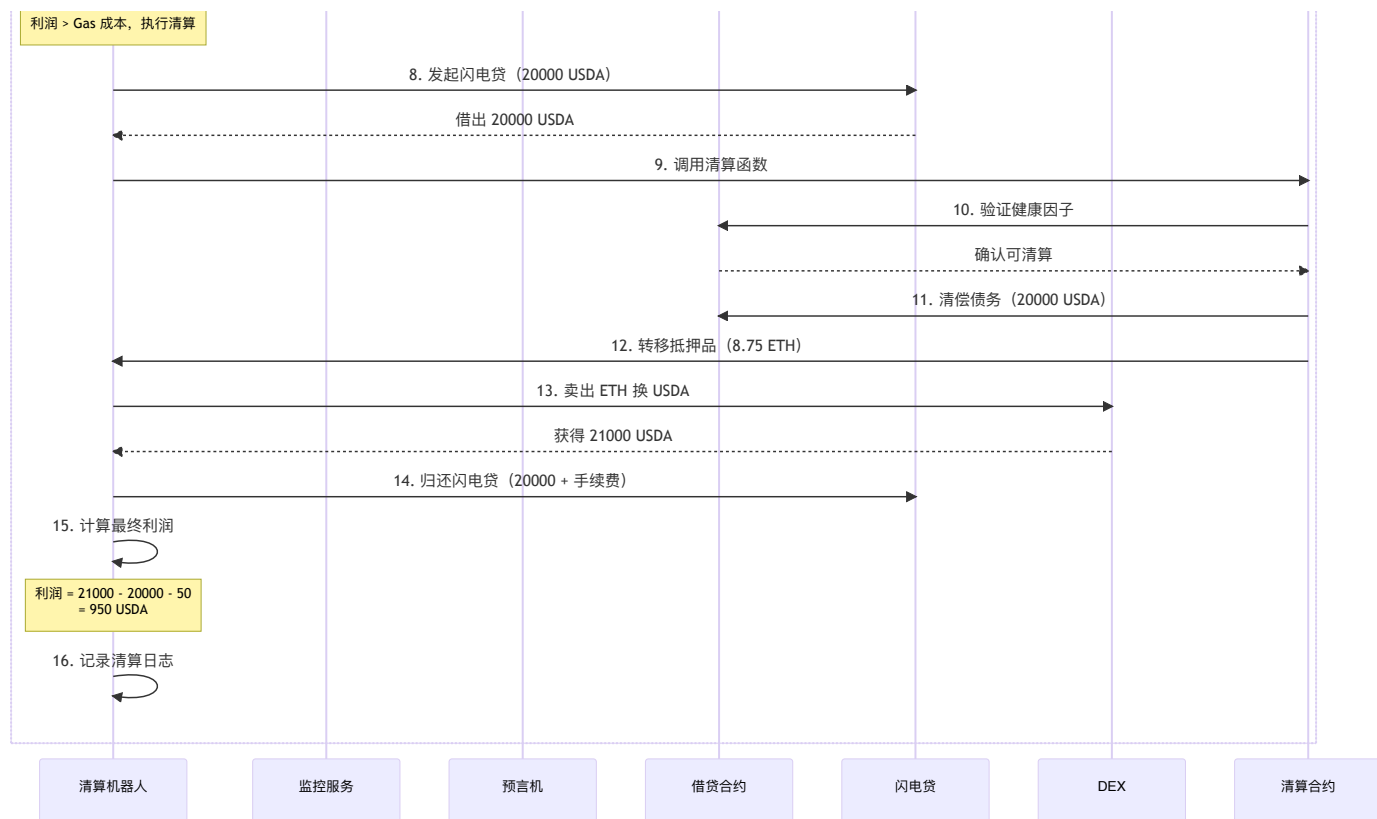
清算机器人（LiquidationBot）是保护协议安全的关键组件，负责实时监控所有借款头寸的健康因子，当健康因子低于阈值时自动执行清算，防止协议出现坏账。

清算机器人架构：



清算流程：





6.2 清算机器人核心代码

1. 监控服务实现：

```
// pkg/liquidation/monitor.go

type LiquidationMonitor struct {
    chainClient    *blockchain.ChainClient
    oracleClient    *oracle.Client
    borrowContract *borrow.Borrow

    db          *gorm.DB
    redis        *redis.Client
    logger       *zap.Logger

    // 配置
    minProfit      *big.Int // 最小利润阈值
    maxGasPrice    *big.Int // 最大 Gas 价格
    scanInterval   time.Duration
}

// 借款头寸
type BorrowPosition struct {
    User            common.Address
    CollateralType  string
    CollateralAmount *big.Int
    BorrowedAmount  *big.Int
    HealthFactor     *big.Float
    LastUpdateBlock uint64
}

// 启动监控
func (m *LiquidationMonitor) Start() error {
    m.logger.Info("Starting liquidation monitor")

    // 启动区块监听
    go m.monitorBlocks()

    // 启动价格监听
    go m.monitorPrices()

    // 启动清算执行
    go m.executeLiquidations()

    return nil
}

// 监控区块
func (m *LiquidationMonitor) monitorBlocks() {
    ticker := time.NewTicker(m.scanInterval)
    defer ticker.Stop()

    for {
        select {
```



```

        case <-ticker.C:
            if err := m.scanPositions(); err != nil {
                m.logger.Error("Failed to scan positions", zap.Error(err))
            }
        }
    }
}

// 扫描所有借款头寸
func (m *LiquidationMonitor) scanPositions() error {
    m.logger.Debug("Scanning borrow positions")

    // 1. 从合约获取所有借款用户
    users, err := m.borrowContract.GetAllBorrowers(nil)
    if err != nil {
        return fmt.Errorf("failed to get borrowers: %w", err)
    }

    m.logger.Info("Found borrowers", zap.Int("count", len(users)))

    // 2. 并发处理每个用户的头寸
    var wg sync.WaitGroup
    semaphore := make(chan struct{}, 10) // 限制并发数

    for _, user := range users {
        wg.Add(1)
        semaphore <- struct{}{}

        go func(userAddr common.Address) {
            defer wg.Done()
            defer func() { <-semaphore }()

            if err := m.checkUserPosition(userAddr); err != nil {
                m.logger.Error("Failed to check user position",
                    zap.String("user", userAddr.Hex()),
                    zap.Error(err),
                )
            }
        }(user)
    }

    wg.Wait()

    return nil
}

// 检查用户头寸
func (m *LiquidationMonitor) checkUserPosition(user common.Address) error {
    // 1. 获取用户抵押品信息
    collateral, err := m.borrowContract.GetUserCollateral(nil, user)
    if err != nil {
        return fmt.Errorf("failed to get collateral: %w", err)
    }
}

```

```

}

if collateral.Amount.Cmp(big.NewInt(0)) == 0 {
    return nil // 没有抵押品, 跳过
}

// 2. 获取用户借款信息
borrowed, err := m.borrowContract.GetUserBorrow(nil, user)
if err != nil {
    return fmt.Errorf("failed to get borrow: %w", err)
}

if borrowed.Amount.Cmp(big.NewInt(0)) == 0 {
    return nil // 没有借款, 跳过
}

// 3. 获取抵押品价格
price, err := m.oracleClient.GetPrice(collateral.Token)
if err != nil {
    return fmt.Errorf("failed to get price: %w", err)
}

// 4. 计算健康因子
healthFactor := m.calculateHealthFactor(
    collateral.Amount,
    price,
    collateral.LiquidationThreshold,
    borrowed.Amount,
)

m.logger.Debug("User health factor",
    zap.String("user", user.Hex()),
    zap.String("health_factor", healthFactor.String()),
)

// 5. 检查是否需要清算
if healthFactor.Cmp(big.NewFloat(1.0)) < 0 {
    m.logger.Info("Position can be liquidated",
        zap.String("user", user.Hex()),
        zap.String("health_factor", healthFactor.String()),
    )

    // 创建清算任务
    position := &BorrowPosition{
        User:            user,
        CollateralType:   collateral.Token.Hex(),
        CollateralAmount: collateral.Amount,
        BorrowedAmount:   borrowed.Amount,
        HealthFactor:     healthFactor,
        LastUpdateBlock: m.chainClient.GetCurrentBlock(),
    }
}

```

```

        if err := m.queueLiquidation(position); err != nil {
            return fmt.Errorf("failed to queue liquidation: %w", err)
        }
    }

    // 6. 更新头寸缓存
    if err := m.cachePosition(user, healthFactor); err != nil {
        m.logger.Error("Failed to cache position", zap.Error(err))
    }

    return nil
}

// 计算健康因子
func (m *LiquidationMonitor) calculateHealthFactor(
    collateralAmount *big.Int,
    collateralPrice *big.Int,
    liquidationThreshold *big.Int,
    borrowedAmount *big.Int,
) *big.Float {

    // 健康因子 = (抵押品数量 × 价格 × 清算阈值) / 借款金额

    // 计算抵押品价值
    collateralValue := new(big.Int).Mul(collateralAmount, collateralPrice)
    collateralValue.Div(collateralValue, big.NewInt(1e18)) // 调整精度

    // 应用清算阈值
    adjustedValue := new(big.Int).Mul(collateralValue, liquidationThreshold)
    adjustedValue.Div(adjustedValue, big.NewInt(10000)) // 清算阈值是基点

    // 计算健康因子
    healthFactor := new(big.Float).SetInt(adjustedValue)
    borrowedFloat := new(big.Float).SetInt(borrowedAmount)
    healthFactor.Quo(healthFactor, borrowedFloat)

    return healthFactor
}

// 将清算任务加入队列
func (m *LiquidationMonitor) queueLiquidation(position *BorrowPosition) error {
    // 序列化头寸数据
    data, err := json.Marshal(position)
    if err != nil {
        return fmt.Errorf("failed to marshal position: %w", err)
    }

    // 加入 Redis 队列
    ctx := context.Background()
    key := "liquidation_queue"

    if err := m.redis.LPush(ctx, key, data).Err(); err != nil {

```

```

        return fmt.Errorf("failed to push to queue: %w", err)
    }

    m.logger.Info("Liquidation queued",
        zap.String("user", position.User.Hex()),
    )

    return nil
}

```

2. 清算执行器实现:

```

// pkg/liquidation/executor.go

type LiquidationExecutor struct {
    chainClient      *blockchain.ChainClient
    liquidationContract *liquidation.Liquidation
    flashLoanContract *flashloan.FlashLoan
    dexRouter        *dex.Router

    privateKey      *ecdsa.PrivateKey
    executorAddress common.Address

    db      *gorm.DB
    redis    *redis.Client
    logger   *zap.Logger

    // 配置
    minProfit      *big.Int
    maxGasPrice    *big.Int
    slippageTolerance float64
}

// 执行清算
func (e *LiquidationExecutor) executeLiquidations() {
    for {
        // 从队列获取清算任务
        position, err := e.dequeueLiquidation()
        if err != nil {
            if err != redis.Nil {
                e.logger.Error("Failed to dequeue liquidation", zap.Error(err))
            }
            time.Sleep(1 * time.Second)
            continue
        }

        // 执行清算
        if err := e.liquidate(position); err != nil {
            e.logger.Error("Failed to liquidate",
                zap.String("user", position.User.Hex()),
                zap.Error(err),
            )
        }
    }
}

```

```

        )

        // 记录失败
        e.recordFailure(position, err)
    }
}

// 从队列获取清算任务
func (e *LiquidationExecutor) dequeueLiquidation() (*BorrowPosition, error) {
    ctx := context.Background()
    key := "liquidation_queue"

    // 从 Redis 队列弹出
    result, err := e.redis.RPop(ctx, key).Result()
    if err != nil {
        return nil, err
    }

    // 反序列化
    var position BorrowPosition
    if err := json.Unmarshal([]byte(result), &position); err != nil {
        return nil, fmt.Errorf("failed to unmarshal position: %w", err)
    }

    return &position, nil
}

// 执行清算
func (e *LiquidationExecutor) liquidate(position *BorrowPosition) error {
    e.logger.Info("Executing liquidation",
        zap.String("user", position.User.Hex()),
        zap.String("borrowed", position.BorrowedAmount.String()),
    )

    // 1. 再次验证健康因子 (防止抢跑)
    currentHealthFactor, err := e.getCurrentHealthFactor(position.User)
    if err != nil {
        return fmt.Errorf("failed to get health factor: %w", err)
    }

    if currentHealthFactor.Cmp(big.NewFloat(1.0)) >= 0 {
        e.logger.Info("Position no longer liquidatable",
            zap.String("health_factor", currentHealthFactor.String()),
        )
        return nil
    }

    // 2. 计算预期利润
    profit, err := e.calculateExpectedProfit(position)
    if err != nil {
        return fmt.Errorf("failed to calculate profit: %w", err)
    }
}

```

```

}

if profit.Cmp(e.minProfit) < 0 {
    e.logger.Info("Profit too low",
        zap.String("profit", profit.String()),
        zap.String("min_profit", e.minProfit.String()),
    )
    return nil
}

// 3. 检查 Gas 价格
gasPrice, err := e.chainClient.SuggestGasPrice()
if err != nil {
    return fmt.Errorf("failed to get gas price: %w", err)
}

if gasPrice.Cmp(e.maxGasPrice) > 0 {
    e.logger.Warn("Gas price too high",
        zap.String("gas_price", gasPrice.String()),
    )
    // 重新加入队列
    return e.requeueLiquidation(position)
}

// 4. 构造清算交易
tx, err := e.buildLiquidationTx(position, gasPrice)
if err != nil {
    return fmt.Errorf("failed to build tx: %w", err)
}

// 5. 签名交易
signedTx, err := e.signTransaction(tx)
if err != nil {
    return fmt.Errorf("failed to sign tx: %w", err)
}

// 6. 发送交易
txHash, err := e.chainClient.SendTransaction(signedTx)
if err != nil {
    return fmt.Errorf("failed to send tx: %w", err)
}

e.logger.Info("Liquidation transaction sent",
    zap.String("tx_hash", txHash.Hex()),
)

// 7. 等待交易确认
receipt, err := e.waitForConfirmation(txHash)
if err != nil {
    return fmt.Errorf("failed to wait for confirmation: %w", err)
}

```

```

if receipt.Status == 0 {
    return fmt.Errorf("transaction failed")
}

// 8. 计算实际利润
actualProfit, err := e.calculateActualProfit(receipt)
if err != nil {
    e.logger.Error("Failed to calculate actual profit", zap.Error(err))
}

// 9. 记录清算成功
if err := e.recordSuccess(position, txHash, actualProfit); err != nil {
    e.logger.Error("Failed to record success", zap.Error(err))
}

e.logger.Info("Liquidation successful",
    zap.String("tx_hash", txHash.Hex()),
    zap.String("profit", actualProfit.String()),
)

return nil
}

// 计算预期利润
func (e *LiquidationExecutor) calculateExpectedProfit(
    position *BorrowPosition,
) (*big.Int, error) {

    // 1. 获取抵押品价格
    collateralToken := common.HexToAddress(position.CollateralType)
    collateralPrice, err := e.chainClient.GetPrice(collateralToken)
    if err != nil {
        return nil, fmt.Errorf("failed to get collateral price: %w", err)
    }

    // 2. 计算清算金额 (需要偿还的债务)
    liquidationAmount := position.BorrowedAmount

    // 3. 计算可获得的抵押品数量
    // 抵押品数量 = 清算金额 / 抵押品价格
    collateralReceived := new(big.Int).Mul(
        liquidationAmount,
        big.NewInt(1e18),
    )
    collateralReceived.Div(collateralReceived, collateralPrice)

    // 4. 加上清算奖励 (如 5%)
    liquidationBonus := big.NewInt(500) // 5% = 500 基点
    bonusAmount := new(big.Int).Mul(collateralReceived, liquidationBonus)
    bonusAmount.Div(bonusAmount, big.NewInt(10000))
    collateralReceived.Add(collateralReceived, bonusAmount)
}

```

```

// 5. 计算抵押品价值
collateralValue := new(big.Int).Mul(collateralReceived, collateralPrice)
collateralValue.Div(collateralValue, big.NewInt(1e18))

// 6. 计算利润 (抵押品价值 - 清算金额)
profit := new(big.Int).Sub(collateralValue, liquidationAmount)

// 7. 减去预估的 Gas 成本
gasPrice, _ := e.chainClient.SuggestGasPrice()
gasLimit := big.NewInt(500000) // 预估 Gas Limit
gasCost := new(big.Int).Mul(gasPrice, gasLimit)

// 将 Gas 成本转换为 USD (假设 ETH = $3000)
ethPrice := big.NewInt(3000)
gasCostUSD := new(big.Int).Mul(gasCost, ethPrice)
gasCostUSD.Div(gasCostUSD, big.NewInt(1e18))

profit.Sub(profit, gasCostUSD)

// 8. 减去 DEX 交易滑点 (假设 0.5%)
slippage := new(big.Int).Mul(profit, big.NewInt(50))
slippage.Div(slippage, big.NewInt(10000))
profit.Sub(profit, slippage)

return profit, nil
}

// 构造清算交易 (使用闪电贷)
func (e *LiquidationExecutor) buildLiquidationTx(
    position *BorrowPosition,
    gasPrice *big.Int,
) (*types.Transaction, error) {

    // 构造闪电贷参数
    // 闪电贷会在一个交易中完成:
    // 1. 借出 USDA
    // 2. 调用清算合约
    // 3. 卖出获得的抵押品
    // 4. 归还闪电贷

    // 编码清算调用数据
    liquidationData, err := e.encodeLiquidationCall(position)
    if err != nil {
        return nil, fmt.Errorf("failed to encode liquidation: %w", err)
    }

    // 编码 DEX 交易数据
    dexData, err := e.encodeDEXSwap(position)
    if err != nil {
        return nil, fmt.Errorf("failed to encode dex swap: %w", err)
    }
}

```



```

// 构造闪电贷调用
flashLoanData := FlashLoanParams{
    Asset:          e.chainClient.USDAAddress(),
    Amount:         position.BorrowedAmount,
    LiquidationData: liquidationData,
    DEXData:        dexData,
}

data, err := e.flashLoanContract.ABI.Pack(
    "executeFlashLoan",
    flashLoanData,
)
if err != nil {
    return nil, fmt.Errorf("failed to pack flash loan: %w", err)
}

// 获取 nonce
nonce, err := e.chainClient.PendingNonceAt(e.executorAddress)
if err != nil {
    return nil, fmt.Errorf("failed to get nonce: %w", err)
}

// 构造交易
tx := types.NewTransaction(
    nonce,
    e.flashLoanContract.Address(),
    big.NewInt(0),
    500000, // Gas Limit
    gasPrice,
    data,
)

return tx, nil
}

// 签名交易
func (e *LiquidationExecutor) signTransaction(
    tx *types.Transaction,
) (*types.Transaction, error) {

    chainID, err := e.chainClient.ChainID()
    if err != nil {
        return nil, fmt.Errorf("failed to get chain ID: %w", err)
    }

    signer := types.NewEIP155Signer(chainID)
    signedTx, err := types.SignTx(tx, signer, e.privateKey)
    if err != nil {
        return nil, fmt.Errorf("failed to sign tx: %w", err)
    }

    return signedTx, nil
}

```

```

}

// 等待交易确认
func (e *LiquidationExecutor) waitForConfirmation(
    txHash common.Hash,
) (*types.Receipt, error) {

    timeout := time.After(5 * time.Minute)
    ticker := time.NewTicker(2 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-timeout:
            return nil, fmt.Errorf("transaction confirmation timeout")

        case <-ticker.C:
            receipt, err := e.chainClient.TransactionReceipt(txHash)
            if err != nil {
                if err == ethereum.NotFound {
                    continue
                }
                return nil, fmt.Errorf("failed to get receipt: %w", err)
            }

            // 等待至少 1 个确认
            currentBlock, err := e.chainClient.BlockNumber()
            if err != nil {
                return nil, fmt.Errorf("failed to get block number: %w", err)
            }

            if currentBlock >= receipt.BlockNumber.Uint64()+1 {
                return receipt, nil
            }
        }
    }
}

// 记录清算成功
func (e *LiquidationExecutor) recordSuccess(
    position *BorrowPosition,
    txHash common.Hash,
    profit *big.Int,
) error {

    record := &models.LiquidationRecord{
        ChainID:      e.chainClient.ChainID(),
        TxHash:       txHash.Hex(),
        User:         position.User.Hex(),
        CollateralType: position.CollateralType,
        CollateralAmount: position.CollateralAmount.String(),
        BorrowedAmount: position.BorrowedAmount.String(),
    }

```

```

        Profit:          profit.String(),
        Status:          "success",
        ExecutedAt:      time.Now(),
    }

    if err := e.db.Create(record).Error; err != nil {
        return fmt.Errorf("failed to save record: %w", err)
    }

    return nil
}

```

3. 利润优化策略：

```

// pkg/liquidation/optimizer.go

type LiquidationOptimizer struct {
    dexRouter      *dex.Router
    aggregator      *aggregator.Client
    logger         *zap.Logger
}

// 计算最优 DEX 路径
func (o *LiquidationOptimizer) FindBestSwapPath(
    tokenIn common.Address,
    tokenOut common.Address,
    amountIn *big.Int,
) (*SwapPath, error) {

    // 查询多个 DEX 的报价
    paths := []SwapPath{}

    // 1. Uniswap V3
    uniswapQuote, err := o.dexRouter.GetQuote("uniswap_v3", tokenIn, tokenOut, amountIn)
    if err == nil {
        paths = append(paths, SwapPath{
            DEX:      "uniswap_v3",
            AmountOut: uniswapQuote,
            Route:     []common.Address{tokenIn, tokenOut},
        })
    }

    // 2. Curve
    curveQuote, err := o.dexRouter.GetQuote("curve", tokenIn, tokenOut, amountIn)
    if err == nil {
        paths = append(paths, SwapPath{
            DEX:      "curve",
            AmountOut: curveQuote,
            Route:     []common.Address{tokenIn, tokenOut},
        })
    }
}

```

```

// 3. linch 聚合器
inchQuote, err := o.aggregator.GetBestQuote(tokenIn, tokenOut, amountIn)
if err == nil {
    paths = append(paths, SwapPath{
        DEX:      "linch",
        AmountOut: inchQuote.AmountOut,
        Route:     inchQuote.Route,
    })
}

// 选择输出金额最大的路径
if len(paths) == 0 {
    return nil, fmt.Errorf("no swap path found")
}

bestPath := &paths[0]
for i := 1; i < len(paths); i++ {
    if paths[i].AmountOut.Cmp(bestPath.AmountOut) > 0 {
        bestPath = &paths[i]
    }
}

o.logger.Info("Best swap path found",
    zap.String("dex", bestPath.DEX),
    zap.String("amount_out", bestPath.AmountOut.String()),
)

return bestPath, nil
}

```

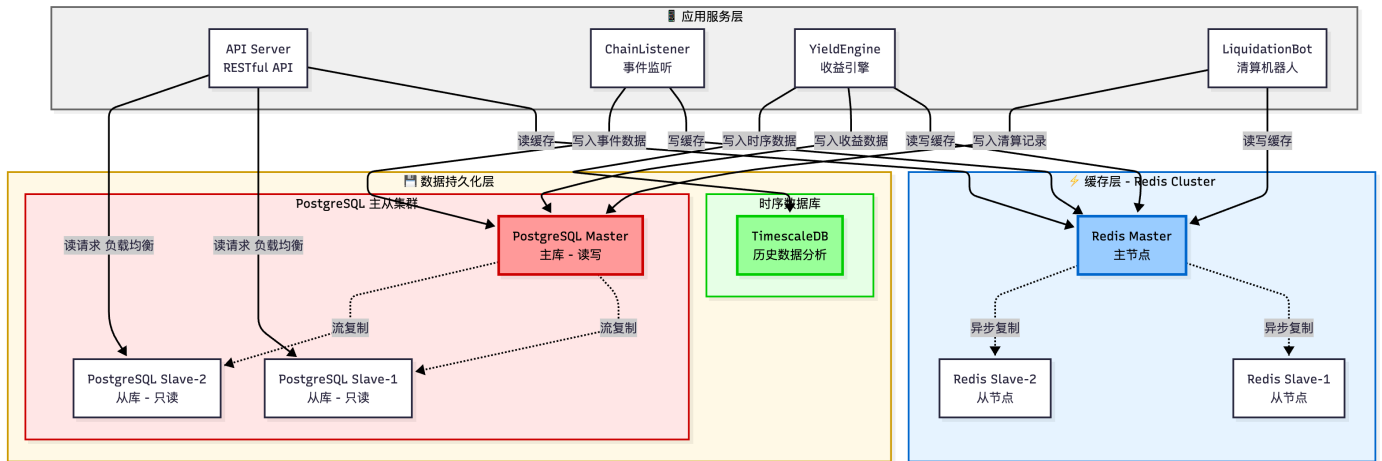
七、数据库设计

7.1 数据库架构

技术选型：

1. **PostgreSQL**：主数据库，存储交易记录、用户数据、配置信息
2. **Redis**：缓存层，存储热点数据、任务队列
3. **TimescaleDB**：时序数据库，存储历史价格、收益率、监控指标

数据库架构：



7.2 核心表结构设计

1. 链同步状态表 (chain_sync_status) :

```
CREATE TABLE chain_sync_status (
    id SERIAL PRIMARY KEY,
    chain_id BIGINT NOT NULL UNIQUE,
    chain_name VARCHAR(50) NOT NULL,
    last_processed_block BIGINT NOT NULL DEFAULT 0,
    current_block BIGINT NOT NULL DEFAULT 0,
    sync_status VARCHAR(20) NOT NULL DEFAULT 'syncing',
    error_message TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_chain_sync_chain_id ON chain_sync_status(chain_id);
CREATE INDEX idx_chain_sync_status ON chain_sync_status(sync_status);

COMMENT ON TABLE chain_sync_status IS '多链同步状态表';
COMMENT ON COLUMN chain_sync_status.sync_status IS '同步状态: syncing, synced, error';
```

2. 交易记录表 (transactions) :

```
CREATE TABLE transactions (
    id BIGSERIAL PRIMARY KEY,
    chain_id BIGINT NOT NULL,
    tx_hash VARCHAR(66) NOT NULL,
    block_number BIGINT NOT NULL,
    block_timestamp TIMESTAMP NOT NULL,

    tx_type VARCHAR(20) NOT NULL,
    user_address VARCHAR(42) NOT NULL,

    amount NUMERIC(78, 0) NOT NULL,
```

```

token_address VARCHAR(42),

from_address VARCHAR(42),
to_address VARCHAR(42),

gas_used BIGINT,
gas_price NUMERIC(78, 0),

status VARCHAR(20) NOT NULL DEFAULT 'success',
error_message TEXT,

created_at TIMESTAMP NOT NULL DEFAULT NOW(),

CONSTRAINT uk_tx_hash_chain UNIQUE (chain_id, tx_hash)
);

CREATE INDEX idx_tx_chain_id ON transactions(chain_id);
CREATE INDEX idx_tx_type ON transactions(tx_type);
CREATE INDEX idx_tx_user ON transactions(user_address);
CREATE INDEX idx_tx_block ON transactions(block_number);
CREATE INDEX idx_tx_timestamp ON transactions(block_timestamp);

COMMENT ON TABLE transactions IS '交易记录表';
COMMENT ON COLUMN transactions.tx_type IS '交易类型: mint, redeem, transfer, borrow, repay, liquidation';

```

3. 用户余额表 (user_balances) :

```

CREATE TABLE user_balances (
    id BIGSERIAL PRIMARY KEY,
    chain_id BIGINT NOT NULL,
    user_address VARCHAR(42) NOT NULL,
    token_type VARCHAR(20) NOT NULL,

    balance NUMERIC(78, 0) NOT NULL DEFAULT 0,

    last_tx_hash VARCHAR(66),
    last_tx_block BIGINT,

    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),

    CONSTRAINT uk_user_balance UNIQUE (chain_id, user_address, token_type)
);

CREATE INDEX idx_balance_chain_user ON user_balances(chain_id, user_address);
CREATE INDEX idx_balance_token ON user_balances(token_type);
CREATE INDEX idx_balance_amount ON user_balances(balance);

COMMENT ON TABLE user_balances IS '用户余额表';
COMMENT ON COLUMN user_balances.token_type IS '代币类型: USDA, EURA, stUSD, stEUR';

```

4. 全局统计表 (global_stats) :

```
CREATE TABLE global_stats (  
  id SERIAL PRIMARY KEY,  
  chain_id BIGINT NOT NULL UNIQUE,  
  
  total_supply NUMERIC(78, 0) NOT NULL DEFAULT 0,  
  total_minted NUMERIC(78, 0) NOT NULL DEFAULT 0,  
  total_redeemed NUMERIC(78, 0) NOT NULL DEFAULT 0,  
  
  total_users INTEGER NOT NULL DEFAULT 0,  
  total_transactions BIGINT NOT NULL DEFAULT 0,  
  
  tvl NUMERIC(78, 0) NOT NULL DEFAULT 0,  
  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_stats_chain ON global_stats(chain_id);  
  
COMMENT ON TABLE global_stats IS '全局统计表';
```

5. 储备金快照表 (reserve_snapshots) :

```
CREATE TABLE reserve_snapshots (  
  id BIGSERIAL PRIMARY KEY,  
  chain_id BIGINT NOT NULL,  
  
  usdc_balance NUMERIC(78, 0) NOT NULL,  
  aave_value NUMERIC(78, 0) NOT NULL,  
  compound_value NUMERIC(78, 0) NOT NULL,  
  rwa_value NUMERIC(78, 0) NOT NULL,  
  
  total_reserve_value NUMERIC(78, 0) NOT NULL,  
  total_supply NUMERIC(78, 0) NOT NULL,  
  
  collateral_ratio INTEGER NOT NULL,  
  surplus_buffer NUMERIC(78, 0) NOT NULL,  
  surplus_ratio INTEGER NOT NULL,  
  
  snapshot_time TIMESTAMP NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_reserve_chain ON reserve_snapshots(chain_id);  
CREATE INDEX idx_reserve_time ON reserve_snapshots(snapshot_time);  
  
COMMENT ON TABLE reserve_snapshots IS '储备金快照表';  
COMMENT ON COLUMN reserve_snapshots.collateral_ratio IS '抵押率 (基点)';  
COMMENT ON COLUMN reserve_snapshots.surplus_ratio IS '盈余缓冲率 (基点)';
```

6. 借款头寸表 (borrow_positions) :

```
CREATE TABLE borrow_positions (  
  id BIGSERIAL PRIMARY KEY,  
  chain_id BIGINT NOT NULL,  
  user_address VARCHAR(42) NOT NULL,  
  
  collateral_type VARCHAR(42) NOT NULL,  
  collateral_amount NUMERIC(78, 0) NOT NULL,  
  
  borrowed_amount NUMERIC(78, 0) NOT NULL,  
  interest_rate INTEGER NOT NULL,  
  
  health_factor NUMERIC(18, 6) NOT NULL,  
  liquidation_threshold INTEGER NOT NULL,  
  
  borrow_block BIGINT NOT NULL,  
  last_update_block BIGINT NOT NULL,  
  
  status VARCHAR(20) NOT NULL DEFAULT 'active',  
  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT uk_borrow_position UNIQUE (chain_id, user_address, collateral_type)  
);  
  
CREATE INDEX idx_borrow_chain_user ON borrow_positions(chain_id, user_address);  
CREATE INDEX idx_borrow_health ON borrow_positions(health_factor);  
CREATE INDEX idx_borrow_status ON borrow_positions(status);  
  
COMMENT ON TABLE borrow_positions IS '借款头寸表';  
COMMENT ON COLUMN borrow_positions.status IS '状态: active, liquidated, repaid';
```

7. 清算记录表 (liquidation_records) :

```
CREATE TABLE liquidation_records (  
  id BIGSERIAL PRIMARY KEY,  
  chain_id BIGINT NOT NULL,  
  tx_hash VARCHAR(66) NOT NULL,  
  
  user_address VARCHAR(42) NOT NULL,  
  liquidator_address VARCHAR(42) NOT NULL,  
  
  collateral_type VARCHAR(42) NOT NULL,  
  collateral_amount NUMERIC(78, 0) NOT NULL,  
  collateral_value NUMERIC(78, 0) NOT NULL,  
  
  borrowed_amount NUMERIC(78, 0) NOT NULL,  
  liquidation_bonus NUMERIC(78, 0) NOT NULL,
```



```

profit NUMERIC(78, 0) NOT NULL,
gas_cost NUMERIC(78, 0) NOT NULL,

health_factor_before NUMERIC(18, 6) NOT NULL,

status VARCHAR(20) NOT NULL,
error_message TEXT,

executed_at TIMESTAMP NOT NULL,
created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_liquidation_chain ON liquidation_records(chain_id);
CREATE INDEX idx_liquidation_user ON liquidation_records(user_address);
CREATE INDEX idx_liquidation_tx ON liquidation_records(tx_hash);
CREATE INDEX idx_liquidation_time ON liquidation_records(executed_at);

COMMENT ON TABLE liquidation_records IS '清算记录表';
COMMENT ON COLUMN liquidation_records.status IS '状态: success, failed, pending';

```

8. 收益率历史表 (yield_history) - TimescaleDB:

```

CREATE TABLE yield_history (
    time TIMESTAMPTZ NOT NULL,
    protocol VARCHAR(50) NOT NULL,

    apy NUMERIC(10, 4) NOT NULL,
    tvl NUMERIC(78, 0) NOT NULL,
    risk_score NUMERIC(4, 2) NOT NULL,

    PRIMARY KEY (time, protocol)
);

SELECT create_hypertable('yield_history', 'time');

CREATE INDEX idx_yield_protocol ON yield_history(protocol, time DESC);

COMMENT ON TABLE yield_history IS '收益率历史表 (时序数据)';

```

9. 价格历史表 (price_history) - TimescaleDB:

```

CREATE TABLE price_history (
    time TIMESTAMPTZ NOT NULL,
    asset VARCHAR(42) NOT NULL,
    source VARCHAR(50) NOT NULL,

    price NUMERIC(78, 0) NOT NULL,

    PRIMARY KEY (time, asset, source)
);

```

```

SELECT create_hypertable('price_history', 'time');

CREATE INDEX idx_price_asset ON price_history(asset, time DESC);
CREATE INDEX idx_price_source ON price_history(source, time DESC);

COMMENT ON TABLE price_history IS '价格历史表（时序数据）';

```

10. 系统监控指标表 (system_metrics) - TimescaleDB:

```

CREATE TABLE system_metrics (
    time TIMESTAMPTZ NOT NULL,
    service_name VARCHAR(50) NOT NULL,
    metric_name VARCHAR(100) NOT NULL,

    metric_value NUMERIC(20, 4) NOT NULL,
    tags JSONB,

    PRIMARY KEY (time, service_name, metric_name)
);

SELECT create_hypertable('system_metrics', 'time');

CREATE INDEX idx_metrics_service ON system_metrics(service_name, time DESC);
CREATE INDEX idx_metrics_name ON system_metrics(metric_name, time DESC);

COMMENT ON TABLE system_metrics IS '系统监控指标表（时序数据）';

```

7.3 数据库优化策略

1. 分区策略:

```

-- 对交易表按月分区
CREATE TABLE transactions_2024_01 PARTITION OF transactions
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE transactions_2024_02 PARTITION OF transactions
FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- 自动创建分区的函数
CREATE OR REPLACE FUNCTION create_monthly_partitions()
RETURNS void AS $$
DECLARE
    start_date DATE;
    end_date DATE;
    partition_name TEXT;
BEGIN
    start_date := DATE_TRUNC('month', CURRENT_DATE);
    end_date := start_date + INTERVAL '1 month';
    partition_name := 'transactions_' || TO_CHAR(start_date, 'YYYY_MM');

    EXECUTE format(

```

```

        'CREATE TABLE IF NOT EXISTS %I PARTITION OF transactions
        FOR VALUES FROM (%L) TO (%L)',
        partition_name, start_date, end_date
    );
END;
$$ LANGUAGE plpgsql;

```

2. 索引优化:

-- 复合索引优化查询

```

CREATE INDEX idx_tx_user_type_time ON transactions(
    user_address, tx_type, block_timestamp DESC
);

```

-- 部分索引（只索引活跃头寸）

```

CREATE INDEX idx_active_positions ON borrow_positions(health_factor)
WHERE status = 'active';

```

-- 表达式索引

```

CREATE INDEX idx_low_health_factor ON borrow_positions(
    (health_factor < 1.05)
) WHERE status = 'active';

```

3. 物化视图:

-- 用户统计物化视图

```

CREATE MATERIALIZED VIEW user_statistics AS
SELECT
    user_address,
    COUNT(DISTINCT chain_id) as chains_count,
    SUM(CASE WHEN tx_type = 'mint' THEN amount ELSE 0 END) as total_minted,
    SUM(CASE WHEN tx_type = 'redeem' THEN amount ELSE 0 END) as total_redeemed,
    COUNT(*) as tx_count,
    MIN(created_at) as first_tx_at,
    MAX(created_at) as last_tx_at
FROM transactions
GROUP BY user_address;

```

```

CREATE UNIQUE INDEX idx_user_stats_address ON user_statistics(user_address);

```

-- 定时刷新物化视图

```

CREATE OR REPLACE FUNCTION refresh_user_statistics()
RETURNS void AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY user_statistics;
END;
$$ LANGUAGE plpgsql;

```

4. 查询优化示例:

```

-- 高效查询用户交易历史
EXPLAIN ANALYZE
SELECT
    tx_hash,
    tx_type,
    amount,
    block_timestamp
FROM transactions
WHERE user_address = '0x1234...'
    AND chain_id = 1
    AND block_timestamp >= NOW() - INTERVAL '30 days'
ORDER BY block_timestamp DESC
LIMIT 100;

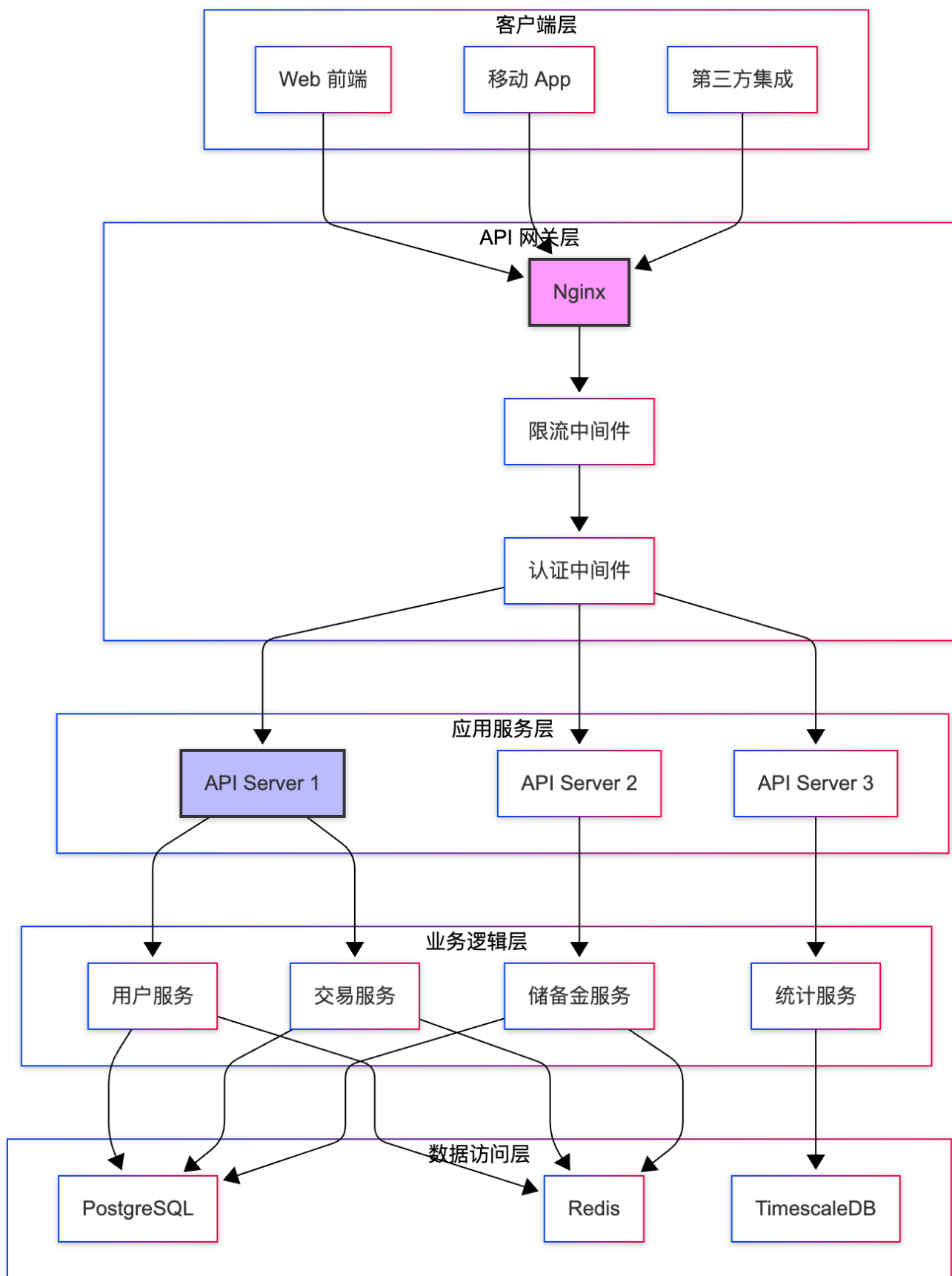
-- 使用 CTE 优化复杂查询
WITH user_positions AS (
    SELECT
        user_address,
        SUM(borrowed_amount) as total_borrowed,
        AVG(health_factor) as avg_health_factor
    FROM borrow_positions
    WHERE status = 'active'
    GROUP BY user_address
)
SELECT
    up.user_address,
    up.total_borrowed,
    up.avg_health_factor,
    ub.balance
FROM user_positions up
JOIN user_balances ub ON up.user_address = ub.user_address
WHERE up.avg_health_factor < 1.2
ORDER BY up.total_borrowed DESC;

```

八、RESTful API 设计

8.1 API 架构总览

API 服务架构：



8.2 核心 API 端点设计

API 路由设计:

```

/api/v1
├─ /health                # 健康检查
├─ /stats                 # 全局统计
│   └─ /global            # 全局统计
│   └─ /chains/:chainId   # 单链统计
│   └─ /tv1               # TVL 统计
├─ /reserves              # 储备金
│   └─ /snapshot          # 最新快照
│   └─ /history           # 历史数据
│   └─ /collateral-ratio  # 抵押率
├─ /users                 # 用户相关
│   └─ /:address/balances # 用户余额
│   └─ /:address/transactions # 用户交易
│   └─ /:address/positions # 用户借款头寸
├─ /transactions          # 交易记录
│   └─ /                  # 交易列表
│   └─ /:txHash           # 交易详情
│   └─ /recent            # 最近交易
├─ /liquidations          # 清算记录
│   └─ /                  # 清算列表
│   └─ /pending           # 待清算头寸
│   └─ /stats             # 清算统计
├─ /yields                # 收益相关
│   └─ /apy               # 当前 APY
│   └─ /history           # 历史收益率
│   └─ /allocation        # 资产配置
├─ /prices                # 价格相关
│   └─ /current           # 当前价格
│   └─ /history           # 历史价格

```

8.3 API 实现代码

1. 主服务器实现：

```

// cmd/api/main.go

package main

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
    "gorm.io/gorm"

```

```

    "angle-protocol/pkg/api/handlers"
    "angle-protocol/pkg/api/middleware"
    "angle-protocol/pkg/config"
    "angle-protocol/pkg/database"
    "angle-protocol/pkg/logger"
)

func main() {
    // 加载配置
    cfg, err := config.Load()
    if err != nil {
        panic(fmt.Sprintf("Failed to load config: %v", err))
    }

    // 初始化日志
    log, err := logger.NewLogger(cfg.Log)
    if err != nil {
        panic(fmt.Sprintf("Failed to init logger: %v", err))
    }
    defer log.Sync()

    // 连接数据库
    db, err := database.Connect(cfg.Database)
    if err != nil {
        log.Fatal("Failed to connect database", zap.Error(err))
    }

    // 连接 Redis
    redisClient, err := database.ConnectRedis(cfg.Redis)
    if err != nil {
        log.Fatal("Failed to connect redis", zap.Error(err))
    }

    // 初始化 Gin
    if cfg.Env == "production" {
        gin.SetMode(gin.ReleaseMode)
    }
    router := gin.New()

    // 全局中间件
    router.Use(middleware.Logger(log))
    router.Use(middleware.Recovery(log))
    router.Use(middleware.CORS())

    // 初始化处理器
    h := handlers.New(db, redisClient, log)

    // 注册路由
    setupRoutes(router, h)

    // 启动服务器
    srv := &http.Server{

```

```

    Addr:          fmt.Sprintf(":%d", cfg.Server.Port),
    Handler:       router,
    ReadTimeout:   10 * time.Second,
    WriteTimeout:  10 * time.Second,
    MaxHeaderBytes: 1 << 20,
}

// 优雅关闭
go func() {
    log.Info("Starting server",
        zap.Int("port", cfg.Server.Port),
        zap.String("env", cfg.Env),
    )
    if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        log.Fatal("Failed to start server", zap.Error(err))
    }
}()

// 等待中断信号
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

log.Info("Shutting down server...")

ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
    log.Fatal("Server forced to shutdown", zap.Error(err))
}

log.Info("Server exited")
}

func setupRoutes(router *gin.Engine, h *handlers.Handler) {
    // 健康检查
    router.GET("/health", h.Health)

    // API v1
    v1 := router.Group("/api/v1")
    {
        // 全局统计
        stats := v1.Group("/stats")
        {
            stats.GET("/global", h.GetGlobalStats)
            stats.GET("/chains/:chainId", h.GetChainStats)
            stats.GET("/tv1", h.GetTVL)
        }

        // 储备金
        reserves := v1.Group("/reserves")
    }
}

```



```

{
    reserves.GET("/snapshot", h.GetReserveSnapshot)
    reserves.GET("/history", h.GetReserveHistory)
    reserves.GET("/collateral-ratio", h.GetCollateralRatio)
}

// 用户相关
users := v1.Group("/users")
{
    users.GET("/:address/balances", h.GetUserBalances)
    users.GET("/:address/transactions", h.GetUserTransactions)
    users.GET("/:address/positions", h.GetUserPositions)
}

// 交易记录
transactions := v1.Group("/transactions")
{
    transactions.GET("/", h.GetTransactions)
    transactions.GET("/:txHash", h.GetTransactionDetail)
    transactions.GET("/recent", h.GetRecentTransactions)
}

// 清算记录
liquidations := v1.Group("/liquidations")
{
    liquidations.GET("/", h.GetLiquidations)
    liquidations.GET("/pending", h.GetPendingLiquidations)
    liquidations.GET("/stats", h.GetLiquidationStats)
}

// 收益相关
yields := v1.Group("/yields")
{
    yields.GET("/apy", h.GetCurrentAPY)
    yields.GET("/history", h.GetYieldHistory)
    yields.GET("/allocation", h.GetYieldAllocation)
}

// 价格相关
prices := v1.Group("/prices")
{
    prices.GET("/current", h.GetCurrentPrices)
    prices.GET("/history", h.GetPriceHistory)
}
}
}

```

2. 统计 API 实现:

```
// pkg/api/handlers/stats.go
```

```

package handlers

import (
    "net/http"
    "strconv"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

// GetGlobalStats 获取全局统计
func (h *Handler) GetGlobalStats(c *gin.Context) {
    // 尝试从缓存获取
    cacheKey := "stats:global"
    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }

    // 从数据库查询
    var stats []struct {
        ChainID      uint64 `json:"chain_id"`
        ChainName     string `json:"chain_name"`
        TotalSupply   string `json:"total_supply"`
        TotalMinted   string `json:"total_minted"`
        TotalRedeemed string `json:"total_redeemed"`
        TotalUsers    int    `json:"total_users"`
        TotalTransactions int64 `json:"total_transactions"`
        TVL           string `json:"tvl"`
    }

    query := `
        SELECT
            gs.chain_id,
            cs.chain_name,
            gs.total_supply,
            gs.total_minted,
            gs.total_redeemed,
            gs.total_users,
            gs.total_transactions,
            gs.tvl
        FROM global_stats gs
        JOIN chain_sync_status cs ON gs.chain_id = cs.chain_id
        ORDER BY gs.tvl DESC
    `

    if err := h.db.Raw(query).Scan(&stats).Error; err != nil {
        h.logger.Error("Failed to query global stats", zap.Error(err))
    }
}

```

```

        c.JSON(http.StatusInternalServerError, ErrorResponse{
            Code:    500,
            Message: "Internal server error",
        })
        return
    }

    // 计算总计
    result := map[string]interface{}{
        "chains": stats,
        "total": h.calculateTotals(stats),
    }

    // 缓存结果
    h.setCache(cacheKey, result, 60) // 缓存 60 秒

    c.JSON(http.StatusOK, Response{
        Code:    0,
        Message: "success",
        Data:    result,
    })
}

// GetChainStats 获取单链统计
func (h *Handler) GetChainStats(c *gin.Context) {
    chainIDStr := c.Param("chainId")
    chainID, err := strconv.ParseUint(chainIDStr, 10, 64)
    if err != nil {
        c.JSON(http.StatusBadRequest, ErrorResponse{
            Code:    400,
            Message: "Invalid chain ID",
        })
        return
    }

    // 尝试从缓存获取
    cacheKey := "stats:chain:" + chainIDStr
    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }

    // 查询链统计
    var stats struct {
        ChainID      uint64 `json:"chain_id"`
        ChainName    string `json:"chain_name"`
        TotalSupply  string `json:"total_supply"`
        TotalMinted  string `json:"total_minted"`
    }

```

```

    TotalRedeemed      string `json:"total_redeemed"`
    TotalUsers          int    `json:"total_users"`
    TotalTransactions  int64   `json:"total_transactions"`
    TVL                 string `json:"tv1"`
    UpdatedAt          string `json:"updated_at"`
}

query := `
    SELECT
        gs.chain_id,
        cs.chain_name,
        gs.total_supply,
        gs.total_minted,
        gs.total_redeemed,
        gs.total_users,
        gs.total_transactions,
        gs.tv1,
        gs.updated_at
    FROM global_stats gs
    JOIN chain_sync_status cs ON gs.chain_id = cs.chain_id
    WHERE gs.chain_id = ?
`

if err := h.db.Raw(query, chainID).Scan(&stats).Error; err != nil {
    h.logger.Error("Failed to query chain stats", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

// 查询最近 24 小时交易量
var volume24h string
query24h := `
    SELECT COALESCE(SUM(amount), 0) as volume
    FROM transactions
    WHERE chain_id = ?
        AND tx_type IN ('mint', 'redeem')
        AND block_timestamp >= NOW() - INTERVAL '24 hours'
`

h.db.Raw(query24h, chainID).Scan(&volume24h)

result := map[string]interface{}{
    "stats":    stats,
    "volume_24h": volume24h,
}

// 缓存结果
h.setCache(cacheKey, result, 60)

c.JSON(http.StatusOK, Response{

```

```

        Code:    0,
        Message: "success",
        Data:    result,
    })
}

// GetTVL 获取 TVL 统计
func (h *Handler) GetTVL(c *gin.Context) {
    // 获取查询参数
    period := c.DefaultQuery("period", "7d") // 默认 7 天

    // 尝试从缓存获取
    cacheKey := "stats:tv1:" + period
    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }

    // 解析时间范围
    interval := h.parsePeriod(period)

    // 查询 TVL 历史
    var tvlHistory []struct {
        Time    string `json:"time"`
        TVL     string `json:"tv1"`
        ChainID uint64 `json:"chain_id,omitempty"`
    }

    query := `
        SELECT
            DATE_TRUNC('hour', snapshot_time) as time,
            SUM(total_reserve_value) as tv1
        FROM reserve_snapshots
        WHERE snapshot_time >= NOW() - INTERVAL ?
        GROUP BY DATE_TRUNC('hour', snapshot_time)
        ORDER BY time ASC
    `

    if err := h.db.Raw(query, interval).Scan(&tvlHistory).Error; err != nil {
        h.logger.Error("Failed to query TVL history", zap.Error(err))
        c.JSON(http.StatusInternalServerError, ErrorResponse{
            Code:    500,
            Message: "Internal server error",
        })
        return
    }

    // 计算当前 TVL

```

```

var currentTVL string
h.db.Raw(`
    SELECT COALESCE(SUM(tvl), 0) as total_tvl
    FROM global_stats
`).Scan(&currentTVL)

result := map[string]interface{}{
    "current_tvl": currentTVL,
    "history":     tvlHistory,
}

// 缓存结果
h.setCache(cacheKey, result, 300) // 缓存 5 分钟

c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    result,
})
}

```

3. 储备金 API 实现:

```

// pkg/api/handlers/reserves.go

package handlers

import (
    "net/http"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

// GetReserveSnapshot 获取最新储备金快照
func (h *Handler) GetReserveSnapshot(c *gin.Context) {
    chainIDStr := c.Query("chain_id")

    // 尝试从缓存获取
    cacheKey := "reserves:snapshot"
    if chainIDStr != "" {
        cacheKey += ":" + chainIDStr
    }

    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }
}

```

```

}

// 查询最新快照
var snapshots []struct {
    ChainID      uint64 `json:"chain_id"`
    USDCBalance  string `json:"usdc_balance"`
    AaveValue    string `json:"aave_value"`
    CompoundValue string `json:"compound_value"`
    RWAValue     string `json:"rwa_value"`
    TotalReserveValue string `json:"total_reserve_value"`
    TotalSupply  string `json:"total_supply"`
    CollateralRatio int    `json:"collateral_ratio"`
    SurplusBuffer string `json:"surplus_buffer"`
    SurplusRatio int    `json:"surplus_ratio"`
    SnapshotTime string `json:"snapshot_time"`
}

query := `
    SELECT DISTINCT ON (chain_id)
        chain_id,
        usdc_balance,
        aave_value,
        compound_value,
        rwa_value,
        total_reserve_value,
        total_supply,
        collateral_ratio,
        surplus_buffer,
        surplus_ratio,
        snapshot_time
    FROM reserve_snapshots
`

if chainIDStr != "" {
    query += " WHERE chain_id = " + chainIDStr
}

query += " ORDER BY chain_id, snapshot_time DESC"

if err := h.db.Raw(query).Scan(&snapshots).Error; err != nil {
    h.logger.Error("Failed to query reserve snapshots", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

// 缓存结果
h.setCache(cacheKey, snapshots, 30) // 缓存 30 秒

c.JSON(http.StatusOK, Response{

```

```

        Code:    0,
        Message: "success",
        Data:    snapshots,
    })
}

```

// GetReserveHistory 获取储备金历史

```

func (h *Handler) GetReserveHistory(c *gin.Context) {
    chainIDStr := c.Query("chain_id")
    period := c.DefaultQuery("period", "7d")

    if chainIDStr == "" {
        c.JSON(http.StatusBadRequest, ErrorResponse{
            Code:    400,
            Message: "chain_id is required",
        })
        return
    }

    // 尝试从缓存获取
    cacheKey := "reserves:history:" + chainIDStr + ":" + period
    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }
}

```

```

interval := h.parsePeriod(period)

```

```

var history []struct {
    SnapshotTime      string `json:"snapshot_time"`
    TotalReserveValue string `json:"total_reserve_value"`
    TotalSupply        string `json:"total_supply"`
    CollateralRatio    int    `json:"collateral_ratio"`
    SurplusRatio       int    `json:"surplus_ratio"`
}

```

```

query := `
    SELECT
        snapshot_time,
        total_reserve_value,
        total_supply,
        collateral_ratio,
        surplus_ratio
    FROM reserve_snapshots
    WHERE chain_id = ?
        AND snapshot_time >= NOW() - INTERVAL ?
    ORDER BY snapshot_time ASC
`

```



```

if err := h.db.Raw(query, chainIDStr, interval).Scan(&history).Error; err != nil {
    h.logger.Error("Failed to query reserve history", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

// 缓存结果
h.setCache(cacheKey, history, 300) // 缓存 5 分钟

c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    history,
})
}

// GetCollateralRatio 获取实时抵押率
func (h *Handler) GetCollateralRatio(c *gin.Context) {
    // 尝试从缓存获取
    cacheKey := "reserves:collateral_ratio"
    if cached, err := h.getFromCache(cacheKey); err == nil {
        c.JSON(http.StatusOK, Response{
            Code:    0,
            Message: "success",
            Data:    cached,
        })
        return
    }

    // 查询所有链的抵押率
    var ratios []struct {
        ChainID        uint64 `json:"chain_id"`
        ChainName      string `json:"chain_name"`
        CollateralRatio int     `json:"collateral_ratio"`
        Status         string `json:"status"`
        UpdatedAt      string `json:"updated_at"`
    }

    query := `
        SELECT
            rs.chain_id,
            cs.chain_name,
            rs.collateral_ratio,
            CASE
                WHEN rs.collateral_ratio >= 11000 THEN 'healthy'
                WHEN rs.collateral_ratio >= 10500 THEN 'warning'
                WHEN rs.collateral_ratio >= 10200 THEN 'danger'
                ELSE 'critical'
            END
    `

```

```

        END as status,
        rs.snapshot_time as updated_at
    FROM (
        SELECT DISTINCT ON (chain_id)
            chain_id,
            collateral_ratio,
            snapshot_time
        FROM reserve_snapshots
        ORDER BY chain_id, snapshot_time DESC
    ) rs
    JOIN chain_sync_status cs ON rs.chain_id = cs.chain_id
    ORDER BY rs.chain_id
`

if err := h.db.Raw(query).Scan(&ratios).Error; err != nil {
    h.logger.Error("Failed to query collateral ratios", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

// 缓存结果
h.setCache(cacheKey, ratios, 30) // 缓存 30 秒

c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    ratios,
})
}

```

4. 用户 API 实现:

```

// pkg/api/handlers/users.go

package handlers

import (
    "net/http"
    "strconv"
    "strings"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

// GetUserBalances 获取用户余额
func (h *Handler) GetUserBalances(c *gin.Context) {
    address := c.Param("address")
}

```

```

// 验证地址格式
if !h.isValidAddress(address) {
    c.JSON(http.StatusBadRequest, ErrorResponse{
        Code:    400,
        Message: "Invalid address format",
    })
    return
}

// 规范化地址
address = strings.ToLower(address)

// 尝试从缓存获取
cacheKey := "user:balances:" + address
if cached, err := h.getFromCache(cacheKey); err == nil {
    c.JSON(http.StatusOK, Response{
        Code:    0,
        Message: "success",
        Data:    cached,
    })
    return
}

// 查询用户余额
var balances []struct {
    ChainID    uint64 `json:"chain_id"`
    ChainName  string `json:"chain_name"`
    TokenType  string `json:"token_type"`
    Balance    string `json:"balance"`
    UpdatedAt  string `json:"updated_at"`
}

query := `
    SELECT
        ub.chain_id,
        cs.chain_name,
        ub.token_type,
        ub.balance,
        ub.updated_at
    FROM user_balances ub
    JOIN chain_sync_status cs ON ub.chain_id = cs.chain_id
    WHERE ub.user_address = ?
        AND ub.balance != '0'
    ORDER BY ub.chain_id, ub.token_type
`

if err := h.db.Raw(query, address).Scan(&balances).Error; err != nil {
    h.logger.Error("Failed to query user balances", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
}

```

```

    })
    return
}

// 缓存结果
h.setCache(cacheKey, balances, 60) // 缓存 60 秒

c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    balances,
})
}

// GetUserTransactions 获取用户交易历史
func (h *Handler) GetUserTransactions(c *gin.Context) {
    address := c.Param("address")

    // 验证地址
    if !h.isValidAddress(address) {
        c.JSON(http.StatusBadRequest, ErrorResponse{
            Code:    400,
            Message: "Invalid address format",
        })
        return
    }

    address = strings.ToLower(address)

    // 分页参数
    page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
    pageSize, _ := strconv.Atoi(c.DefaultQuery("page_size", "20"))
    txType := c.Query("type")

    if page < 1 {
        page = 1
    }
    if pageSize < 1 || pageSize > 100 {
        pageSize = 20
    }

    offset := (page - 1) * pageSize

    // 查询交易记录
    var transactions []struct {
        TxHash      string `json:"tx_hash"`
        ChainID     uint64 `json:"chain_id"`
        ChainName   string `json:"chain_name"`
        BlockNumber uint64 `json:"block_number"`
        TxType      string `json:"tx_type"`
        Amount      string `json:"amount"`
        BlockTimestamp string `json:"block_timestamp"`
    }

```

```

}

query := `
    SELECT
        t.tx_hash,
        t.chain_id,
        cs.chain_name,
        t.block_number,
        t.tx_type,
        t.amount,
        t.block_timestamp
    FROM transactions t
    JOIN chain_sync_status cs ON t.chain_id = cs.chain_id
    WHERE t.user_address = ?
`

args := []interface{}{address}

if txType != "" {
    query += " AND t.tx_type = ?"
    args = append(args, txType)
}

query += " ORDER BY t.block_timestamp DESC LIMIT ? OFFSET ?"
args = append(args, pageSize, offset)

if err := h.db.Raw(query, args...).Scan(&transactions).Error; err != nil {
    h.logger.Error("Failed to query user transactions", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

// 查询总数
var total int64
countQuery := `
    SELECT COUNT(*)
    FROM transactions
    WHERE user_address = ?
`

countArgs := []interface{}{address}

if txType != "" {
    countQuery += " AND tx_type = ?"
    countArgs = append(countArgs, txType)
}

h.db.Raw(countQuery, countArgs...).Scan(&total)

result := map[string]interface{}{

```

```

        "list": transactions,
        "pagination": map[string]interface{}{
            "page":      page,
            "page_size": pageSize,
            "total":      total,
            "pages":      (total + int64(pageSize) - 1) / int64(pageSize),
        },
    },
}

c.JSON(http.StatusOK, Response{
    Code:      0,
    Message:   "success",
    Data:      result,
})
}

```

// GetUserPositions 获取用户借款头寸

```

func (h *Handler) GetUserPositions(c *gin.Context) {
    address := c.Param("address")

    if !h.isValidAddress(address) {
        c.JSON(http.StatusBadRequest, ErrorResponse{
            Code:      400,
            Message:   "Invalid address format",
        })
        return
    }

    address = strings.ToLower(address)

```

// 查询借款头寸

```

var positions []struct {
    ChainID          uint64 `json:"chain_id"`
    ChainName        string `json:"chain_name"`
    CollateralType    string `json:"collateral_type"`
    CollateralAmount string `json:"collateral_amount"`
    BorrowedAmount    string `json:"borrowed_amount"`
    HealthFactor      string `json:"health_factor"`
    LiquidationThreshold int    `json:"liquidation_threshold"`
    Status            string `json:"status"`
    UpdatedAt         string `json:"updated_at"`
}

```

```

query := `
    SELECT
        bp.chain_id,
        cs.chain_name,
        bp.collateral_type,
        bp.collateral_amount,
        bp.borrowed_amount,
        bp.health_factor,
        bp.liquidation_threshold,

```

```

        bp.status,
        bp.updated_at
    FROM borrow_positions bp
    JOIN chain_sync_status cs ON bp.chain_id = cs.chain_id
    WHERE bp.user_address = ?
    ORDER BY bp.updated_at DESC
`

if err := h.db.Raw(query, address).Scan(&positions).Error; err != nil {
    h.logger.Error("Failed to query user positions", zap.Error(err))
    c.JSON(http.StatusInternalServerError, ErrorResponse{
        Code:    500,
        Message: "Internal server error",
    })
    return
}

c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    positions,
})
}

```

8.4 中间件实现

1. 日志中间件:

```

// pkg/api/middleware/logger.go

package middleware

import (
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

func Logger(logger *zap.Logger) gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        path := c.Request.URL.Path
        query := c.Request.URL.RawQuery

        c.Next()

        latency := time.Since(start)

        logger.Info("HTTP request",
            zap.String("method", c.Request.Method),

```

```

        zap.String("path", path),
        zap.String("query", query),
        zap.Int("status", c.Writer.Status()),
        zap.Duration("latency", latency),
        zap.String("ip", c.ClientIP()),
        zap.String("user_agent", c.Request.UserAgent()),
    )
}
}

```

2. 限流中间件:

```

// pkg/api/middleware/ratelimit.go

package middleware

import (
    "net/http"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

type IPRateLimiter struct {
    ips map[string]*rate.Limiter
    mu   *sync.RWMutex
    r    rate.Limit
    b    int
}

func NewIPRateLimiter(r rate.Limit, b int) *IPRateLimiter {
    return &IPRateLimiter{
        ips: make(map[string]*rate.Limiter),
        mu:   &sync.RWMutex{},
        r:    r,
        b:    b,
    }
}

func (i *IPRateLimiter) AddIP(ip string) *rate.Limiter {
    i.mu.Lock()
    defer i.mu.Unlock()

    limiter := rate.NewLimiter(i.r, i.b)
    i.ips[ip] = limiter

    return limiter
}

```



```

func (i *IPRateLimiter) GetLimiter(ip string) *rate.Limiter {
    i.mu.Lock()
    limiter, exists := i.ips[ip]

    if !exists {
        i.mu.Unlock()
        return i.AddIP(ip)
    }

    i.mu.Unlock()
    return limiter
}

func RateLimit(limiter *IPRateLimiter) gin.HandlerFunc {
    return func(c *gin.Context) {
        ip := c.ClientIP()
        l := limiter.GetLimiter(ip)

        if !l.Allow() {
            c.JSON(http.StatusTooManyRequests, gin.H{
                "code": 429,
                "message": "Too many requests",
            })
            c.Abort()
            return
        }

        c.Next()
    }
}

```

3. CORS 中间件:

```

// pkg/api/middleware/cors.go

package middleware

import (
    "github.com/gin-gonic/gin"
)

func CORS() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Writer.Header().Set("Access-Control-Allow-Origin", "*")
        c.Writer.Header().Set("Access-Control-Allow-Credentials", "true")
        c.Writer.Header().Set("Access-Control-Allow-Headers", "Content-Type, Content-
Length, Accept-Encoding, X-CSRF-Token, Authorization, accept, origin, Cache-Control, X-
Requested-With")
        c.Writer.Header().Set("Access-Control-Allow-Methods", "POST, OPTIONS, GET, PUT,
DELETE")
    }
}

```

```

    if c.Request.Method == "OPTIONS" {
        c.AbortWithStatus(204)
        return
    }

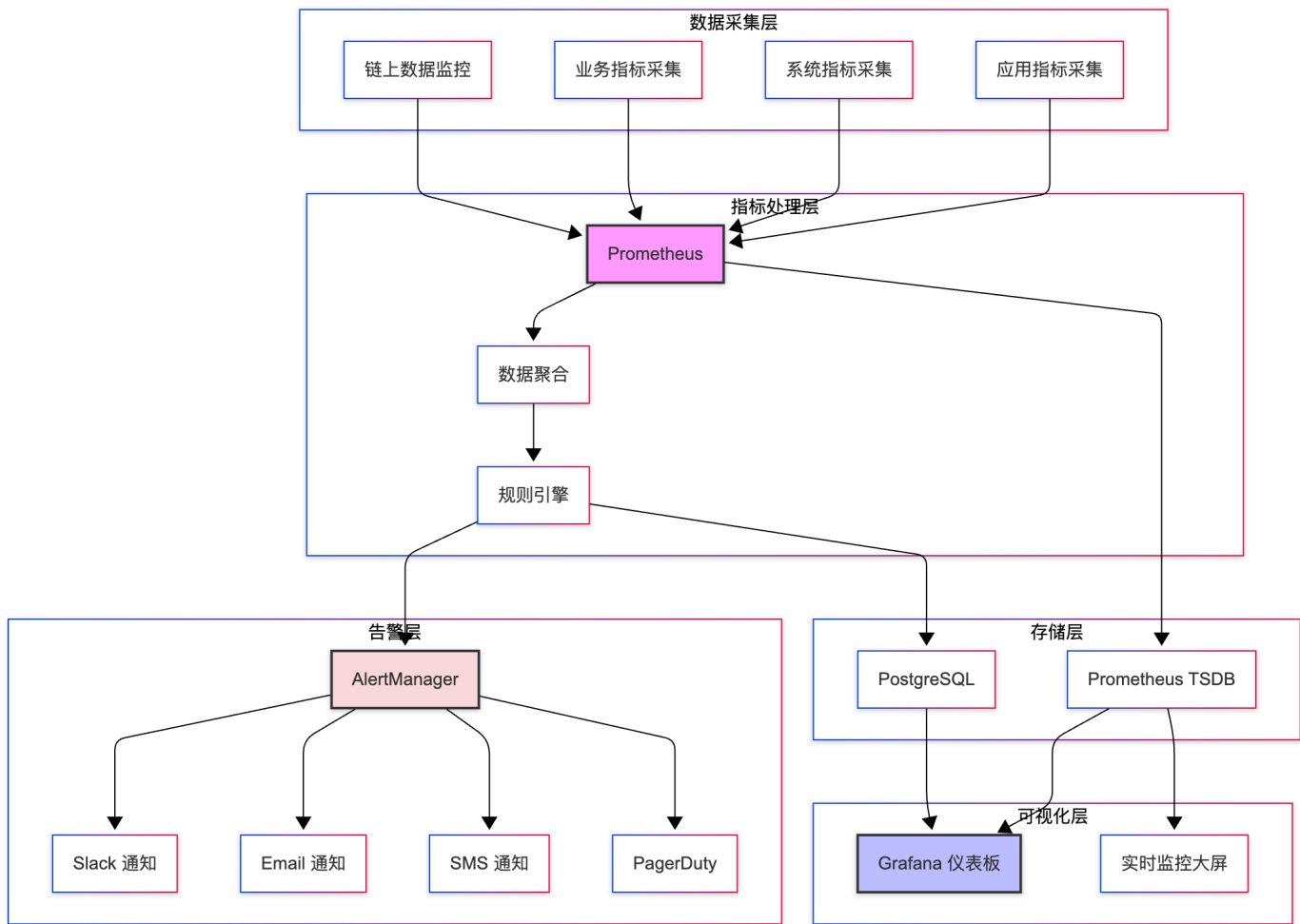
    c.Next()
}
}

```

九、风控监控系统

9.1 监控系统架构

监控系统总体架构：



9.2 指标采集实现

1. Prometheus 指标定义：

```

// pkg/metrics/metrics.go

package metrics

```

```

import (
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
)

var (
    // HTTP 请求指标
    HttpRequestsTotal = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "angle_http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "endpoint", "status"},
    )

    HttpRequestDuration = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name: "angle_http_request_duration_seconds",
            Help: "Duration of HTTP requests in seconds",
            Buckets: prometheus.DefBuckets,
        },
        []string{"method", "endpoint"},
    )

    // 区块链同步指标
    ChainSyncHeight = promauto.NewGaugeVec(
        prometheus.GaugeOpts{
            Name: "angle_chain_sync_height",
            Help: "Current synced block height",
        },
        []string{"chain_id", "chain_name"},
    )

    ChainSyncLag = promauto.NewGaugeVec(
        prometheus.GaugeOpts{
            Name: "angle_chain_sync_lag",
            Help: "Block lag between current and latest",
        },
        []string{"chain_id", "chain_name"},
    )

    // 交易处理指标
    TransactionsProcessed = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "angle_transactions_processed_total",
            Help: "Total number of transactions processed",
        },
        []string{"chain_id", "tx_type"},
    )

    TransactionProcessingErrors = promauto.NewCounterVec(
        prometheus.CounterOpts{

```

```

        Name: "angle_transaction_processing_errors_total",
        Help: "Total number of transaction processing errors",
    },
    []string{"chain_id", "error_type"},
)

// 抵押率指标
CollateralRatio = promauto.NewGaugeVec(
    prometheus.GaugeOpts{
        Name: "angle_collateral_ratio",
        Help: "Current collateral ratio (basis points)",
    },
    []string{"chain_id"},
)

// 清算指标
LiquidationsExecuted = promauto.NewCounterVec(
    prometheus.CounterOpts{
        Name: "angle_liquidations_executed_total",
        Help: "Total number of liquidations executed",
    },
    []string{"chain_id", "status"},
)

LiquidationProfit = promauto.NewCounterVec(
    prometheus.CounterOpts{
        Name: "angle_liquidation_profit_total",
        Help: "Total profit from liquidations",
    },
    []string{"chain_id"},
)

// 数据库指标
DatabaseQueryDuration = promauto.NewHistogramVec(
    prometheus.HistogramOpts{
        Name: "angle_database_query_duration_seconds",
        Help: "Duration of database queries in seconds",
        Buckets: prometheus.DefBuckets,
    },
    []string{"query_type"},
)

DatabaseConnectionsActive = promauto.NewGauge(
    prometheus.GaugeOpts{
        Name: "angle_database_connections_active",
        Help: "Number of active database connections",
    },
)

// Redis 指标
RedisOperationDuration = promauto.NewHistogramVec(
    prometheus.HistogramOpts{

```

```

        Name:      "angle_redis_operation_duration_seconds",
        Help:      "Duration of Redis operations in seconds",
        Buckets:   prometheus.DefBuckets,
    },
    []string{"operation"},
)

// 收益率指标
CurrentAPY = promauto.NewGaugeVec(
    prometheus.GaugeOpts{
        Name: "angle_current_apy",
        Help: "Current APY for yield strategies",
    },
    []string{"protocol"},
)

// 业务指标
TotalSupply = promauto.NewGaugeVec(
    prometheus.GaugeOpts{
        Name: "angle_total_supply",
        Help: "Total supply of stablecoins",
    },
    []string{"chain_id", "token"},
)

TVL = promauto.NewGaugeVec(
    prometheus.GaugeOpts{
        Name: "angle_tvl",
        Help: "Total value locked",
    },
    []string{"chain_id"},
)
)

// RecordHTTPRequest 记录 HTTP 请求指标
func RecordHTTPRequest(method, endpoint, status string, duration float64) {
    HttpRequestsTotal.WithLabelValues(method, endpoint, status).Inc()
    HttpRequestDuration.WithLabelValues(method, endpoint).Observe(duration)
}

// UpdateChainSync 更新链同步指标
func UpdateChainSync(chainID uint64, chainName string, currentHeight, latestHeight uint64) {
    ChainSyncHeight.WithLabelValues(
        fmt.Sprintf("%d", chainID),
        chainName,
    ).Set(float64(currentHeight))

    lag := int64(latestHeight) - int64(currentHeight)
    if lag < 0 {
        lag = 0
    }
}

```

```

ChainSyncLag.WithLabelValues(
    fmt.Sprintf("%d", chainID),
    chainName,
).Set(float64(lag))
}

// RecordTransaction 记录交易处理
func RecordTransaction(chainID uint64, txType string) {
    TransactionsProcessed.WithLabelValues(
        fmt.Sprintf("%d", chainID),
        txType,
    ).Inc()
}

// RecordTransactionError 记录交易错误
func RecordTransactionError(chainID uint64, errorType string) {
    TransactionProcessingErrors.WithLabelValues(
        fmt.Sprintf("%d", chainID),
        errorType,
    ).Inc()
}

// UpdateCollateralRatio 更新抵押率
func UpdateCollateralRatio(chainID uint64, ratio int) {
    CollateralRatio.WithLabelValues(
        fmt.Sprintf("%d", chainID),
    ).Set(float64(ratio))
}

// RecordLiquidation 记录清算
func RecordLiquidation(chainID uint64, status string, profit *big.Int) {
    LiquidationsExecuted.WithLabelValues(
        fmt.Sprintf("%d", chainID),
        status,
    ).Inc()

    if profit != nil && status == "success" {
        profitFloat, _ := new(big.Float).SetInt(profit).Float64()
        LiquidationProfit.WithLabelValues(
            fmt.Sprintf("%d", chainID),
        ).Add(profitFloat)
    }
}

```

2. 业务指标采集服务：

```

// pkg/service/metrics_collector.go

package service

import (

```

```

"context"
"time"

"go.uber.org/zap"
"gorm.io/gorm"

"angle-protocol/pkg/metrics"
)

type MetricsCollector struct {
    db      *gorm.DB
    logger  *zap.Logger

    interval time.Duration
    stopCh   chan struct{}
}

func NewMetricsCollector(db *gorm.DB, logger *zap.Logger) *MetricsCollector {
    return &MetricsCollector{
        db:      db,
        logger:  logger,
        interval: 30 * time.Second,
        stopCh:   make(chan struct{}),
    }
}

func (c *MetricsCollector) Start() error {
    c.logger.Info("Starting metrics collector")

    ticker := time.NewTicker(c.interval)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            if err := c.collectMetrics(); err != nil {
                c.logger.Error("Failed to collect metrics", zap.Error(err))
            }
        case <-c.stopCh:
            c.logger.Info("Stopping metrics collector")
            return nil
        }
    }
}

func (c *MetricsCollector) Stop() {
    close(c.stopCh)
}

func (c *MetricsCollector) collectMetrics() error {
    ctx := context.Background()

```

```

// 1. 收集总供应量指标
if err := c.collectSupplyMetrics(ctx); err != nil {
    return err
}

// 2. 收集 TVL 指标
if err := c.collectTVLMetrics(ctx); err != nil {
    return err
}

// 3. 收集抵押率指标
if err := c.collectCollateralRatioMetrics(ctx); err != nil {
    return err
}

// 4. 收集收益率指标
if err := c.collectAPYMetrics(ctx); err != nil {
    return err
}

// 5. 收集数据库连接指标
if err := c.collectDatabaseMetrics(ctx); err != nil {
    return err
}

return nil
}

func (c *MetricsCollector) collectSupplyMetrics(ctx context.Context) error {
    var results []struct {
        ChainID      uint64
        TotalSupply string
    }

    err := c.db.WithContext(ctx).
        Raw("SELECT chain_id, total_supply FROM global_stats").
        Scan(&results).Error

    if err != nil {
        return err
    }

    for _, r := range results {
        supply := new(big.Float)
        supply.SetString(r.TotalSupply)
        supplyFloat, _ := supply.Float64()

        metrics.TotalSupply.WithLabelValues(
            fmt.Sprintf("%d", r.ChainID),
            "USDA",
        ).Set(supplyFloat)
    }
}

```



```

    return nil
}

func (c *MetricsCollector) collectTVLMetrics(ctx context.Context) error {
    var results []struct {
        ChainID uint64
        TVL      string
    }

    err := c.db.WithContext(ctx).
        Raw("SELECT chain_id, tvl FROM global_stats").
        Scan(&results).Error

    if err != nil {
        return err
    }

    for _, r := range results {
        tvl := new(big.Float)
        tvl.SetString(r.TVL)
        tvlFloat, _ := tvl.Float64()

        metrics.TVL.WithLabelValues(
            fmt.Sprintf("%d", r.ChainID),
        ).Set(tvlFloat)
    }

    return nil
}

func (c *MetricsCollector) collectCollateralRatioMetrics(ctx context.Context) error {
    var results []struct {
        ChainID          uint64
        CollateralRatio int
    }

    err := c.db.WithContext(ctx).
        Raw(`
            SELECT DISTINCT ON (chain_id)
                chain_id,
                collateral_ratio
            FROM reserve_snapshots
            ORDER BY chain_id, snapshot_time DESC
        `).
        Scan(&results).Error

    if err != nil {
        return err
    }

    for _, r := range results {

```

```

        metrics.UpdateCollateralRatio(r.ChainID, r.CollateralRatio)
    }

    return nil
}

func (c *MetricsCollector) collectAPYMetrics(ctx context.Context) error {
    var results []struct {
        Protocol string
        APY      float64
    }

    err := c.db.WithContext(ctx).
        Raw(`
            SELECT DISTINCT ON (protocol)
                protocol,
                apy
            FROM yield_history
            ORDER BY protocol, time DESC
        `).
        Scan(&results).Error

    if err != nil {
        return err
    }

    for _, r := range results {
        metrics.CurrentAPY.WithLabelValues(r.Protocol).Set(r.APY)
    }

    return nil
}

func (c *MetricsCollector) collectDatabaseMetrics(ctx context.Context) error {
    sqlDB, err := c.db.DB()
    if err != nil {
        return err
    }

    stats := sqlDB.Stats()
    metrics.DatabaseConnectionsActive.Set(float64(stats.InUse))

    return nil
}

```

9.3 告警系统

1. AlertManager 配置:

```
# alertmanager.yml
```

```

global:
  resolve_timeout: 5m
  slack_api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'

route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 12h
  receiver: 'default'
  routes:
    - match:
        severity: critical
      receiver: 'critical-alerts'
      continue: true
    - match:
        severity: warning
      receiver: 'warning-alerts'

receivers:
- name: 'default'
  slack_configs:
    - channel: '#angle-alerts'
      title: 'Angle Protocol Alert'
      text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'

- name: 'critical-alerts'
  slack_configs:
    - channel: '#angle-critical'
      title: '🚨 CRITICAL: {{ .GroupLabels.alertname }}'
      text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'
  pagerduty_configs:
    - service_key: 'YOUR_PAGERDUTY_KEY'

- name: 'warning-alerts'
  slack_configs:
    - channel: '#angle-warnings'
      title: '⚠️ WARNING: {{ .GroupLabels.alertname }}'
      text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'

```

2. Prometheus 告警规则:

```

# alerts.yml

groups:
- name: collateral_ratio_alerts
  interval: 30s
  rules:
    - alert: CollateralRatioCritical
      expr: angle_collateral_ratio < 10200
      for: 1m

```

```
labels:
  severity: critical
annotations:
  description: "抵押率低于 102%! 链 {{ $labels.chain_id }} 当前抵押率: {{ $value }} 基点"
  summary: "紧急: 抵押率过低"

- alert: CollateralRatioDanger
  expr: angle_collateral_ratio < 10500
  for: 5m
  labels:
    severity: warning
  annotations:
    description: "抵押率低于 105%! 链 {{ $labels.chain_id }} 当前抵押率: {{ $value }} 基点"
    summary: "警告: 抵押率接近阈值"

- name: chain_sync_alerts
  interval: 1m
  rules:
    - alert: ChainSyncLagHigh
      expr: angle_chain_sync_lag > 100
      for: 5m
      labels:
        severity: warning
      annotations:
        description: "链同步延迟过高! {{ $labels.chain_name }} 延迟 {{ $value }} 个区块"
        summary: "链同步延迟"

    - alert: ChainSyncStalled
      expr: rate(angle_chain_sync_height[5m]) == 0
      for: 5m
      labels:
        severity: critical
      annotations:
        description: "链同步停滞! {{ $labels.chain_name }} 5分钟内没有新区块"
        summary: "链同步停滞"

- name: liquidation_alerts
  interval: 1m
  rules:
    - alert: LiquidationFailed
      expr: rate(angle_liquidations_executed_total{status="failed"}[5m]) > 0.1
      for: 2m
      labels:
        severity: warning
      annotations:
        description: "清算失败率高! 链 {{ $labels.chain_id }}"
        summary: "清算失败"

    - alert: HighRiskPositions
      expr: count(angle_borrow_position_health_factor < 1.05) > 10
      for: 5m
      labels:
```

```

    severity: warning
  annotations:
    description: "高风险借款头寸数量: {{ $value }}"
    summary: "高风险头寸告警"

- name: application_alerts
  interval: 30s
  rules:
    - alert: HighErrorRate
      expr: rate(angle_transaction_processing_errors_total[5m]) > 0.05
      for: 2m
      labels:
        severity: warning
      annotations:
        description: "交易处理错误率高! 链 {{ $labels.chain_id }}, 错误类型: {{ $labels.error_type }}"
        summary: "交易处理错误"

    - alert: APIHighLatency
      expr: histogram_quantile(0.95, rate(angle_http_request_duration_seconds_bucket[5m])) >
2
      for: 5m
      labels:
        severity: warning
      annotations:
        description: "API 响应延迟高! P95 延迟: {{ $value }}s, 端点: {{ $labels.endpoint }}"
        summary: "API 延迟告警"

    - alert: DatabaseConnectionPoolExhausted
      expr: angle_database_connections_active > 90
      for: 5m
      labels:
        severity: critical
      annotations:
        description: "数据库连接池即将耗尽! 当前活跃连接: {{ $value }}"
        summary: "数据库连接池告警"

- name: business_alerts
  interval: 1m
  rules:
    - alert: TVLDropSignificant
      expr: (angle_tvl - angle_tvl offset 1h) / angle_tvl offset 1h < -0.1
      for: 5m
      labels:
        severity: warning
      annotations:
        description: "TVL 在过去 1 小时下降超过 10%! 链 {{ $labels.chain_id }}"
        summary: "TVL 显著下降"

    - alert: APYAnomalous
      expr: abs(angle_current_apy - angle_current_apy offset 1h) > 5
      for: 10m

```

```
labels:
  severity: warning
annotations:
  description: "APY 异常波动! 协议: {{ $labels.protocol }}, 当前 APY: {{ $value }}%"
  summary: "APY 异常"
```

3. 自定义告警通知实现:

```
// pkg/alert/notifier.go

package alert

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "go.uber.org/zap"
)

type Notifier struct {
    slackWebhook string
    emailAPI      string
    smsAPI        string
    logger        *zap.Logger
}

type Alert struct {
    Level          string    `json:"level"`           // critical, warning, info
    Title          string    `json:"title"`
    Message        string    `json:"message"`
    Timestamp      time.Time `json:"timestamp"`
    ChainID        uint64    `json:"chain_id,omitempty"`
    MetricValue    string    `json:"metric_value,omitempty"`
}

func NewNotifier(slackWebhook, emailAPI, smsAPI string, logger *zap.Logger) *Notifier {
    return &Notifier{
        slackWebhook: slackWebhook,
        emailAPI:     emailAPI,
        smsAPI:       smsAPI,
        logger:       logger,
    }
}

// SendAlert 发送告警
func (n *Notifier) SendAlert(alert *Alert) error {
    n.logger.Info("Sending alert",
        zap.String("level", alert.Level),
```

```

        zap.String("title", alert.Title),
    )

    // 根据告警级别选择通知渠道
    switch alert.Level {
    case "critical":
        // 紧急告警: Slack + Email + SMS
        if err := n.sendSlackAlert(alert); err != nil {
            n.logger.Error("Failed to send Slack alert", zap.Error(err))
        }
        if err := n.sendEmailAlert(alert); err != nil {
            n.logger.Error("Failed to send email alert", zap.Error(err))
        }
        if err := n.sendSMSAlert(alert); err != nil {
            n.logger.Error("Failed to send SMS alert", zap.Error(err))
        }

    case "warning":
        // 警告: Slack + Email
        if err := n.sendSlackAlert(alert); err != nil {
            n.logger.Error("Failed to send Slack alert", zap.Error(err))
        }
        if err := n.sendEmailAlert(alert); err != nil {
            n.logger.Error("Failed to send email alert", zap.Error(err))
        }

    case "info":
        // 信息: 仅 Slack
        if err := n.sendSlackAlert(alert); err != nil {
            n.logger.Error("Failed to send Slack alert", zap.Error(err))
        }
    }

    return nil
}

func (n *Notifier) sendSlackAlert(alert *Alert) error {
    emoji := n.getEmojiForLevel(alert.Level)
    color := n.getColorForLevel(alert.Level)

    payload := map[string]interface{}{
        "attachments": []map[string]interface{}{
            {
                "color":      color,
                "title":      fmt.Sprintf("%s %s", emoji, alert.Title),
                "text":       alert.Message,
                "footer":     "Angle Protocol Monitoring",
                "ts":         alert.Timestamp.Unix(),
                "fields": []map[string]interface{}{
                    {
                        "title": "Level",
                        "value": alert.Level,
                    },
                },
            },
        },
    }

```

```

        "short": true,
    },
    {
        "title": "Chain ID",
        "value": fmt.Sprintf("%d", alert.ChainID),
        "short": true,
    },
},
},
},
}

jsonData, err := json.Marshal(payload)
if err != nil {
    return err
}

resp, err := http.Post(n.slackWebhook, "application/json", bytes.NewBuffer(jsonData))
if err != nil {
    return err
}
defer resp.Body.Close()

if resp.StatusCode != http.StatusOK {
    return fmt.Errorf("slack API returned status %d", resp.StatusCode)
}

return nil
}

func (n *Notifier) sendEmailAlert(alert *Alert) error {
    // 实现邮件发送逻辑
    // 这里使用第三方邮件服务 API (如 SendGrid, Mailgun)
    return nil
}

func (n *Notifier) sendSMSAlert(alert *Alert) error {
    // 实现短信发送逻辑
    // 这里使用第三方短信服务 API (如 Twilio)
    return nil
}

func (n *Notifier) getEmojiForLevel(level string) string {
    switch level {
    case "critical":
        return "🔴"
    case "warning":
        return "⚠️"
    case "info":
        return "ℹ️"
    default:
        return "🇩🇪"
    }
}

```



```

    }
}

func (n *Notifier) getColorForLevel(level string) string {
    switch level {
    case "critical":
        return "danger"
    case "warning":
        return "warning"
    case "info":
        return "good"
    default:
        return "#808080"
    }
}

```

9.4 Grafana 仪表板

Grafana 仪表板 JSON 配置（简化示例）：

```

{
  "dashboard": {
    "title": "Angle Protocol Monitoring",
    "panels": [
      {
        "id": 1,
        "title": "Collateral Ratio",
        "type": "graph",
        "targets": [
          {
            "expr": "angle_collateral_ratio",
            "legendFormat": "Chain {{ chain_id }}"
          }
        ],
        "alert": {
          "conditions": [
            {
              "evaluator": {
                "params": [10500],
                "type": "lt"
              },
              "operator": {
                "type": "and"
              },
              "query": {
                "params": ["A", "5m", "now"]
              },
              "reducer": {
                "type": "avg"
              },
              "type": "query"
            }
          ]
        }
      }
    ]
  }
}

```

```

    }
  ]
}
},
{
  "id": 2,
  "title": "TVL",
  "type": "stat",
  "targets": [
    {
      "expr": "sum(angle_tvl)",
      "legendFormat": "Total TVL"
    }
  ]
},
{
  "id": 3,
  "title": "Transaction Rate",
  "type": "graph",
  "targets": [
    {
      "expr": "rate(angle_transactions_processed_total[5m])",
      "legendFormat": "{{ tx_type }}"
    }
  ]
},
{
  "id": 4,
  "title": "API Latency (P95)",
  "type": "graph",
  "targets": [
    {
      "expr": "histogram_quantile(0.95,
rate(angle_http_request_duration_seconds_bucket[5m]))",
      "legendFormat": "{{ endpoint }}"
    }
  ]
},
{
  "id": 5,
  "title": "Active Borrow Positions",
  "type": "table",
  "targets": [
    {
      "expr": "topk(10, angle_borrow_position_health_factor < 1.2)",
      "format": "table"
    }
  ]
}
]
}
}

```

十、系统性能优化

10.1 数据库优化

1. 连接池优化：

```
// pkg/database/postgres.go

func Connect(config DatabaseConfig) (*gorm.DB, error) {
    dsn := fmt.Sprintf(
        "host=%s user=%s password=%s dbname=%s port=%d sslmode=%s TimeZone=UTC",
        config.Host,
        config.User,
        config.Password,
        config.Database,
        config.Port,
        config.SSLMode,
    )

    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
        Logger: logger.Default.LogMode(logger.Info),
        PrepareStmt: true, // 预编译 SQL
    })

    if err != nil {
        return nil, err
    }

    sqlDB, err := db.DB()
    if err != nil {
        return nil, err
    }

    // 优化连接池配置
    sqlDB.SetMaxIdleConns(10)           // 最大空闲连接数
    sqlDB.SetMaxOpenConns(100)         // 最大打开连接数
    sqlDB.SetConnMaxLifetime(time.Hour) // 连接最大生命周期

    return db, nil
}
```

2. 查询优化示例：

```
// 使用索引优化查询
func (h *Handler) GetUserTransactionsOptimized(userAddress string, page, pageSize int)
([]Transaction, error) {
    var transactions []Transaction

    // 使用覆盖索引
    err := h.db.
```

```

        Select("tx_hash, chain_id, tx_type, amount, block_timestamp").
        Where("user_address = ?", userAddress).
        Order("block_timestamp DESC").
        Limit(pageSize).
        Offset((page - 1) * pageSize).
        Find(&transactions).Error

    return transactions, err
}

// 使用批量插入
func (h *Handler) BulkInsertTransactions(transactions []Transaction) error {
    // 分批插入, 每批 1000 条
    batchSize := 1000

    for i := 0; i < len(transactions); i += batchSize {
        end := i + batchSize
        if end > len(transactions) {
            end = len(transactions)
        }

        batch := transactions[i:end]
        if err := h.db.CreateInBatches(batch, batchSize).Error; err != nil {
            return err
        }
    }

    return nil
}

```

10.2 Redis 缓存优化

1. 缓存策略:

```

// pkg/cache/strategy.go

type CacheStrategy struct {
    redis *redis.Client
    logger *zap.Logger
}

// 缓存穿透保护: 使用布隆过滤器
func (c *CacheStrategy) GetWithBloomFilter(key string, fetch func() (interface{}, error)) (interface{}, error) {
    // 1. 检查布隆过滤器
    exists, err := c.bloomFilterCheck(key)
    if err != nil {
        return nil, err
    }

    if !exists {

```

```

        return nil, ErrNotFound
    }

    // 2. 查询缓存
    cached, err := c.redis.Get(context.Background(), key).Result()
    if err == nil {
        return cached, nil
    }

    // 3. 查询数据库
    data, err := fetch()
    if err != nil {
        return nil, err
    }

    // 4. 写入缓存
    c.redis.Set(context.Background(), key, data, 5*time.Minute)

    return data, nil
}

// 缓存雪崩保护: 使用随机过期时间
func (c *CacheStrategy) SetWithRandomTTL(key string, value interface{}, baseTTL
time.Duration) error {
    // 在基础 TTL 上增加随机时间 (±20%)
    randomOffset := time.Duration(rand.Intn(int(baseTTL.Seconds()*0.4)) -
int(baseTTL.Seconds()*0.2))
    ttl := baseTTL + randomOffset*time.Second

    return c.redis.Set(context.Background(), key, value, ttl).Err()
}

// 缓存击穿保护: 使用互斥锁
func (c *CacheStrategy) GetWithMutex(key string, fetch func() (interface{}, error))
(interface{}, error) {
    // 1. 尝试获取缓存
    cached, err := c.redis.Get(context.Background(), key).Result()
    if err == nil {
        return cached, nil
    }

    // 2. 获取分布式锁
    lockKey := "lock:" + key
    acquired, err := c.redis.SetNX(context.Background(), lockKey, "1",
10*time.Second).Result()
    if err != nil {
        return nil, err
    }

    if !acquired {
        // 等待锁释放, 然后重试
        time.Sleep(100 * time.Millisecond)
    }
}

```

```

        return c.GetWithMutex(key, fetch)
    }

    defer c.redis.Del(context.Background(), lockKey)

    // 3. 再次检查缓存 (double check)
    cached, err = c.redis.Get(context.Background(), key).Result()
    if err == nil {
        return cached, nil
    }

    // 4. 查询数据库
    data, err := fetch()
    if err != nil {
        return nil, err
    }

    // 5. 写入缓存
    c.redis.Set(context.Background(), key, data, 5*time.Minute)

    return data, nil
}

```

10.3 并发优化

1. Goroutine 池:

```

// pkg/worker/pool.go

type WorkerPool struct {
    workers    int
    taskQueue  chan Task
    wg         sync.WaitGroup
}

type Task func() error

func NewWorkerPool(workers int, queueSize int) *WorkerPool {
    return &WorkerPool{
        workers:    workers,
        taskQueue:  make(chan Task, queueSize),
    }
}

func (p *WorkerPool) Start() {
    for i := 0; i < p.workers; i++ {
        p.wg.Add(1)
        go p.worker()
    }
}

```

```

func (p *WorkerPool) worker() {
    defer p.wg.Done()

    for task := range p.taskQueue {
        if err := task(); err != nil {
            // 处理错误
            log.Error("Task failed", zap.Error(err))
        }
    }
}

func (p *WorkerPool) Submit(task Task) {
    p.taskQueue <- task
}

func (p *WorkerPool) Stop() {
    close(p.taskQueue)
    p.wg.Wait()
}

```

2. 批量处理优化：

```

// 批量处理事件
func (h *Handler) ProcessEventsBatch(events []Event) error {
    pool := NewWorkerPool(10, 100)
    pool.Start()
    defer pool.Stop()

    // 分批处理
    batchSize := 100
    for i := 0; i < len(events); i += batchSize {
        end := i + batchSize
        if end > len(events) {
            end = len(events)
        }

        batch := events[i:end]
        pool.Submit(func() error {
            return h.processBatch(batch)
        })
    }

    return nil
}

```

十一、生产环境高频场景题

场景题 1：抵押率突然跌破阈值，如何应急处理？

场景描述：

某天凌晨 3 点，由于 ETH 价格暴跌 15%，Angle Protocol 在以太坊主网的抵押率从 115% 迅速跌至 102%，触发紧急告警。系统面临以下风险：

- 多个借款头寸接近清算线
- 用户可能出现挤兑赎回 USDA
- 协议可能出现资不抵债

作为后端工程师，你需要：

1. 立即响应（0-5分钟）：

- 确认告警是否为真实情况
- 评估风险等级和影响范围
- 通知核心团队成员

2. 紧急措施（5-30分钟）：

- 启动熔断机制，暂停新的借款
- 限制大额赎回（如单笔超过 100万 USDA）
- 加速清算高风险头寸
- 调整收益策略，从 DeFi 协议快速提取流动性

3. 中期应对（30分钟-24小时）：

- DAO 治理提案注入资金
- 调整 PSM 手续费（临时收取 0.5% 赎回费）
- 与做市商协调，稳定 USDA 价格
- 增加清算机器人的 Gas 价格，确保清算及时

技术实现要点：

```
// pkg/emergency/handler.go

type EmergencyHandler struct {
    chainClient    *blockchain.ChainClient
    psmContract    *psm.PSM
    borrowContract *borrow.Borrow
    db             *gorm.DB
    notifier       *alert.Notifier
    logger         *zap.Logger
}

// HandleCollateralRatioCrisis 处理抵押率危机
func (h *EmergencyHandler) HandleCollateralRatioCrisis(chainID uint64, ratio int) error {
    h.logger.Error("Collateral ratio crisis detected",
        zap.Uint64("chain_id", chainID),
        zap.Int("ratio", ratio),
    )

    // 1. 发送紧急告警
    alert := &alert.Alert{
        Level:      "critical",
        Title:      "抵押率危机",
        Message:    fmt.Sprintf("链 %d 抵押率跌至 %d 基点", chainID, ratio),
        Timestamp:  time.Now(),
    }
```



```

        ChainID:      chainID,
        MetricValue: fmt.Sprintf("%d", ratio),
    }
    h.notifier.SendAlert(alert)

    // 2. 根据严重程度执行不同策略
    if ratio < 10200 {
        // 极度危险: 立即暂停铸造和赎回
        return h.executeEmergencyShutdown(chainID)
    } else if ratio < 10500 {
        // 危险: 限制大额操作
        return h.executeLimitedMode(chainID)
    }

    return nil
}

// 执行紧急关闭
func (h *EmergencyHandler) executeEmergencyShutdown(chainID uint64) error {
    h.logger.Error("Executing emergency shutdown", zap.Uint64("chain_id", chainID))

    // 1. 暂停 PSM 铸造
    tx, err := h.psmContract.PauseMint(h.getAuth())
    if err != nil {
        return fmt.Errorf("failed to pause mint: %w", err)
    }

    h.logger.Info("PSM mint paused", zap.String("tx", tx.Hash().Hex()))

    // 2. 限制赎回
    tx, err = h.psmContract.SetRedeemCap(h.getAuth(), big.NewInt(100000e18)) // 限制单笔 10
万
    if err != nil {
        return fmt.Errorf("failed to set redeem cap: %w", err)
    }

    h.logger.Info("Redeem cap set", zap.String("tx", tx.Hash().Hex()))

    // 3. 暂停新借款
    tx, err = h.borrowContract.PauseBorrow(h.getAuth())
    if err != nil {
        return fmt.Errorf("failed to pause borrow: %w", err)
    }

    h.logger.Info("Borrow paused", zap.String("tx", tx.Hash().Hex()))

    // 4. 触发清算所有危险头寸
    if err := h.triggerMassLiquidation(chainID); err != nil {
        h.logger.Error("Failed to trigger mass liquidation", zap.Error(err))
    }

    // 5. 从 DeFi 协议快速提取资金

```

```

    if err := h.withdrawFromDeFi(chainID); err != nil {
        h.logger.Error("Failed to withdraw from DeFi", zap.Error(err))
    }

    return nil
}

// 触发批量清算
func (h *EmergencyHandler) triggerMassLiquidation(chainID uint64) error {
    // 查询所有健康因子 < 1.1 的头寸
    var positions []models.BorrowPosition
    err := h.db.Where("chain_id = ? AND status = 'active' AND health_factor < 1.1",
        chainID,
    ).Find(&positions).Error

    if err != nil {
        return err
    }

    h.logger.Info("Found high-risk positions",
        zap.Int("count", len(positions)),
    )

    // 加入清算队列，提高优先级
    for _, pos := range positions {
        if err := h.queueHighPriorityLiquidation(&pos); err != nil {
            h.logger.Error("Failed to queue liquidation", zap.Error(err))
        }
    }

    return nil
}

```

考察点：

- 紧急情况下的决策能力
- 对协议风控机制的理解
- 多系统协调能力
- 应急预案的设计和执行

场景题 2：清算机器人抢跑被 MEV Bot 攻击，如何优化？

场景描述：

你的清算机器人发现了一个可清算的头寸，计算利润约 5000 USDA。但在发送交易后，发现被 MEV Bot 抢跑了，导致：

- 清算交易失败，损失 Gas 费 \$100
- 一周内被抢跑 30 次，累计损失 \$3000
- 清算成功率从 95% 下降到 60%

问题分析：

MEV (Maximal Extractable Value) 机器人通过以下方式抢跑：

1. 监听 Mempool 中的清算交易
2. 复制交易内容，提高 Gas 价格
3. 通过 Flashbots 等渠道优先打包

优化方案：

方案 1：使用 Flashbots 私有交易池

```
// pkg/liquidation/flashbots.go

type FlashbotsExecutor struct {
    flashbotsRelay string
    signingKey      *ecdsa.PrivateKey
    logger          *zap.Logger
}

func (e *FlashbotsExecutor) SendPrivateTransaction(tx *types.Transaction) error {
    // 1. 签名交易
    signedTx, err := e.signTransaction(tx)
    if err != nil {
        return err
    }

    // 2. 构造 Flashbots Bundle
    bundle := &FlashbotsBundle{
        Transactions: []string{
            hexutil.Encode(signedTx.Data()),
        },
        BlockNumber: e.getTargetBlock(),
    }

    // 3. 签名 Bundle
    signature, err := e.signBundle(bundle)
    if err != nil {
        return err
    }

    // 4. 发送到 Flashbots Relay
    resp, err := e.sendBundle(bundle, signature)
    if err != nil {
        return err
    }

    e.logger.Info("Bundle sent to Flashbots",
        zap.String("bundle_hash", resp.BundleHash),
    )

    return nil
}
```

方案 2：动态 Gas 价格策略

```
// pkg/liquidation/gas_strategy.go

type GasStrategy struct {
    baseline      *big.Int
    maxMultiple float64
}

func (s *GasStrategy) CalculateOptimalGasPrice(
    expectedProfit *big.Int,
    competition int, // 竞争清算者数量
) *big.Int {
    // 1. 获取当前基础 Gas 价格
    baseGas, _ := s.getBaseGasPrice()

    // 2. 根据利润计算最高可接受 Gas 价格
    // 假设愿意用 20% 的利润支付 Gas
    profitFloat, _ := new(big.Float).SetInt(expectedProfit).Float64()
    maxGas := profitFloat * 0.2

    // 3. 根据竞争情况调整
    competitionMultiple := 1.0 + float64(competition)*0.1 // 每个竞争者增加 10%

    gasPrice := new(big.Float).SetInt(baseGas)
    gasPrice.Mul(gasPrice, big.NewFloat(competitionMultiple))

    // 4. 确保不超过最大倍数
    maxAllowed := new(big.Float).SetInt(baseGas)
    maxAllowed.Mul(maxAllowed, big.NewFloat(s.maxMultiple))

    if gasPrice.Cmp(maxAllowed) > 0 {
        gasPrice = maxAllowed
    }

    result, _ := gasPrice.Int(nil)
    return result
}
```

方案 3：使用闪电贷 + 原子性操作

```
// 在一个交易中完成：
// 1. 闪电贷借出 USDA
// 2. 清算头寸
// 3. 卖出抵押品
// 4. 归还闪电贷
// 5. 转移利润

// 这样即使被抢跑，对方也无法在同一区块内重复清算
func (e *LiquidationExecutor) ExecuteAtomicLiquidation(position *BorrowPosition) error {
```

```

// 构造组合调用
calls := []Call{
    // 1. 闪电贷
    {
        Target: flashLoanContract,
        Data:   encodeBorrow(position.BorrowedAmount),
    },
    // 2. 清算
    {
        Target: liquidationContract,
        Data:   encodeLiquidate(position.User),
    },
    // 3. DEX 卖出
    {
        Target: dexRouter,
        Data:   encodeSwap(collateral, usda),
    },
    // 4. 还款
    {
        Target: flashLoanContract,
        Data:   encodeRepay(position.BorrowedAmount),
    },
}

// 通过多重调用合约执行
return e.multicall.Execute(calls)
}

```

考察点：

- 对 MEV 的理解
- Gas 优化策略
- 闪电贷的使用
- 私有交易池的应用

场景题 3：数据库主库宕机，如何实现高可用切换？

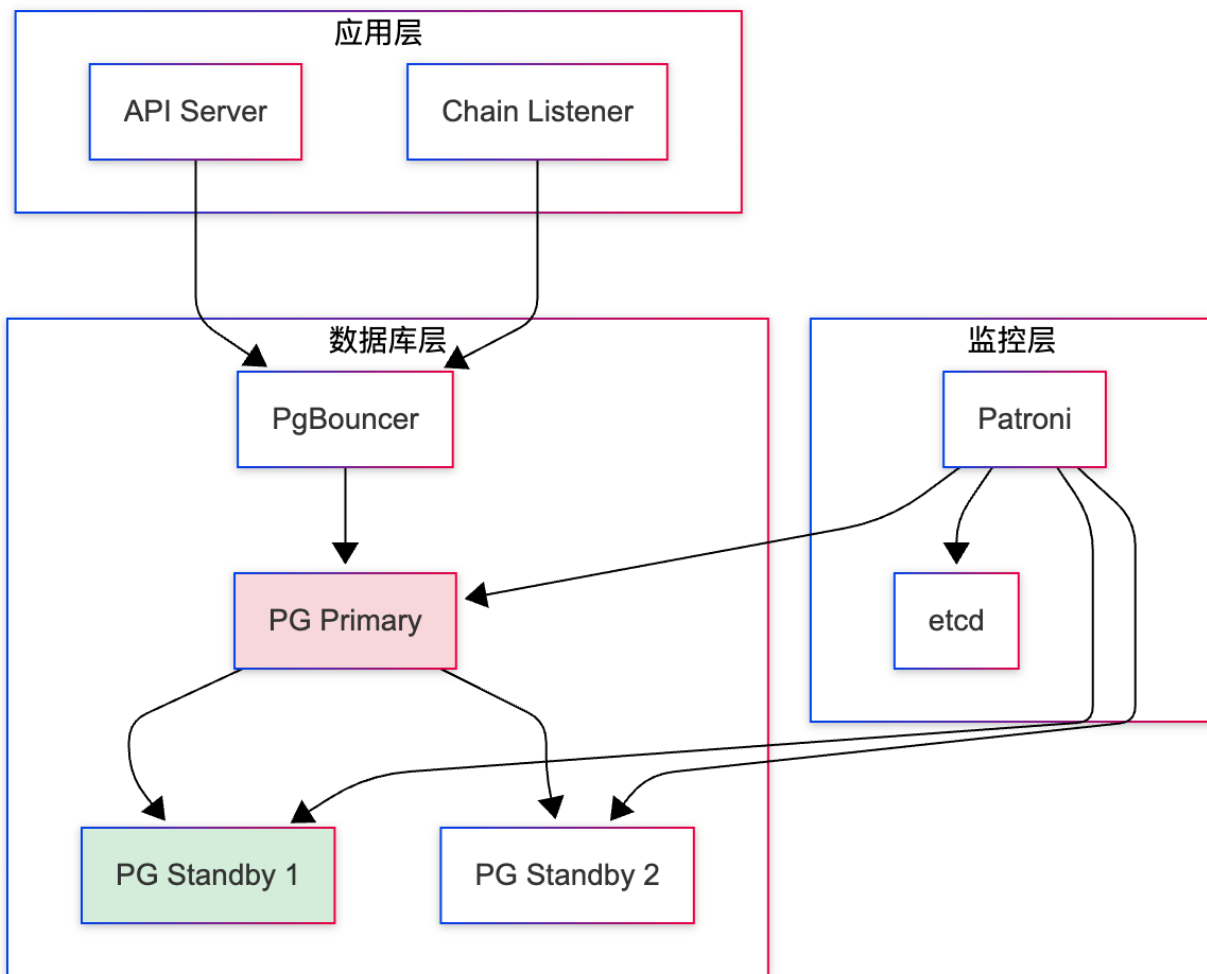
场景描述：

凌晨 2 点，PostgreSQL 主库突然宕机，监控显示：

- 主库无法连接
- 从库数据延迟 30 秒
- API 请求开始报错
- 事件监听服务停止写入

高可用切换方案：

架构设计：



实现代码:

```
// pkg/database/ha.go

type DatabaseCluster struct {
    primary    *gorm.DB
    standbys   []*gorm.DB
    etcdClient *clientv3.Client
    logger     *zap.Logger

    currentPrimary string
    mu              sync.RWMutex
}

func NewDatabaseCluster(config ClusterConfig) (*DatabaseCluster, error) {
    cluster := &DatabaseCluster{
        etcdClient: config.EtcdClient,
        logger:     config.Logger,
    }

    // 从 etcd 获取当前主库
    primary, err := cluster.getPrimaryFromEtcd()
```

```

    if err != nil {
        return nil, err
    }

    // 连接主库
    cluster.primary, err = Connect(primary)
    if err != nil {
        return nil, err
    }

    // 连接所有从库
    for _, standby := range config.Standbys {
        db, err := Connect(standby)
        if err != nil {
            cluster.logger.Error("Failed to connect standby", zap.Error(err))
            continue
        }
        cluster.standbys = append(cluster.standbys, db)
    }

    // 启动健康检查
    go cluster.healthCheck()

    // 监听 etcd 变更
    go cluster.watchPrimaryChange()

    return cluster, nil
}

// 健康检查
func (c *DatabaseCluster) healthCheck() {
    ticker := time.NewTicker(5 * time.Second)
    defer ticker.Stop()

    for range ticker.C {
        if err := c.checkPrimaryHealth(); err != nil {
            c.logger.Error("Primary health check failed", zap.Error(err))

            // 触发故障转移
            if err := c.failover(); err != nil {
                c.logger.Error("Failover failed", zap.Error(err))
            }
        }
    }
}

// 检查主库健康
func (c *DatabaseCluster) checkPrimaryHealth() error {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()

    sqlDB, err := c.primary.DB()

```

```

    if err != nil {
        return err
    }

    return sqlDB.PingContext(ctx)
}

// 故障转移
func (c *DatabaseCluster) failover() error {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.logger.Warn("Starting failover process")

    // 1. 选择最合适的从库作为新主库
    newPrimary, err := c.selectNewPrimary()
    if err != nil {
        return fmt.Errorf("failed to select new primary: %w", err)
    }

    c.logger.Info("New primary selected", zap.String("db", newPrimary))

    // 2. 连接新主库
    db, err := Connect(newPrimary)
    if err != nil {
        return fmt.Errorf("failed to connect new primary: %w", err)
    }

    // 3. 切换连接
    oldPrimary := c.primary
    c.primary = db
    c.currentPrimary = newPrimary

    // 4. 关闭旧主库连接
    if oldPrimary != nil {
        sqlDB, _ := oldPrimary.DB()
        sqlDB.Close()
    }

    // 5. 更新 etcd
    if err := c.updateEtcdPrimary(newPrimary); err != nil {
        c.logger.Error("Failed to update etcd", zap.Error(err))
    }

    // 6. 发送告警
    alert := &alert.Alert{
        Level:    "critical",
        Title:    "数据库故障转移",
        Message:  fmt.Sprintf("主库已切换到 %s", newPrimary),
        Timestamp: time.Now(),
    }
    c.notifier.SendAlert(alert)
}

```



```

    c.logger.Info("Failover completed successfully")
    return nil
}

// 选择新主库 (选择复制延迟最小的从库)
func (c *DatabaseCluster) selectNewPrimary() (string, error) {
    type candidateInfo struct {
        address string
        lag      time.Duration
    }

    var candidates []candidateInfo

    for i, standby := range c.standbys {
        // 检查从库健康
        if err := c.checkHealth(standby); err != nil {
            c.logger.Warn("Standby unhealthy", zap.Int("index", i))
            continue
        }

        // 获取复制延迟
        lag, err := c.getReplicationLag(standby)
        if err != nil {
            c.logger.Warn("Failed to get replication lag", zap.Int("index", i))
            continue
        }

        candidates = append(candidates, candidateInfo{
            address: c.config.Standbys[i],
            lag:     lag,
        })
    }

    if len(candidates) == 0 {
        return "", fmt.Errorf("no healthy standby available")
    }

    // 选择延迟最小的
    sort.Slice(candidates, func(i, j int) bool {
        return candidates[i].lag < candidates[j].lag
    })

    return candidates[0].address, nil
}

// 获取复制延迟
func (c *DatabaseCluster) getReplicationLag(db *gorm.DB) (time.Duration, error) {
    var lag float64
    err := db.Raw(`
        SELECT EXTRACT(EPOCH FROM (now() - pg_last_xact_replay_timestamp()))
    `).Scan(&lag).Error

```

```

    if err != nil {
        return 0, err
    }

    return time.Duration(lag * float64(time.Second)), nil
}

// GetDB 获取数据库连接（读写分离）
func (c *DatabaseCluster) GetDB(readonly bool) *gorm.DB {
    c.mu.RLock()
    defer c.mu.RUnlock()

    if readonly && len(c.standbys) > 0 {
        // 读请求：随机选择一个从库
        idx := rand.Intn(len(c.standbys))
        return c.standbys[idx]
    }

    // 写请求：使用主库
    return c.primary
}

```

考察点：

- 高可用架构设计
- 故障检测和自动切换
- 数据一致性保证
- 读写分离

场景题 4：Redis 缓存穿透导致数据库压力过大

场景描述：

用户端发起大量查询不存在的地址余额请求：

- 每秒 10000+ 请求查询随机地址
- Redis 缓存全部未命中
- 数据库 CPU 飙升至 90%
- API 响应时间从 100ms 上升到 5s

解决方案：

1. 布隆过滤器拦截无效请求：

```

// pkg/cache/bloom_filter.go

type BloomFilter struct {
    redis *redis.Client
    key    string
    size   uint64
}

```

```

    hashes int
}

func NewBloomFilter(redis *redis.Client, key string) *BloomFilter {
    return &BloomFilter{
        redis:  redis,
        key:    key,
        size:   100000000, // 1000万
        hashes: 7,
    }
}

// Add 添加元素到布隆过滤器
func (bf *BloomFilter) Add(item string) error {
    positions := bf.getPositions(item)

    pipe := bf.redis.Pipeline()
    for _, pos := range positions {
        pipe.SetBit(context.Background(), bf.key, int64(pos), 1)
    }

    _, err := pipe.Exec(context.Background())
    return err
}

// Exists 检查元素是否可能存在
func (bf *BloomFilter) Exists(item string) (bool, error) {
    positions := bf.getPositions(item)

    pipe := bf.redis.Pipeline()
    cmds := make([]*redis.IntCmd, len(positions))

    for i, pos := range positions {
        cmds[i] = pipe.GetBit(context.Background(), bf.key, int64(pos))
    }

    _, err := pipe.Exec(context.Background())
    if err != nil {
        return false, err
    }

    for _, cmd := range cmds {
        if cmd.Val() == 0 {
            return false, nil
        }
    }

    return true, nil
}

func (bf *BloomFilter) getPositions(item string) []uint64 {
    positions := make([]uint64, bf.hashes)

```

```

h := fnv.New64a()
h.Write([]byte(item))
hash := h.Sum64()

for i := 0; i < bf.hashes; i++ {
    positions[i] = (hash + uint64(i)*hash) % bf.size
}

return positions
}

// 使用示例
func (h *Handler) GetUserBalance(address string) (*Balance, error) {
    // 1. 检查布隆过滤器
    exists, err := h.bloomFilter.Exists(address)
    if err != nil {
        return nil, err
    }

    if !exists {
        // 地址不存在, 直接返回
        return &Balance{Balance: "0"}, nil
    }

    // 2. 查询缓存
    cacheKey := "balance:" + address
    cached, err := h.redis.Get(context.Background(), cacheKey).Result()
    if err == nil {
        var balance Balance
        json.Unmarshal([]byte(cached), &balance)
        return &balance, nil
    }

    // 3. 查询数据库
    var balance Balance
    err = h.db.Where("user_address = ?", address).First(&balance).Error
    if err != nil {
        if err == gorm.ErrRecordNotFound {
            // 缓存空结果, 防止穿透
            h.redis.Set(context.Background(), cacheKey, "{}", 5*time.Minute)
            return &Balance{Balance: "0"}, nil
        }
        return nil, err
    }

    // 4. 写入缓存
    data, _ := json.Marshal(balance)
    h.redis.Set(context.Background(), cacheKey, data, 10*time.Minute)

    return &balance, nil
}

```

2. 限流中间件:

```
// pkg/middleware/advanced_ratelimit.go

type AdaptiveRateLimiter struct {
    redis      *redis.Client
    baseRate   int
    burstSize  int
}

// 基于令牌桶算法的自适应限流
func (l *AdaptiveRateLimiter) Allow(key string) (bool, error) {
    now := time.Now().Unix()

    // Lua 脚本实现原子性操作
    script := `
        local key = KEYS[1]
        local rate = tonumber(ARGV[1])
        local burst = tonumber(ARGV[2])
        local now = tonumber(ARGV[3])

        local tokens_key = key .. ":tokens"
        local timestamp_key = key .. ":timestamp"

        local last_tokens = tonumber(redis.call("get", tokens_key))
        if last_tokens == nil then
            last_tokens = burst
        end

        local last_time = tonumber(redis.call("get", timestamp_key))
        if last_time == nil then
            last_time = now
        end

        local delta = now - last_time
        local tokens = math.min(burst, last_tokens + delta * rate)

        if tokens < 1 then
            return 0
        end

        redis.call("setex", tokens_key, 3600, tokens - 1)
        redis.call("setex", timestamp_key, 3600, now)

        return 1
    `

    result, err := l.redis.Eval(
        context.Background(),
        script,
        []string{key},
        l.baseRate,
    )
}
```

```

        l.burstSize,
        now,
    ).Int()

    return result == 1, err
}

```

考察点：

- 缓存穿透的理解
- 布隆过滤器的应用
- 限流算法
- 系统保护机制

场景题 5：跨链消息丢失，如何保证数据一致性？

场景描述：

用户从以太坊跨链转移 10000 USDA 到 Polygon：

- 以太坊上 USDA 已销毁
- LayerZero 消息未到达 Polygon
- 用户资产凭空消失
- 客服收到投诉

解决方案：

1. 跨链消息追踪系统：

```

// pkg/bridge/tracker.go

type CrossChainTracker struct {
    db          *gorm.DB
    chainClients map[uint64]*blockchain.ChainClient
    layerzero    *layerzero.Client
    logger       *zap.Logger
}

type CrossChainTransfer struct {
    ID                uint64    `gorm:"primaryKey"`
    SourceChainID     uint64    `gorm:"index"`
    DestChainID       uint64    `gorm:"index"`
    User              string     `gorm:"index"`
    Amount            string
    SourceTxHash      string     `gorm:"uniqueIndex"`
    DestTxHash        string
    LayerZeroNonce    uint64
    Status            string     // pending, confirmed, failed
    CreatedAt         time.Time
    ConfirmedAt       *time.Time
}

```

```

// 监控跨链转账
func (t *CrossChainTracker) TrackTransfer(transfer *CrossChainTransfer) error {
    // 1. 保存转账记录
    if err := t.db.Create(transfer).Error; err != nil {
        return err
    }

    // 2. 启动后台监控
    go t.monitorTransfer(transfer)

    return nil
}

func (t *CrossChainTracker) monitorTransfer(transfer *CrossChainTransfer) {
    timeout := time.After(1 * time.Hour)
    ticker := time.NewTicker(30 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            // 检查目标链是否收到
            confirmed, destTxHash, err := t.checkDestChain(transfer)
            if err != nil {
                t.logger.Error("Failed to check dest chain", zap.Error(err))
                continue
            }

            if confirmed {
                // 更新状态为已确认
                transfer.Status = "confirmed"
                transfer.DestTxHash = destTxHash
                now := time.Now()
                transfer.ConfirmedAt = &now

                t.db.Save(transfer)

                t.logger.Info("Transfer confirmed",
                    zap.String("source_tx", transfer.SourceTxHash),
                    zap.String("dest_tx", destTxHash),
                )
                return
            }

        case <-timeout:
            // 超时，标记为失败
            transfer.Status = "failed"
            t.db.Save(transfer)

            // 发送告警
            t.sendAlert(transfer)
        }
    }
}

```

```

        // 触发补偿流程
        t.triggerCompensation(transfer)
        return
    }
}

func (t *CrossChainTracker) checkDestChain(transfer *CrossChainTransfer) (bool, string,
error) {
    destClient := t.chainClients[transfer.DestChainID]

    // 查询 LayerZero 消息状态
    delivered, err := t.layerzero.IsMessageDelivered(
        transfer.SourceChainID,
        transfer.DestChainID,
        transfer.LayerZeroNonce,
    )

    if err != nil {
        return false, "", err
    }

    if !delivered {
        return false, "", nil
    }

    // 查询目标链的 Mint 事件
    filter := &bind.FilterOpts{
        Start: transfer.CreatedAt.Unix() - 300, // 前后 5 分钟
        End:   time.Now().Unix() + 300,
    }

    iter, err := destClient.USDAContract().FilterBridgeMint(
        filter,
        []common.Address{common.HexToAddress(transfer.User)},
    )

    if err != nil {
        return false, "", err
    }
    defer iter.Close()

    for iter.Next() {
        event := iter.Event

        // 检查金额是否匹配
        if event.Amount.String() == transfer.Amount {
            return true, event.Raw.TxHash.Hex(), nil
        }
    }
}

```



```

    return false, "", nil
}

// 触发补偿流程
func (t *CrossChainTracker) triggerCompensation(transfer *CrossChainTransfer) {
    t.logger.Error("Triggering compensation",
        zap.String("source_tx", transfer.SourceTxHash),
        zap.String("user", transfer.User),
        zap.String("amount", transfer.Amount),
    )

    // 创建补偿任务
    compensation := &models.Compensation{
        TransferID:    transfer.ID,
        User:          transfer.User,
        Amount:        transfer.Amount,
        SourceChainID: transfer.SourceChainID,
        DestChainID:   transfer.DestChainID,
        Status:        "pending",
        CreatedAt:     time.Now(),
    }

    if err := t.db.Create(compensation).Error; err != nil {
        t.logger.Error("Failed to create compensation", zap.Error(err))
        return
    }

    // 发送到人工审核队列
    t.sendToManualReview(compensation)
}

```

2. 自动重试机制:

```

// LayerZero 消息重试
func (t *CrossChainTracker) retryFailedTransfers() {
    var failedTransfers []CrossChainTransfer

    // 查询失败的转账 (24小时内)
    t.db.Where("status = 'pending' AND created_at > ?",
        time.Now().Add(-24*time.Hour),
    ).Find(&failedTransfers)

    for _, transfer := range failedTransfers {
        // 手动触发 LayerZero 重试
        if err := t.layerzero.RetryMessage(
            transfer.SourceChainID,
            transfer.DestChainID,
            transfer.LayerZeroNonce,
        ); err != nil {
            t.logger.Error("Failed to retry message", zap.Error(err))
            continue
        }
    }
}

```

```

    }

    t.logger.Info("Message retried",
        zap.String("source_tx", transfer.SourceTxHash),
    )
}
}

```

考察点：

- 跨链消息可靠性
- 分布式系统一致性
- 补偿机制设计
- 异常监控和告警

场景题 6：RPC 节点不稳定，如何保证服务可用性？

场景描述：

你依赖的 Infura RPC 节点频繁出现问题：

- 请求超时率 10%
- 返回错误的区块数据
- 突然断开连接
- 影响事件监听和交易发送

解决方案：多 RPC 节点容错架构

```

// pkg/blockchain/multi_rpc.go

type MultiRPCClient struct {
    providers []RPCProvider
    current   int
    mu        sync.RWMutex
    logger    *zap.Logger

    // 健康度评分
    healthScores map[string]*HealthScore
}

type RPCProvider struct {
    Name        string
    Endpoint    string
    Priority     int
    Client      *ethclient.Client
}

type HealthScore struct {
    SuccessCount int
    FailureCount int
    AvgLatency   time.Duration
    LastCheck    time.Time
}

```

```

}

func NewMultiRPCClient(config []RPCConfig, logger *zap.Logger) (*MultiRPCClient, error) {
    client := &MultiRPCClient{
        providers:    make([]RPCProvider, 0),
        healthScores: make(map[string]*HealthScore),
        logger:       logger,
    }

    // 连接所有 RPC 节点
    for _, cfg := range config {
        ethClient, err := ethclient.Dial(cfg.Endpoint)
        if err != nil {
            logger.Warn("Failed to connect to RPC",
                zap.String("name", cfg.Name),
                zap.Error(err),
            )
            continue
        }

        provider := RPCProvider{
            Name:      cfg.Name,
            Endpoint:  cfg.Endpoint,
            Priority:  cfg.Priority,
            Client:    ethClient,
        }

        client.providers = append(client.providers, provider)
        client.healthScores[cfg.Name] = &HealthScore{}
    }

    if len(client.providers) == 0 {
        return nil, fmt.Errorf("no available RPC provider")
    }

    // 启动健康检查
    go client.healthCheck()

    return client, nil
}

// 智能选择最佳 RPC
func (c *MultiRPCClient) GetBestProvider() *RPCProvider {
    c.mu.RLock()
    defer c.mu.RUnlock()

    var best *RPCProvider
    var bestScore float64 = -1

    for i := range c.providers {
        provider := &c.providers[i]
        health := c.healthScores[provider.Name]
    }

```

```

        // 计算综合评分
        score := c.calculateScore(provider, health)

        if score > bestScore {
            bestScore = score
            best = provider
        }
    }

    return best
}

func (c *MultiRPCClient) calculateScore(
    provider *RPCProvider,
    health *HealthScore,
) float64 {
    // 成功率
    total := health.SuccessCount + health.FailureCount
    if total == 0 {
        return float64(provider.Priority)
    }

    successRate := float64(health.SuccessCount) / float64(total)

    // 延迟因子 (延迟越低越好)
    latencyFactor := 1.0
    if health.AvgLatency > 0 {
        latencyFactor = 1.0 / (health.AvgLatency.Seconds() + 0.1)
    }

    // 优先级权重
    priorityWeight := float64(provider.Priority) / 10.0

    // 综合评分
    score := successRate*0.5 + latencyFactor*0.3 + priorityWeight*0.2

    return score
}

// 带重试的请求
func (c *MultiRPCClient) BlockByNumber(
    ctx context.Context,
    number *big.Int,
) (*types.Block, error) {
    maxRetries := len(c.providers)

    for retry := 0; retry < maxRetries; retry++ {
        provider := c.GetBestProvider()

        start := time.Now()

```

```

    block, err := provider.Client.BlockByNumber(ctx, number)
    latency := time.Since(start)

    if err == nil {
        // 成功, 更新健康度
        c.recordSuccess(provider.Name, latency)
        return block, nil
    }

    // 失败, 更新健康度
    c.recordFailure(provider.Name)

    c.logger.Warn("RPC request failed, retrying",
        zap.String("provider", provider.Name),
        zap.Int("retry", retry),
        zap.Error(err),
    )

    time.Sleep(time.Duration(retry+1) * 100 * time.Millisecond)
}

return nil, fmt.Errorf("all RPC providers failed")
}

func (c *MultiRPCClient) recordSuccess(name string, latency time.Duration) {
    c.mu.Lock()
    defer c.mu.Unlock()

    health := c.healthScores[name]
    health.SuccessCount++

    // 更新平均延迟 (指数移动平均)
    if health.AvgLatency == 0 {
        health.AvgLatency = latency
    } else {
        health.AvgLatency = (health.AvgLatency*9 + latency) / 10
    }

    health.LastCheck = time.Now()
}

func (c *MultiRPCClient) recordFailure(name string) {
    c.mu.Lock()
    defer c.mu.Unlock()

    health := c.healthScores[name]
    health.FailureCount++
    health.LastCheck = time.Now()
}

// 健康检查
func (c *MultiRPCClient) healthCheck() {

```

```

ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()

for range ticker.C {
    for _, provider := range c.providers {
        go func(p RPCProvider) {
            ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
            defer cancel()

            start := time.Now()
            _, err := p.Client.BlockNumber(ctx)
            latency := time.Since(start)

            if err != nil {
                c.recordFailure(p.Name)
                c.logger.Warn("Health check failed",
                    zap.String("provider", p.Name),
                    zap.Error(err),
                )
            } else {
                c.recordSuccess(p.Name, latency)
            }
        }(provider)
    }

    // 每小时重置计数器, 避免历史数据影响
    c.resetCounters()
}

func (c *MultiRPCClient) resetCounters() {
    c.mu.Lock()
    defer c.mu.Unlock()

    now := time.Now()
    for name, health := range c.healthScores {
        if now.Sub(health.LastCheck) > 1*time.Hour {
            c.healthScores[name] = &HealthScore{}
        }
    }
}

```

配置示例:

```

# config.yaml
rpc_providers:
- name: "Infura"
  endpoint: "https://mainnet.infura.io/v3/YOUR_KEY"
  priority: 8
- name: "Alchemy"
  endpoint: "https://eth-mainnet.alchemyapi.io/v2/YOUR_KEY"

```

```
priority: 9
- name: "QuickNode"
  endpoint: "https://YOUR_ENDPOINT.quiknode.pro/YOUR_KEY"
  priority: 7
- name: "Self-Hosted"
  endpoint: "http://your-node:8545"
  priority: 10
```

考察点:

- 多 RPC 容错设计
- 健康度评分算法
- 智能路由选择
- 自动故障切换

场景题 7: Gas 价格暴涨, 清算成本过高

场景描述:

网络拥堵期间:

- Gas Price 从 50 Gwei 涨到 500 Gwei
- 清算一笔需要 \$500 Gas 费
- 预期利润只有 \$300
- 清算变成亏本买卖

解决方案: 动态清算策略

```
// pkg/liquidation/dynamic_strategy.go

type DynamicLiquidationStrategy struct {
    minProfitRatio    float64 // 最小利润率
    maxGasPrice       *big.Int
    deferQueue        *DeferQueue
    batchProcessor    *BatchProcessor
    logger            *zap.Logger
}

// 评估清算可行性
func (s *DynamicLiquidationStrategy) ShouldLiquidate(
    position *BorrowPosition,
    gasPrice *big.Int,
) (bool, string) {

    // 1. 计算预期利润
    expectedProfit, err := s.calculateProfit(position)
    if err != nil {
        return false, "failed to calculate profit"
    }

    // 2. 计算 Gas 成本
```

```

gasCost := s.estimateGasCost(gasPrice)

// 3. 计算利润率
profitRatio := new(big.Float).Quo(
    new(big.Float).SetInt(expectedProfit),
    new(big.Float).SetInt(gasCost),
)

profitRatioFloat, _ := profitRatio.Float64()

s.logger.Info("Liquidation feasibility check",
    zap.String("user", position.User.Hex()),
    zap.String("profit", expectedProfit.String()),
    zap.String("gas_cost", gasCost.String()),
    zap.Float64("profit_ratio", profitRatioFloat),
)

// 4. 判断是否执行
if profitRatioFloat < s.minProfitRatio {
    // 利润率不足, 延迟清算
    s.logger.Info("Profit ratio too low, deferring liquidation",
        zap.Float64("ratio", profitRatioFloat),
        zap.Float64("min_ratio", s.minProfitRatio),
    )

    s.deferLiquidation(position)
    return false, "profit_ratio_too_low"
}

// 5. 检查 Gas 价格上限
if gasPrice.Cmp(s.maxGasPrice) > 0 {
    // Gas 价格过高, 延迟清算
    s.logger.Info("Gas price too high, deferring liquidation",
        zap.String("gas_price", gasPrice.String()),
        zap.String("max_gas", s.maxGasPrice.String()),
    )

    s.deferLiquidation(position)
    return false, "gas_price_too_high"
}

// 6. 检查是否可以批量清算
if s.canBatchLiquidate(position) {
    s.logger.Info("Adding to batch liquidation queue")
    s.batchProcessor.Add(position)
    return false, "added_to_batch"
}

return true, "ok"
}

// 延迟清算队列

```



```

type DeferQueue struct {
    positions []*BorrowPosition
    mu        sync.Mutex
}

func (s *DynamicLiquidationStrategy) deferLiquidation(position *BorrowPosition) {
    s.deferQueue.Add(position)
}

// 定期检查延迟队列
func (s *DynamicLiquidationStrategy) ProcessDeferredQueue() {
    ticker := time.NewTicker(5 * time.Minute)
    defer ticker.Stop()

    for range ticker.C {
        gasPrice, err := s.chainClient.SuggestGasPrice()
        if err != nil {
            s.logger.Error("Failed to get gas price", zap.Error(err))
            continue
        }

        // Gas 价格下降, 重新评估延迟的清算
        if gasPrice.Cmp(s.maxGasPrice) <= 0 {
            s.logger.Info("Gas price acceptable, processing deferred liquidations",
                zap.String("gas_price", gasPrice.String()),
            )

            positions := s.deferQueue.GetAll()
            for _, pos := range positions {
                shouldLiquidate, reason := s.ShouldLiquidate(pos, gasPrice)
                if shouldLiquidate {
                    s.executor.Execute(pos, gasPrice)
                } else {
                    s.logger.Info("Still not feasible",
                        zap.String("reason", reason),
                    )
                }
            }
        }
    }
}

// 批量清算处理器
type BatchProcessor struct {
    positions []*BorrowPosition
    mu        sync.Mutex
    batchSize int
    maxWait   time.Duration
    lastBatch time.Time
}

func (bp *BatchProcessor) Add(position *BorrowPosition) {

```

```

bp.mu.Lock()
defer bp.mu.Unlock()

bp.positions = append(bp.positions, position)

// 达到批量大小或等待超时, 执行批量清算
if len(bp.positions) >= bp.batchSize ||
    time.Since(bp.lastBatch) > bp.maxWait {
    go bp.executeBatch()
}
}

func (bp *BatchProcessor) executeBatch() {
    bp.mu.Lock()
    positions := bp.positions
    bp.positions = []*BorrowPosition{}
    bp.lastBatch = time.Now()
    bp.mu.Unlock()

    if len(positions) == 0 {
        return
    }

    logger.Info("Executing batch liquidation",
        zap.Int("count", len(positions)),
    )

    // 构造批量清算交易
    tx, err := bp.buildBatchLiquidationTx(positions)
    if err != nil {
        logger.Error("Failed to build batch tx", zap.Error(err))
        return
    }

    // 发送交易
    txHash, err := bp.chainClient.SendTransaction(tx)
    if err != nil {
        logger.Error("Failed to send batch tx", zap.Error(err))
        return
    }

    logger.Info("Batch liquidation tx sent",
        zap.String("tx_hash", txHash.Hex()),
        zap.Int("positions", len(positions)),
    )
}

// 构造批量清算交易
func (bp *BatchProcessor) buildBatchLiquidationTx(
    positions []*BorrowPosition,
) (*types.Transaction, error) {

```

```

// 编码多个清算调用
var calls []MultiCall

for _, pos := range positions {
    data, err := bp.liquidationContract.ABI.Pack(
        "liquidate",
        pos.User,
        pos.BorrowedAmount,
    )
    if err != nil {
        return nil, err
    }

    calls = append(calls, MultiCall{
        Target: bp.liquidationContract.Address,
        Data:    data,
    })
}

// 通过 Multicall 合约批量执行
multiCallData, err := bp.multicallContract.ABI.Pack(
    "aggregate",
    calls,
)
if err != nil {
    return nil, err
}

// 构造交易
nonce, _ := bp.chainClient.PendingNonceAt(bp.executorAddress)
gasPrice, _ := bp.chainClient.SuggestGasPrice()

tx := types.NewTransaction(
    nonce,
    bp.multicallContract.Address,
    big.NewInt(0),
    uint64(len(positions))*500000, // 估算 Gas
    gasPrice,
    multiCallData,
)

return tx, nil
}

```

考察点：

- Gas 价格感知的策略
- 批量处理优化
- 延迟执行队列
- 成本效益分析

场景题 8：时序数据库查询性能问题

场景描述：

Grafana 仪表板查询 30 天的历史数据时：

- 查询耗时 30 秒
- 返回 100 万条记录
- 浏览器卡死
- TimescaleDB CPU 100%

解决方案：时序数据优化

-- 1. 使用连续聚合 (Continuous Aggregates)

```
CREATE MATERIALIZED VIEW price_history_hourly
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', time) AS bucket,
    asset,
    source,
    AVG(price) as avg_price,
    MAX(price) as max_price,
    MIN(price) as min_price,
    STDDEV(price) as stddev_price,
    COUNT(*) as sample_count
FROM price_history
GROUP BY bucket, asset, source
WITH NO DATA;
```

-- 启用自动刷新

```
SELECT add_continuous_aggregate_policy('price_history_hourly',
    start_offset => INTERVAL '2 hours',
    end_offset => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour');
```

-- 2. 创建日聚合视图

```
CREATE MATERIALIZED VIEW price_history_daily
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 day', time) AS bucket,
    asset,
    AVG(price) as avg_price,
    MAX(price) as max_price,
    MIN(price) as min_price
FROM price_history
GROUP BY bucket, asset
WITH NO DATA;
```

-- 3. 数据保留策略

```
SELECT add_retention_policy('price_history', INTERVAL '90 days');
SELECT add_retention_policy('price_history_hourly', INTERVAL '1 year');
SELECT add_retention_policy('price_history_daily', INTERVAL '5 years');
```

API 查询优化:

```
// pkg/api/handlers/timeseries.go

func (h *Handler) GetPriceHistory(c *gin.Context) {
    asset := c.Query("asset")
    period := c.Query("period") // 1h, 1d, 7d, 30d

    // 根据时间范围选择合适的聚合级别
    var table string
    var interval string

    switch period {
    case "1h", "6h":
        table = "price_history"
        interval = "1 minute"
    case "1d", "7d":
        table = "price_history_hourly"
        interval = "1 hour"
    case "30d", "90d":
        table = "price_history_daily"
        interval = "1 day"
    default:
        c.JSON(http.StatusBadRequest, ErrorResponse{
            Code:    400,
            Message: "Invalid period",
        })
        return
    }

    // 使用聚合表查询
    query := fmt.Sprintf(`
        SELECT
            bucket as time,
            avg_price,
            max_price,
            min_price
        FROM %s
        WHERE asset = $1
            AND bucket >= NOW() - INTERVAL '%s'
        ORDER BY bucket ASC
    `, table, period)

    var results []PricePoint
    if err := h.db.Raw(query, asset).Scan(&results).Error; err != nil {
        c.JSON(http.StatusInternalServerError, ErrorResponse{
            Code:    500,
            Message: "Query failed",
        })
        return
    }
}
```

```
c.JSON(http.StatusOK, Response{
    Code:    0,
    Message: "success",
    Data:    results,
})
}
```

考察点：

- 时序数据库优化
- 连续聚合
- 数据分层存储
- 查询性能优化

场景题 9：恶意用户发起闪电贷攻击

场景描述：

攻击者通过闪电贷执行以下攻击：

1. 借出大量 USDC
2. 大量铸造 USDA，拉低市场价格
3. 在 DEX 上低价买入 USDA
4. 通过 PSM 1:1 赎回 USDC
5. 归还闪电贷，获利

防御方案：

```
// pkg/security/flash_loan_detector.go

type FlashLoanDetector struct {
    db      *gorm.DB
    redis   *redis.Client
    logger  *zap.Logger

    // 检测阈值
    largeAmountThreshold *big.Int
    shortTimeWindow      time.Duration
}

// 检测可疑的闪电贷攻击
func (d *FlashLoanDetector) DetectSuspiciousActivity(
    user common.Address,
    amount *big.Int,
    txType string,
) (bool, string) {

    // 1. 检查是否为大额操作
    if amount.Cmp(d.largeAmountThreshold) < 0 {
        return false, ""
    }
}
```

```

// 2. 检查短时间内的操作频率
key := fmt.Sprintf("user_activity:%s", user.Hex())
count, err := d.redis.Incr(context.Background(), key).Result()
if err == nil {
    d.redis.Expire(context.Background(), key, d.shortTimeWindow)

    if count > 5 {
        // 短时间内操作过于频繁
        d.logger.Warn("High frequency detected",
            zap.String("user", user.Hex()),
            zap.Int64("count", count),
        )
        return true, "high_frequency"
    }
}

// 3. 检查同一区块内的多次操作
blockNumber := d.chainClient.GetCurrentBlock()
blockKey := fmt.Sprintf("block_activity:%d:%s", blockNumber, user.Hex())

blockCount, _ := d.redis.Incr(context.Background(), blockKey).Result()
d.redis.Expire(context.Background(), blockKey, 1*time.Minute)

if blockCount > 3 {
    d.logger.Warn("Multiple operations in same block",
        zap.String("user", user.Hex()),
        zap.Uint64("block", blockNumber),
    )
    return true, "same_block_attack"
}

// 4. 检查铸造后立即赎回的模式
if txType == "mint" {
    // 记录铸造操作
    mintKey := fmt.Sprintf("recent_mint:%s", user.Hex())
    d.redis.Set(context.Background(), mintKey, amount.String(), 5*time.Minute)
} else if txType == "redeem" {
    // 检查是否刚刚铸造
    mintKey := fmt.Sprintf("recent_mint:%s", user.Hex())
    mintAmount, err := d.redis.Get(context.Background(), mintKey).Result()

    if err == nil {
        // 5 分钟内铸造后立即赎回
        d.logger.Warn("Mint-then-redeem pattern detected",
            zap.String("user", user.Hex()),
            zap.String("mint_amount", mintAmount),
            zap.String("redeem_amount", amount.String()),
        )
        return true, "mint_redeem_pattern"
    }
}
}

```

```

// 5. 检查价格操纵
if d.isPriceManipulation(user, amount, txType) {
    return true, "price_manipulation"
}

return false, ""
}

// 价格操纵检测
func (d *FlashLoanDetector) isPriceManipulation(
    user common.Address,
    amount *big.Int,
    txType string,
) bool {

    // 获取当前市场价格
    marketPrice, err := d.getMarketPrice()
    if err != nil {
        return false
    }

    // 获取 PSM 价格 (固定 1:1)
    psmPrice := big.NewFloat(1.0)

    // 计算偏差
    deviation := new(big.Float).Sub(marketPrice, psmPrice)
    deviation.Abs(deviation)

    deviationPercent, _ := new(big.Float).Quo(deviation, psmPrice).Float64()

    // 如果市场价格偏离 1% 以上, 可能存在价格操纵
    if deviationPercent > 0.01 {
        d.logger.Warn("Price deviation detected",
            zap.Float64("market_price", marketPrice),
            zap.Float64("deviation", deviationPercent),
        )
        return true
    }

    return false
}

```

智能合约层面的防护:

```

// contracts/PSM.sol

contract PSM {
    // 单笔操作限额
    uint256 public maxMintPerTx = 1000000e18; // 100 万
    uint256 public maxRedeemPerTx = 1000000e18;
}

```



```

// 单用户每区块操作限制
mapping(address => mapping(uint256 => uint256)) public userBlockMints;
mapping(address => mapping(uint256 => uint256)) public userBlockRedeems;

// 冷静期: 铸造后需要等待 N 个区块才能赎回
uint256 public cooldownBlocks = 5; // 约 1 分钟
mapping(address => uint256) public lastMintBlock;

function mint(uint256 amount) external nonReentrant {
    require(amount <= maxMintPerTx, "Amount exceeds limit");

    // 检查单区块限额
    uint256 blockMinted = userBlockMints[msg.sender][block.number];
    require(blockMinted + amount <= maxMintPerTx * 2, "Block limit exceeded");

    // 执行铸造
    // ...

    // 记录
    userBlockMints[msg.sender][block.number] = blockMinted + amount;
    lastMintBlock[msg.sender] = block.number;

    emit Mint(msg.sender, amount);
}

function redeem(uint256 amount) external nonReentrant {
    require(amount <= maxRedeemPerTx, "Amount exceeds limit");

    // 检查冷静期
    require(
        block.number >= lastMintBlock[msg.sender] + cooldownBlocks,
        "Cooldown period not passed"
    );

    // 检查单区块限额
    uint256 blockRedeemed = userBlockRedeems[msg.sender][block.number];
    require(blockRedeemed + amount <= maxRedeemPerTx * 2, "Block limit exceeded");

    // 执行赎回
    // ...

    // 记录
    userBlockRedeems[msg.sender][block.number] = blockRedeemed + amount;

    emit Redeem(msg.sender, amount);
}
}

```

考察点:

- 闪电贷攻击原理

- 异常行为检测
- 智能合约安全防护
- 限流和冷静期设计

场景题 10：系统大规模重构，如何保证零停机迁移？

场景描述：

需要将系统从单体架构迁移到微服务架构：

- 数据库需要重新设计
- API 接口需要升级
- 不能影响现有用户
- 需要逐步灰度发布

迁移方案：

1. 数据双写策略：

```
// pkg/migration/dual_write.go

type DualWriteService struct {
    oldDB    *gorm.DB
    newDB    *gorm.DB
    logger   *zap.Logger

    // 迁移开关
    enableNewDB    bool
    writeToOld     bool
    writeToNew     bool
    readFromNew    bool
}

func (s *DualWriteService) SaveTransaction(tx *Transaction) error {
    var errors []error

    // 写入旧数据库
    if s.writeToOld {
        if err := s.oldDB.Create(tx).Error; err != nil {
            s.logger.Error("Failed to write to old DB", zap.Error(err))
            errors = append(errors, err)
        }
    }

    // 写入新数据库
    if s.writeToNew {
        newTx := s.transformToNewSchema(tx)
        if err := s.newDB.Create(newTx).Error; err != nil {
            s.logger.Error("Failed to write to new DB", zap.Error(err))
            errors = append(errors, err)
        }
    }
}
```

```

        // 新数据库写入失败不影响主流程
    }
}

if len(errors) > 0 && s.writeToOld {
    return errors[0]
}

return nil
}

func (s *DualWriteService) GetTransaction(txHash string) (*Transaction, error) {
    // 根据开关选择数据源
    if s.readFromNew {
        // 从新数据库读取
        var tx NewTransaction
        err := s.newDB.Where("tx_hash = ?", txHash).First(&tx).Error
        if err != nil {
            // 降级到旧数据库
            s.logger.Warn("Failed to read from new DB, falling back to old",
                zap.Error(err),
            )
            return s.readFromOld(txHash)
        }
        return s.transformToOldSchema(&tx), nil
    }

    // 从旧数据库读取
    return s.readFromOld(txHash)
}

```

2. 灰度发布策略：

```

// pkg/migration/canary.go

type CanaryDeployment struct {
    percentage int // 新版本流量百分比
    mu         sync.RWMutex
}

func (c *CanaryDeployment) RouteRequest(userID string) string {
    c.mu.RLock()
    defer c.mu.RUnlock()

    // 基于用户 ID 的哈希进行分流
    hash := fnv.New32a()
    hash.Write([]byte(userID))
    bucket := hash.Sum32() % 100

    if int(bucket) < c.percentage {
        return "new_version"
    }
}

```

```

    }

    return "old_version"
}

// 逐步增加新版本流量
func (c *CanaryDeployment) IncreaseTraffic(step int) {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.percentage += step
    if c.percentage > 100 {
        c.percentage = 100
    }
}

```

3. 数据一致性校验:

```

// pkg/migration/validator.go

type DataValidator struct {
    oldDB    *gorm.DB
    newDB    *gorm.DB
    logger   *zap.Logger
}

func (v *DataValidator) ValidateData() error {
    // 1. 校验总数
    var oldCount, newCount int64
    v.oldDB.Model(&Transaction{}).Count(&oldCount)
    v.newDB.Model(&NewTransaction{}).Count(&newCount)

    if oldCount != newCount {
        v.logger.Error("Record count mismatch",
            zap.Int64("old", oldCount),
            zap.Int64("new", newCount),
        )
        return fmt.Errorf("count mismatch")
    }

    // 2. 抽样校验数据一致性
    var oldTxns []Transaction
    v.oldDB.Order("RANDOM()").Limit(1000).Find(&oldTxns)

    for _, oldTx := range oldTxns {
        var newTx NewTransaction
        err := v.newDB.Where("tx_hash = ?", oldTx.TxHash).First(&newTx).Error
        if err != nil {
            v.logger.Error("Transaction not found in new DB",
                zap.String("tx_hash", oldTx.TxHash),
            )
        }
    }
}

```

```

        continue
    }

    if !v.compare(&oldTx, &newTx) {
        v.logger.Error("Data mismatch",
            zap.String("tx_hash", oldTx.TxHash),
        )
    }
}

return nil
}

```

4. 回滚策略:

```

// pkg/migration/rollback.go

type RollbackManager struct {
    canary    *CanaryDeployment
    dualWrite *DualWriteService
    logger    *zap.Logger

    errorThreshold float64
}

func (rm *RollbackManager) MonitorAndRollback() {
    ticker := time.NewTicker(1 * time.Minute)
    defer ticker.Stop()

    for range ticker.C {
        // 检查新版本错误率
        errorRate := rm.getNewVersionErrorRate()

        if errorRate > rm.errorThreshold {
            rm.logger.Error("Error rate too high, rolling back",
                zap.Float64("error_rate", errorRate),
            )

            // 立即回滚
            rm.canary.SetPercentage(0)
            rm.dualWrite.SetReadFromNew(false)

            // 发送告警
            rm.sendAlert("Automatic rollback triggered")
        }
    }
}

```

考察点:

- 零停机迁移策略

- 双写和数据同步
- 灰度发布
- 回滚机制